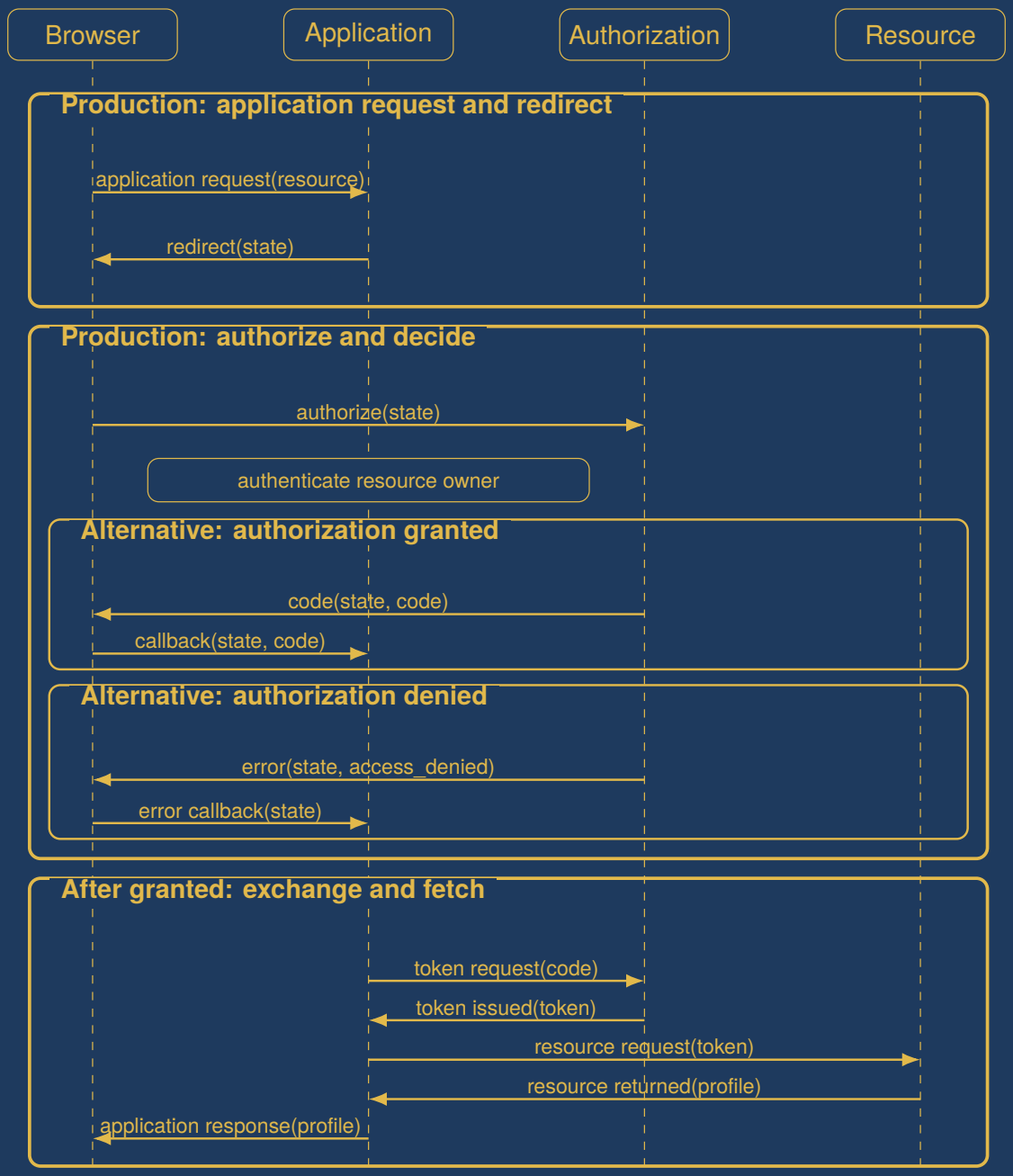
L∃∀NFM Software Engineering

The protocol is the program



Rob Fielding and ChatGPT

Working draft—November 12, 2026

Preface

LeanFM is a programming language whose programs are protocols. A protocol names the roles in a conversation, the messages between them and their order, who decides at each choice, what is assumed about decisions nobody else can see, and what the system promises. Everything else in a LeanFM project is generated from the protocol or checked against it: the diagrams people read, the requirements and implementation mapping reviewed in git, the interfaces, the code each role is written on, and the verdicts on what the running system logs.

Most LeanFM is written by an LLM. A person says in ordinary words what the software must do; the model writes the protocol; LeanFM checks it and shows what it means, as diagrams, obligations, witnesses and counterexamples. The person pushes back, the protocol changes, the pictures change with it, and only then is code written. The model learns the language from LeanFM itself: its server speaks the Model Context Protocol and hands any agent that connects the language reference and the tools.

The language starts from multiparty session types, one global description of a conversation from which each role's obligations are derived [11]. To that it adds stated assumptions on hidden decisions, time and value on messages, promises judged over observation periods, temporal questions answered with paths, and attackers who see every channel.

How to read this book

Part I is the language: what a protocol says (Chapter 1), what it means (Chapter 2), its assumptions and promises (Chapter 3), the questions you can ask it (Chapter 4), and executions as event streams (Chapter 5). Part II is what a protocol generates and how the outputs are reviewed. Part III works examples end to end: a bakery with declared and measured rates, TFTP and its physics, OAuth, a legal settlement, and security protocols. The appendices list the MCP tools and the server's own protocol, and define how a new project is set up.

Every listing, diagram and verdict in Part I comes from LeanFM, through the same functions its tools call; the generator is named beside each. What LeanFM does not yet do is listed in `CRITIQUE.md`. The source is public [6].

Contents

Preface i

I The LeanFM Language 1

1 The Protocol Is the Program 2
1.1 Who writes it . . . 2
1.2 A protocol . . . 2
1.3 Message types . . . 3
1.4 Statements . . . 5
1.5 Only what crosses a boundary . . . 5
1.6 Where the language comes from . . . 6

2 What a Protocol Means 7
2.1 Two rules every protocol must keep . . . 7
2.2 Projection: what each role owes . . . 8
2.3 Calls . . . 9

3 Assumptions and Promises 10
3.1 Assumptions on choices . . . 10
3.2 Time, amounts and rates . . . 11
3.3 Promises . . . 11

4 Questions You Ask a Protocol 15
4.1 The operators . . . 15
4.2 Asking `PlaceOrder` . . . 15
4.3 What an answer costs . . . 16
4.4 Who can know what . . . 16
4.5 Will two versions work together? . . . 16

5 An Execution Is an Event Stream 18
5.1 Events . . . 18
5.2 Conformance . . . 18
5.3 State lives in the stream . . . 20
5.4 Promises on a stream . . . 21
5.5 Streams from the protocol . . . 21

II What a Protocol Generates 22

6 What a Protocol Generates 23
6.1 Diagrams . . . 23
6.2 Code each role is written on . . . 23
6.3 Interfaces . . . 24
6.4 Requirements and implementation, reviewed in git . . . 26
6.5 Building software from a protocol . . . 26

7 Feature Completion as Positive and Negative Proof 28
7.1 The logical shape of a feature . . . 28
7.2 Stability review before security review . . . 29
7.3 Security as forbidden observable behavior . . . 29
7.4 The feature-completion ledger . . . 30
7.5 Worked example: the project Source view . . . 30
7.6 Worked example: chat auto-scroll . . . 31
7.7 When the result is not complete . . . 31

III Worked Examples 32

8 The Bakery: Declared and Measured 33
8.1 The morning, generated . . . 33
8.2 A sale: what can happen . . . 33
8.3 What we expected, and what the corpus shows . . . 34
8.4 The same questions under both overlays . . . 35
8.5 The deadline lesson: a mean hides the spread . . . 35
8.6 Generated runs . . . 36
8.7 Money and waste . . . 36

9 Declared Physics: TFTP and the Sorcerer's Apprentice 38
9.1 The grammar . . . 38
9.2 The physics . . . 39
9.3 The question . . . 39
9.4 Liveness needs fairness . . . 40
9.5 What the classic session-type reading leaves out . . . 40

10 Worked Example: OAuth Authorization Code Flow 42
10.1 Actors, values, and observable events . . . 42
10.2 Production 1: application request and redirect . . . 43
10.3 Production 2: authorize, then grant or deny . . . 44
10.4 Scenario 3: exchange the code and use the token . . . 45
10.5 Reading the scenarios together . . . 45
10.6 The parallel protobuf alphabet . . . 46
10.7 Generate one legal trace and its artifacts . . . 47
10.7.1 Envelope stream and concrete bytes . . . 49
10.7.2 Event graph with stored backpointers . . . 49

10.7.3 Actor projections . . . 50
10.7.4 Event order, queues, and memory . . . 50
10.7.5 Measurements, plots, and verdicts . . . 50
10.8 Interaction diagrams by scenario . . . 50
10.9 Requirements as observable constraints . . . 50
10.9.1 Who knows the password? . . . 52
10.9.2 Byte-level refinement without choosing a transport . . . 52
10.10 Probabilities and charts . . . 53
10.11 Residual state machine . . . 53

11 A Legal Example: Lawful, Right, Known, Required 55
11.1 Three meanings of “legal” . . . 55
11.2 The Model Tenancy Deposit Act and a declared code . . . 55
11.3 Knowledge from the causal past . . . 56
11.4 Violation and reparation . . . 57
11.5 Lawful, right, and the four quadrants . . . 58
11.6 What CTL says, and what it does not . . . 58
11.7 Computing what is owed: Catala . . . 59
11.8 Status . . . 60

12 Protocol Exercises: Roles, Attackers, and Knowledge 61
12.1 Roles compose by sharing events . . . 61
12.2 A transport-neutral byte algebra . . . 62
12.3 Example 1: an on-call handoff . . . 62
12.3.1 Three roles, written separately . . . 63
12.3.2 What composition catches . . . 63
12.4 Example 2: a sealed-bid auction . . . 64
12.4.1 Secrecy holds, and the auction is still broken . . . 65
12.4.2 The fix: bind the bidder into the commitment . . . 65
12.5 Knowledge and agreement as theorems . . . 65
12.5.1 Calculated knowledge and policy inside CTL . . . 66
12.6 Example 3: pairing a speaker by a code on its screen . . . 67
12.7 Where each modality attaches . . . 68
12.8 Needham–Schroeder, checked . . . 70
12.8.1 Two sessions: reflection . . . 71
12.8.2 A defender on the wire . . . 74
12.8.3 Marking secrets . . . 74
12.8.4 Reassembly . . . 75
12.9 What these exercises demand from the library . . . 76

A The MCP Tools 77
A.1 How a consultation travels . . . 79

B The LeanFM Web-Server Protocol 83
B.1 One exchange across four representations . . . 83
B.1.1 Conservative default HTTP projection . . . 84
B.2 The conversion, field by field . . . 84

B.3 The running adapter . . . 85
B.4 The accepted HTTP projection . . . 85
B.5 Accepted message endpoints . . . 87
B.5.1 Canonical byte schema . . . 91
B.6 Complete LeanFM task grammar . . . 94
B.7 Authenticate an individual user . . . 94
B.8 Create an account using a user-supplied provider API key . . . 95
B.9 List, create, open, and delete owned projects . . . 95
B.10 Turn a fresh project conversation into checked requirements and implementation . . . 95
B.11 Select user-funded or platform-funded model access . . . 96
B.12 Conduct one durable project-scoped requirements turn . . . 96
B.13 Compact a project conversation into durable specifications before clearing it . . . 97
B.14 Sketch and promote multimodal requirement artifacts . . . 97
B.15 Inspect current, accepted, superseded, and stale artifacts . . . 97
B.16 Inspect one task's linked interaction, state machine, constraints, and protobuf events 98
B.17 Search the public web, follow supplied HTTP URLs, and return clickable citations . . . 98
B.18 Recall diagram syntax or propose AI-generated detail for one markerboard revision . 99
B.19 The grammar as an executable canvas . . . 99
B.20 What is checked, and what remains . . . 99

C The LeanFM Project Bootstrap Contract 100
C.1 Required project inputs . . . 100
C.2 Conventional generated interface . . . 101
C.3 Required commands and order . . . 101
C.4 Three layers, reviewed in git . . . 101
C.5 Construction is not a runtime dependency . . . 102
C.6 Build, runtime, and publication dependency inventory . . . 102
C.7 Local and server-hosted references . . . 104

Selected Books and Further Reading 105

Bibliography 110

Index 111

Part I

The LeanFM Language

Chapter 1

The Protocol Is the Program

LeanFM is a programming language whose programs are protocols. A protocol names the roles in a conversation, the messages they exchange, the order those messages come in, who decides at each choice, what is assumed about decisions nobody else can see, and what the system promises. Everything else in a LeanFM project is generated from the protocol or checked against it: the diagrams people read, the requirements and implementation mapping reviewed in git, the OpenAPI and protobuf interfaces, the typestate code each role is written on, and the verdicts on what the running system logs.

This book documents that language: how to write it, and what it means.

1.1 Who writes it

Most LeanFM is written by Claude. A person says what the software must do, in ordinary words. Claude writes the protocol, LeanFM checks it, and the person looks at what comes back: a sequence diagram, each role's obligations, a counterexample. They push back, and the protocol changes. Code comes last, written in one pass from the checked protocol.

Claude learns the language from LeanFM itself. The LeanFM server speaks the Model Context Protocol, so an agent that connects to it is handed the language reference and the tools that check, draw, simulate and judge a protocol. MCP is the language's tutorial. Appendix A lists the tools.

1.2 A protocol

Here is a complete program: a client places an order, the shop quotes or says it is sold out, the client pays, and the bank charges the card or declines it. The structures at the top are the message types; the protocol follows them.

```
/-- What the client orders. -/
structure CreateOrder where
  item : String
  quantity : Nat

/-- The shop's price for the order, in cents. -/
structure Quote where
  price : Nat
```

```
/-- The client pays; the card number is a secret, never logged. -/
structure Pay where
  amount : Nat
  card_secr : String

structure Charge where
  amount : Nat
  card_secr : String

structure Receipt where
  amount : Nat

/-- Placing an order: quote, pay, charge the card. -/
protocol PlaceOrder (Client, Shop, Bank) {
  o: CreateOrder from Client to Shop route "POST /orders";
  choice {
    q: Quote from Shop to Client cause o status 201;
    p: Pay from Client to Shop cause q where p.amount = q.price;
    c: Charge from Shop to Bank cause p;
    choice {
      d: Charged from Bank to Shop cause c;
      Receipt from Shop to Client cause d status 200;
    } or {
      x: Declined from Bank to Shop cause c;
      PaymentDeclined from Shop to Client cause x status 402;
    }
  } or {
    SoldOut from Shop to Client cause o status 409;
  }
}
```

LeanFM draws it. This diagram is LeanFM's own rendering of the text above, not a drawing of it; when the text changes, so does the picture.

Each line is one message: a label, a sender and a receiver. The rest is optional. `o:` names the message so later ones can point at it. `cause o` says this message answers `o`. `route` and `status` make the exchange an HTTP call and its response. `where` states a condition on the payloads, and conformance runs it against what the running system logs (Chapter 5): a client that pays 12 against a quote of 10 does not conform.

1.3 Message types

Every message has a type, written as a Lean structure before the protocols. A message carries the structure its label names: `Pay` carries an `amount` and a `card_secr`. A label that names no structure, such as `SoldOut`, carries nothing, and so does an event. A field is a `Nat`, `Int`, `String`, `Bool`, another structure, or a `List` or `Option` of one of those; `x y : Nat` declares two fields, as in Lean, so a move in a game is `structure Play where x y : Nat` and a pass is `structure Pass`. Fields are numbered in the order written, and that number is the field's protobuf number, so new fields go at the end.

The types are what the conditions and the logs are checked against. A `where` condition is typed when the text is read, before anything runs: `p.amount = q.price` compares two declared `Nat` fields; `p.total = q.price` is refused because `Pay` has no `total`, and `p.amount = "ten"` because a

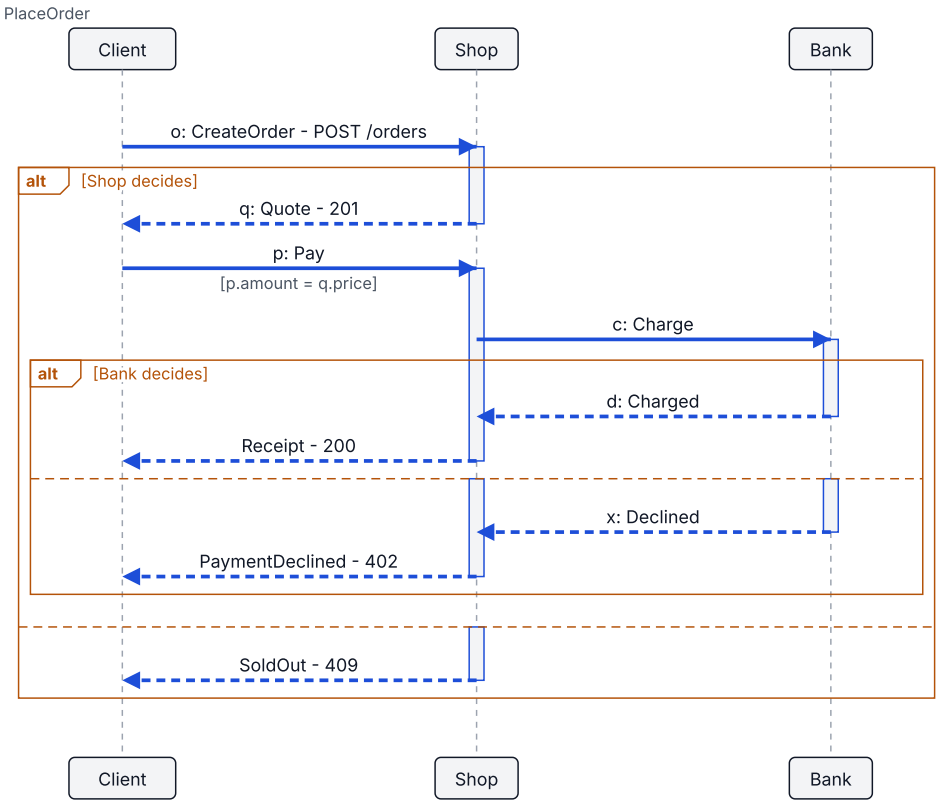


Figure 1.1: `PlaceOrder` as LeanFM draws it. Each `alt` frame names the role that decides; responses are dashed, and each call is an activation bar on its callee.

number is not a string. A name ending in `_secr` is a secret, and no condition may read one, because the fields conditions read are logged. A log is checked against the same types: a field the message does not declare, a value of another type, or a secret in a log is refused.

1.4 Statements

A protocol is a list of statements. These are all of them.

Statement	Meaning
`x: L from A to B cause y route "..." status 201 where e;`	a message; everything after `to B` is optional
`L at A;`	something role `A` does on its own, such as a timer firing
`choice { ...} or { ...}`	exactly one branch happens
`choice at A using declared("N") { probability p: ...}`	the same, with `A`'s decision assumed to have these probabilities (Chapter 3)
`par { ...} and { ...}`	both branches run; what follows waits for all of them
`join 2 of { ...} and { ...} and { ...}`	what follows waits for any two
`rec Loop { ... continue Loop; }`	a loop; `continue` is the last statement of its block
`sessions { ...}`	any number of independent instances, as a server handles requests
`include P A B C;`	an earlier protocol, its roles renamed in order
`require per q { ...}`	what the protocol promises per occurrence of message `q` (Chapter 3)

Statements after a choice continue every branch. Names are Lean identifiers, and `--` and `/- -/` comments are allowed. Several protocols may share a text, so one can include another; the last is the one checked. This is how a large protocol is broken up: each part is its own named protocol, written as if it were the whole conversation, and the top-level protocol assembles them with `include`, renaming roles as it goes. Each part is checked on its own, and the top-level diagram draws each as a frame that names it. A protocol may carry amounts, delays and rates on its messages; those annotations never change its structure, and Chapter 3 covers them.

A plain `choice` does not name who decides, because only one answer can be right: the role that sends the first message of every branch. In `PlaceOrder` the shop decides whether to quote, and the bank whether to charge. If two roles could each start a branch, both could act at once, and LeanFM reports the race (Chapter 2).

The same syntax can be written inside a Lean file with the `protocol` command of `LeanFM.Session`, where a label names a Lean structure and a `where` condition is type-checked Lean. The text form, a `.leanfm` file or the `lean` argument of a tool, is read by a parser for this grammar alone, structures included: nothing in it is elaborated as Lean. It types its `where` conditions itself and runs them with LeanFM's own interpreter, so text that arrives from any client cannot execute code in the server. Anyone who wants more isolation than that can run LeanFM itself in a container or a virtual machine.

1.5 Only what crosses a boundary

A protocol says nothing about what happens inside a role. Whether the shop keeps its stock in a database, a cache or a spreadsheet is left free by not being written down. This is deliberate. A

requirement that depends on a role's internal state is a design decision that leaked out, and checking it costs state space: every variable the checker must track multiplies the states it explores. A TLA+ module of the same order queue with a few cache bits and a clock that cannot affect the order shows the cost (counts from TLC, via `make tla-check`):

Model	Cache bits	Clock bound	Distinct states
Baseline	0	0	26
Cache4	4	0	416
Cache8	8	0	6,656
Clock7	0	7	208

Each irrelevant bit doubles the states, and an unbounded clock makes them infinite [13]. TLA+ can omit such state too; LeanFM starts from the conversation, so there is nowhere to put it. The state that matters lives in the messages, and an execution is the stream of them (Chapter 5).

1.6 Where the language comes from

1. **Session types** give the backbone: one global description of a conversation, projected to each role's obligations, with conditions that rule out choices nobody can implement [9, 11, 22].
2. **Language theory** gives the reading of a protocol as a grammar whose terminals are messages, so the legal conversations are its language. Dana Richards and Henry Hamburger, the author's college professors, are cited in Richards and Hamburger [18] as where the author learned these methods; for symbolization, validity and modal reasoning we also draw on Gensler [7].
3. **Protocol analysis** contributes explicit attackers, adversarial choices and trace semantics. The CSP treatment of Ryan et al. [19], including Steve Schneider, is where the author learned it.
4. **Temporal logic** turns expectations into questions about every possible future, or some [4].

LeanFM adds two things the classic theory leaves out. Decisions nobody else can see carry stated probabilities, and the protocol states what it promises, as minimums and maximums judged over an observation period. Both are Chapter 3.

Chapter 2

What a Protocol Means

A protocol is a grammar. Its terminal symbols are messages, each with a sender and a receiver; its productions are sequence, choice, `par`, `join` and `rec`; the language it generates is the set of legal conversations. A recorded run either is a sentence of that language or it is not. Because every protocol reads as if it were the top-level one, requirements stack: a larger protocol includes smaller ones by name.

This chapter is what LeanFM checks before anything runs: that the protocol can be implemented at all, what each role owes, and which exchanges are calls.

2.1 Two rules every protocol must keep

One role decides each choice. The decider is the role that sends the first message of every branch. When two roles each start a branch, both could act at once and the conversation would leave the protocol:

```
protocol Race (Client, Shop) {
  choice {
    Cancel from Client to Shop;
  } or {
    Ship from Shop to Client;
  }
}
```

```
no single role decides a choice, so it is a race: Client starts Cancel; Shop starts Ship
```

The fix is to pick one decider: the client asks to cancel and the shop answers, or the shop ships and the client receives. In the literature this is the non-local choice, or realizability, condition.

Every role that must know which branch was taken is told. A role whose later behaviour differs between branches has to receive something that tells them apart. Here the bank has taken part, it holds the client's money, and in one branch nobody tells it what happened:

```
protocol Blind (Client, Shop, Bank) {
  h: Hold from Client to Bank;
  choice {
    Capture from Shop to Bank;
    Ship from Shop to Client;
  } or {
```

```
    SoldOut from Shop to Client;
  }
}
```

```
Bank cannot tell which branch Shop chose
```

A role whose part has not started yet owes nothing to a branch that never contacts it. In `PlaceOrder` the bank is never told that the shop sold out, and that is fine: its part starts with `Charge`, which only one branch sends. A service that handles each request on its own, such as a mailer, goes in a nested `sessions` block and never needs to know the conversation around it.

A third rule is about `cause`: a message may only answer something its sender has sent or received. A role decides from its own history, never from something it could not have seen.

2.2 Projection: what each role owes

Projecting the protocol onto a role keeps the messages it sends or receives and turns the rest into its view of the choices: `select` where it decides, `offer` where another role does and the arriving message says which. `PlaceOrder`, projected:

```
-- Client
send CreateOrder to Shop;
offer {
  receive Quote from Shop;
  send Pay to Shop;
  offer {
    receive Receipt from Shop;
  } or {
    receive PaymentDeclined from Shop;
  }
} or {
  receive SoldOut from Shop;
}

-- Shop
receive CreateOrder from Client;
select {
  send Quote to Client;
  receive Pay from Client;
  send Charge to Bank;
  offer {
    receive Charged from Bank;
    send Receipt to Client;
  } or {
    receive Declined from Bank;
    send PaymentDeclined to Client;
  }
} or {
  send SoldOut to Client;
}

-- Bank
offer {
  receive Charge from Shop;
```

```
  select {
    send Charged to Shop;
  } or {
    send Declined to Shop;
  }
} or {
}
```

A local type is an API. Each point in the conversation is a state, and each state allows exactly the moves the protocol allows there. The client's typestate:

```
start : S0
S0.sendCreateOrder(m : CreateOrder) : S1    -- to Shop
S1.receive() : Quote -> S2 (from Shop) | SoldOut -> End (from Shop)
S2.sendPay(m : Pay) : S3    -- to Shop
S3.receive() : Receipt -> End (from Shop) | PaymentDeclined -> End (from Shop)
```

A client in `S1` can only receive; a client in `End` has nothing left to do. Code generated from this (Chapter 6) makes the wrong move a type error.

When the protocol keeps both rules, roles that follow their local types get the classic guarantees: no message arrives that its receiver cannot handle, the run follows the protocol, and no role waits forever [11]. Each holds under the theory's assumption that every message arrives exactly once. Real channels lose, duplicate and reorder, so LeanFM lets a protocol declare each channel's physics beside it; the TFTP chapter (9) shows a protocol that is perfect by the classic rules and has a famous bug once its physics is declared.

2.3 Calls

`cause` also tells LeanFM which messages are calls and which are their returns. A message is the return of the nearest unanswered request the other way in its cause chain; otherwise it is a request. `PlaceOrder`'s calls:

```
o: CreateOrder  Client -> Shop  POST /orders
  returns Quote 201 | SoldOut 409
p: Pay  Client -> Shop
  returns Receipt 200 | PaymentDeclined 402
    c: Charge  Shop -> Bank
      returns Charged | Declined
```

A `route` makes a request an HTTP call, and over HTTP every request gets a response on its own connection. LeanFM checks that, and suggests the line to add when it is missing:

```
protocol Lookup (Browser, Api) {
  q: GetUser from Browser to Api route "GET /users/{id}";
}
```

```
1. [http] GetUser (q) from Browser to Api never gets a response
   try: Answer it: ‘GetUserDone from Api to Browser cause q status 200;‘ (or the status it
    means).
```

A protocol that passes all of this is ready to say more: what is assumed about the decisions nobody else can see, and what is promised.

Chapter 3

Assumptions and Promises

A protocol says what *can* happen. Two more things decide whether a system is good enough: how often each outcome happens, and how long and how much it costs. LeanFM keeps them apart. A probability on a choice is an *assumption*: what a role is expected to decide where nobody else can see. A `require` clause is a *promise*: a minimum or maximum the system keeps, judged over an observation period. LeanFM checks the promises against the assumptions before any code exists, and against the event stream once it runs.

3.1 Assumptions on choices

A choice whose decider's reasons are hidden carries the probabilities it is assumed to have, and a name:

```
/-- A server that is sometimes busy, with one hard deadline and one promised fraction. -/
protocol Ping (Client, Server) {
  q: Ping from Client to Server;
  choice at Server using declared("IsBusy") {
    probability 9/10:
      Pong from Server to Client cause q after 50 ms;
  } or {
    probability 1/10:
      SlowPong from Server to Client cause q after 800 ms;
  }
}

require per q {
  every q reaches Pong, SlowPong within 1 second;
  at least 95/100 of q reach Pong, SlowPong within 100 ms over 1000 entries;
}
```

`choice at Server using declared("IsBusy")` says the server decides, that the probabilities are declared (stated by the author, not measured), and that the decision is called `IsBusy`. The name belongs to the whole choice, like a question; the branches are its answers. Reports name the answer by the first message of the branch taken, so the same choice reads `[IsBusy -> Pong: 9/10]` on one branch and `[IsBusy -> SlowPong: 1/10]` on the other. Probabilities are exact fractions and must sum to one. An overlay can replace them by name without editing the protocol, for example with rates measured from a log (Chapter 8).

3.2 Time, amounts and rates

Messages can take time and move value. None of these annotations changes the protocol's structure: projection, conformance and temporal questions are the same with or without them.

Annotation	Meaning
`after 2 days`, `after sample { 1 days : 1/2; 3 days : 1/2; }`	a delay after the previous step, fixed or drawn from a distribution
`amount 1000 USD`, `amount sample { ...}`	value moved from sender to receiver
`amount { 5 apple; -3 USD; }`	a trade vector: positive entries move forward, negative back
`accrue 10 USD per week from A to B;`	a rate in force until the next `accrue` for the same roles and unit
`at next 1 week`	a periodic gate: wait for the next whole week since the session began
`on breach record Late pays 2000 USD to T;`	in an obligation: the penalty when its deadline passes

Units are names, so `roomweek` (one room for one week) is as good a unit as `USD`. Results are kept per role and per unit.

With its assumptions, a protocol without loops is a decision tree: every path to an end has an exact probability, a time, and each role's balance. LeanFM draws it:

A protocol with loops (`rec`) is solved instead as an absorbing Markov chain. Expected visits v solve $v = s + vQ$ over the transient states, exactly, in rational arithmetic, so a retry loop that runs a geometric number of times has an exact expected cost rather than a truncated one. A session that some state can never leave is reported as one that can loop forever.

3.3 Promises

Promises are written after the protocols, one block per entry message. Every occurrence of the entry is one trial. `Ping`'s block says every ping is answered within a second, and at least 95 in 100 within 100 ms, judged over 1000 pings.

Clause	Meaning
`every q reaches A, B within d;`	a hard deadline: each `q` reaches `A` or `B` within `d`, and one miss is a counterexample
`at least p of q reach A within d over W;`	a promised minimum fraction
`at most p of q reach A over W;`	a promised maximum fraction
`over 1 hour`, `over 500 entries`	the observation period, required for every fraction
`confidence 99/100`	how sure a verdict must be (default 95/100)
`expecting 500 per hour`	the expected traffic, so a time window's size can be checked

A fraction needs an observation period. Without one, the first slow answer after deployment would break a promise about ninety-five answers in a hundred before there was anything to judge it by. A window is judged only once it has closed.

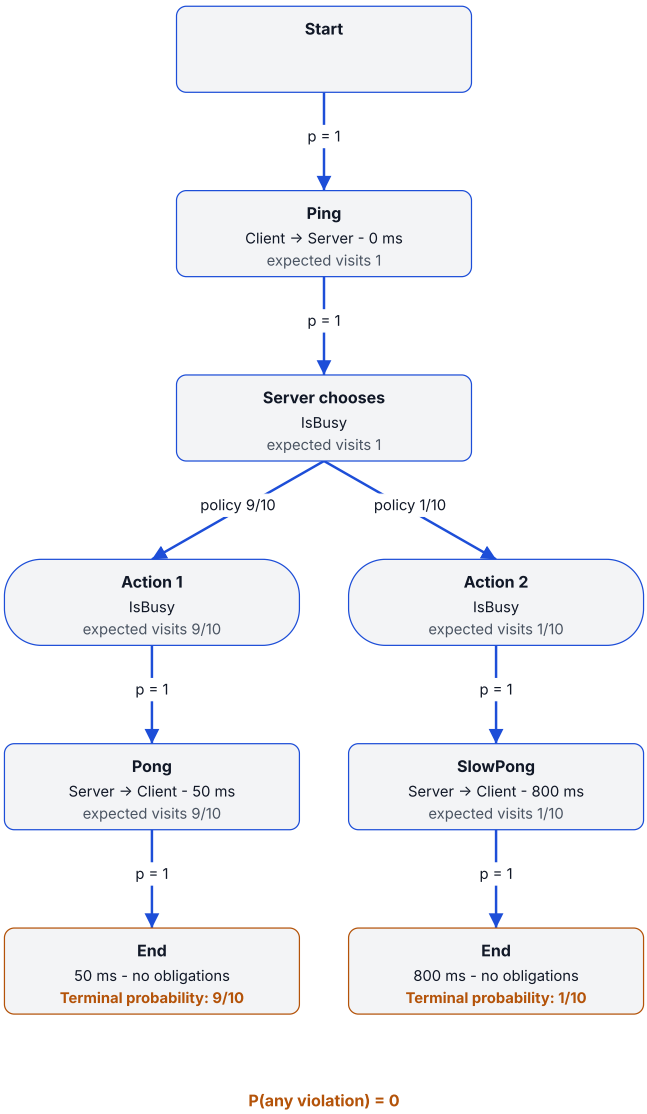


Figure 3.1: `Ping` under its assumption, as LeanFM draws it: the server's choice with its probabilities, each answer's time, and each ending's exact probability.

LeanFM checks each promise against the assumptions on the chain, following every path from each entry with exact probabilities until a target is reached in time (a hit) or the deadline, the next entry or the end comes first (a miss):

```
Requirements against the stated assumptions (exact, per entry):
- every q reaches Pong, SlowPong within 1 second;
  MET: per entry the probability of reaching Pong, SlowPong within 1 second is 1.
- at least 95/100 of q reach Pong, SlowPong within 100 ms over 1000 entries;
  VIOLATED: per entry the probability of reaching Pong, SlowPong within 100 ms is 9/10. The
    most probable path that breaks it: Ping -> [IsBusy -> SlowPong: 1/10] -> SlowPong -> (at
    800 ms, past the 100 ms deadline), through the assumption [IsBusy -> SlowPong: 1/10]. The
    window holds the 60 entries it needs to decide at 19/20 confidence. The assumptions cannot
     meet this bound: change the protocol (a retry, a fallback) or the stated probabilities on
     the path shown.
```

The deadline holds: even the slow answer comes at 800 ms. The fraction fails, because only 9 in 10 answers come within 100 ms, and the report names the assumption responsible. The fix is a faster busy path or a weaker promise. A window needs $\lceil \ln(1/(1-c))/m \rceil$ entries to tell the promise apart from a failure rate of 0 or 1, where m is the margin to the bound; LeanFM reports when a window cannot hold that many.

A larger example, a user lookup through a proxy that can lose a forward and a database that can fail:

```
-- Illustrative assumptions, not measured rates.
-- A user lookup behind a reverse proxy. The probabilities are assumptions: the hidden internal
-- choices of peers (the Db's lookup, the Browser's next request) and of the network (a forward
-- the Proxy loses), as seen from outside. The require block states what every lookup must
-- achieve, per entry of q.
protocol UserLookup (Browser, Proxy, Api, Db) {
  rec Browse {
    q: GetUser from Browser to Proxy;
    choice at Proxy using declared("forward-delivered") {
      probability 999/1000:
        f: Forward from Proxy to Api cause q after 2 ms;
        d: SelectUser from Api to Db cause f after 1 ms;
        choice at Db using declared("lookup") {
          probability 97/100:
            r: Row from Db to Api cause d after sample { 20 ms : 999/1000; 400 ms : 1/1000; };
            Ok200 from Api to Proxy cause r;
            Ok from Proxy to Browser cause q;
        } or {
          probability 2/100:
            r: Empty from Db to Api cause d after 20 ms;
            NotFound404 from Api to Proxy cause r;
            NotFound from Proxy to Browser cause q;
        } or {
          probability 1/100:
            r: Failure from Db to Api cause d after 50 ms;
            Unavailable503 from Api to Proxy cause r;
            Unavailable from Proxy to Browser cause q;
        }
    } or {
      probability 1/1000:
        Dropped at Proxy;
        Timeout504 from Proxy to Browser cause q after 1900 ms;
    }
    choice at Browser using declared("next") {
      probability 4/5:
        Again from Browser to Proxy;
        NextApi from Proxy to Api;
        NextDb from Api to Db;
```

```
        continue Browse;
    } or {
      probability 1/5:
        Quit from Browser to Proxy;
        CloseApi from Proxy to Api;
        CloseDb from Api to Db;
    }
  }
}

require per q {
  every q reaches Ok, NotFound, Unavailable, Timeout504 within 2 seconds;
  at least 98/100 of q reach Ok, NotFound within 300 ms over 1 hour expecting 500 per hour;
  at most 1/200 of q reach Unavailable, Timeout504 over 1 hour expecting 500 per hour;
}
```

```
Requirements against the stated assumptions (exact, per entry):
- every q reaches Ok, NotFound, Unavailable, Timeout504 within 2 seconds;
  MET: per entry the probability of reaching Ok, NotFound, Unavailable, Timeout504 within 2 seconds is 1.
- at least 98/100 of q reach Ok, NotFound within 300 ms over 1 hour expecting 500 per hour;
  MET: per entry the probability of reaching Ok, NotFound within 300 ms is ~0.9880 (exactly
     98804097/100000000). The window holds the 150 entries it needs to decide at 19/20 confidence.
- at most 1/200 of q reach Unavailable, Timeout504 over 1 hour expecting 500 per hour;
  VIOLATED: per entry the probability of reaching Unavailable, Timeout504 is 1099/100000. The most
     probable path that breaks it: GetUser -> [forward-delivered -> Forward: 999/1000] -> Forward ->
     SelectUser -> [lookup -> Failure: 1/100] -> Failure -> Unavailable503 -> Unavailable, through the
     assumptions [forward-delivered -> Forward: 999/1000], [lookup -> Failure: 1/100]. The window will
     not hold the 600 entries it needs to decide at 19/20 confidence: lengthen it or expect more traffic.
      The assumptions cannot meet this bound: change the protocol (a retry, a fallback) or the stated
     probabilities on the path shown.
```

The error budget cannot be met: $\frac{999}{1000} \cdot \frac{1}{100} + \frac{1}{1000} = \frac{1099}{100000}$ is more than $\frac{1}{200}$, and the report names both assumptions on the path. Meeting it is a design change, such as a retry after `Failure`. The window needs 600 entries and an hour at 500 an hour holds only 500, so the promise needs a longer window or more traffic before it can be judged at all.

Chapter 4

Questions You Ask a Protocol

Some rules are visible in a protocol's shape: who decides, who is told, which messages are calls. Others are not. "Never charge a card after saying sold out" and "every charge ends in a receipt" are questions about every possible future of a conversation. LeanFM asks them in CTL, computation tree logic [4], over the protocol's own state graph, and every answer comes with a path: a witness when something is possible, a counterexample when something fails.

4.1 The operators

A CTL operator pairs a path quantifier (**A**, on every path; **E**, on some path) with a time operator (**F**, eventually; **G**, always; **U**, until).

Formula	Read as	Meaning
`EF p`	possibly	some future reaches `p`
`AF p`	inevitably	every future reaches `p`
`AG p`	always	`p` holds now and in every future state
`AG not p`, `never p`	never	no future reaches `p`
`EG p`	can stay	some future keeps `p` forever
`[p AU q]`, `[p EU q]`	until	`p` holds until `q` does, on every path or some
`[p AW q]`, `[p EW q]`	unless	until, without requiring that `q` ever happens

`EF` is "possibly", read forward in time: something can still happen. Its negation is `AG not`: `never p` and `not EF p` are the same question, so "is it possible?" and "can it never happen?" are one question asked from both sides. A counterexample to a `never` is the witness to the `EF`.

The atoms are what an observer can see: `{"after": "Label"}` holds in the state a `Label` message or event has just reached, and `"terminal"` holds where the session has ended. There are no atoms over a role's internal state, because a protocol has none (Chapter 1).

4.2 Asking `PlaceOrder`

Four questions and LeanFM's answers, each with the formula as it is sent:

```
Every session ends.
  {"AF":"terminal"}
```

```
  proved

A receipt is possible.
  {"EF":{"after":"Receipt"}}
  proved: witness: CreateOrder -> Quote -> Pay -> Charge -> Charged -> Receipt

Never charge after sold out.
  {"AG":{"implies":[{"after":"SoldOut"},{"AG":{"not":{"after":"Charge"}}}]}}
  proved

Every charge ends in a receipt.
  {"AG":{"implies":[{"after":"Charge"},{"AF":{"after":"Receipt"}}]}}
  disproved: counterexample: CreateOrder -> Quote -> Pay -> Charge -> Declined ->
    PaymentDeclined (and stays there)
```

The last one fails, and the counterexample says why: a charge can be declined. The rule the author meant was probably “every charge ends in a receipt or a declined payment”, and the counterexample is what makes that visible.

4.3 What an answer costs

Once the state graph exists, each CTL operator is one pass over it: reachability for `EF`, strongly connected components for `EG` and `AF`, a fixpoint for `U`. The cost is about the number of states and transitions times the size of the formula, and breadth-first search returns the shortest witness, which is why the paths read well.

The size of the graph is the real cost. Concurrent branches interleave, a deadline turns each state into a pair of state and elapsed time, and an attacker choosing what to send multiplies every state by its choices. Two steps by different roles that do not interact commute, so interleavings that differ only by swapping them are the same behaviour; the explosion is real only where roles share something. Every search has a bound, and an answer at the bound is `resource-limit`, never `proved`. The server’s `/metrics` reports each check’s time and model size, so the cost is measured, not guessed.

4.4 Who can know what

A security question asks what an attacker can derive. `knows(actor, term)` decides whether an actor can build a term from everything it has seen, taking pairs apart and opening ciphertexts with keys it can build, but never guessing (the Dolev–Yao model). It is a state atom like `after`, so `never knows(Ivo, nb)` is an ordinary CTL question.

Most of what an attacker learns does not matter, so a declaration marks what must stay secret: a name ending in `_secr`, or a term listed in `secrets`. Each mark is the question `never knows(attacker, s)`. Chapter 12 works through Needham–Schroeder, Lowe’s attack, and a password login whose hash leaks when it is sent in clear.

4.5 Will two versions work together?

When a running protocol changes, not every role can be redeployed at once. For each role LeanFM explores the mixed system, the new version of that role among the old versions of the others, to every

reachable state. Adding a branch in which the shop answers `Backorder`:

```
PlaceOrder -> PlaceOrder: upgrade first: Client, Bank
Client cannot stay on the old version: Client waits to receive Quote from Shop, receive
    SoldOut from Shop; Shop is stuck trying to send Backorder to Client (after Client -> Shop:
     CreateOrder; Shop chooses to send Backorder to Client)
Shop cannot upgrade first: Client waits to receive Quote from Shop, receive SoldOut from Shop;
     Shop is stuck trying to send Backorder to Client (after Client -> Shop: CreateOrder; Shop
     chooses to send Backorder to Client)
```

The client and the bank can upgrade first; the shop cannot, because an old client waits for a quote or a sold-out notice and would never accept a backorder.

Chapter 5

An Execution Is an Event Stream

A protocol describes every conversation that may happen; an execution is one that did. LeanFM represents an execution as the stream of its events, and keeps no other state: roles keep none of their own that a requirement may depend on. What a role knows at any moment is the part of the stream it has seen, and every question about a running system is a question about the stream.

5.1 Events

Each event is one message sent, or one thing a role did on its own. It carries its session, its sender and receiver, its label, and the event that caused it. Generated from `PlaceOrder` with its seed 6 , one session:

```
id 0  t=4 ms  CreateOrder: Client -> Shop
id 1  t=5 ms  Quote: Shop -> Client cause 0
id 2  t=6 ms  Pay: Client -> Shop cause 1
id 3  t=7 ms  Charge: Shop -> Bank cause 2
id 4  t=8 ms  Charged: Bank -> Shop cause 3
id 5  t=9 ms  Receipt: Shop -> Client cause 4
```

The cause links form the conversation's spine: each answer points at its question, and the chain from `Receipt` back to `CreateOrder` is the whole call. A real program logs the same shape, one line per message, and the code generated from a protocol logs it automatically (Chapter 6).

5.2 Conformance

A log conforms when every session in it is a sentence of the protocol: each event allowed at its point, naming the cause the protocol names, and each session ending where the protocol can end. Here the shop charges the card before the client has paid:

```
{"from":"Client","id":"e1","label":"CreateOrder","session":"42","to":"Shop"}
{"cause":"e1","from":"Shop","id":"e2","label":"Quote","session":"42","to":"Client"}
{"cause":"e2","from":"Shop","id":"e3","label":"Charge","session":"42","to":"Bank"}
```

```
PlaceOrder: 1 of 1 sessions break the protocol; first: session 42, event e3: Charge from Shop
    to Bank is not allowed here; the protocol allows Pay from Client to Shop
session 42, event e3: Charge from Shop to Bank is not allowed here; the protocol allows Pay
    from Client to Shop
```

The verdict names the first event that breaks the protocol and what the protocol allowed instead.

A `where` condition is run too. An event may carry `fields`, the parts of its payload a condition reads, and conformance evaluates the condition over them when the event matches its message. Here the client pays 12 against a quote of 10:

```
{"from":"Client","id":"e1","label":"CreateOrder","session":"7","to":"Shop"}
{"cause":"e1","fields":{"price":10},"from":"Shop","id":"e2","label":"Quote","session":"7","to
    ":"Client"}
{"cause":"e2","fields":{"amount":12},"from":"Client","id":"e3","label":"Pay","session":"7","to
    ":"Shop"}
```

```
PlaceOrder: 1 of 1 sessions break the protocol; first: session 7, event e3: Pay from Client to
     Shop breaks its condition: where p.amount = q.price is false: p.amount = 12, q.price = 10
```

A false condition means the message does not match, so a condition can also select between branches by data, as a label does. The diagram then draws it where it decides, as the guard of its branch:

```
structure Order where
  total : Nat

protocol Tier (Customer, Shop) {
  o: Order from Customer to Shop;
  choice {
    Gold from Shop to Customer cause o where o.total >= 100;
  } or {
    Plain from Shop to Customer cause o where o.total < 100;
  }
}
```

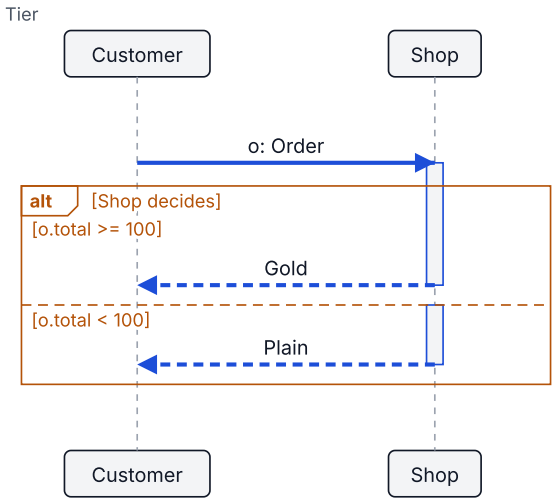


Figure 5.1: A choice made by data: each operand of the `alt` frame carries the condition that selects it.

A condition that cannot be decided, because a field is missing from the log, a message has not happened, or the types do not compare, is reported as unchecked with its reason, never counted as

satisfied. The condition language is small on purpose: field paths, numbers, strings, comparisons, arithmetic, `and`, `or` and `not`. LeanFM interprets it; nothing in it runs as code. Asked after any prefix, LeanFM also says what may happen next and whether the session may end there; that is how a walk through one session is driven, one click at a time.

5.3 State lives in the stream

For questions about value and promises, events are keyed by a unique timestamp, and two links point back in time: `prev`, the event before this one in its thread, and `cause`, the event that made it happen. Both must name earlier events of the same stream. A role's *view* is its own events and everything in their causal past: exactly what it could have learned.

Rewards follow the same rule. A reward vector, the balance of every role in every unit, is written only on an event where it changes, and writing one equal to the vector it inherits is refused. The vector at any event is found by walking `prev` back to the latest one written; finding none means zero. A concurrent event's vector is never taken, because it is not on the walk. In this stream each ask moves the client's spend and the third ask is never answered:


```
[{"ts": 1000,
  "to": "B",
  "reward": {"A": {"USD": -1}},
  "label": "Ask",
  "from": "A"},
 {"ts": 1100,
  "to": "A",
  "prev": 1000,
  "label": "Answer",
  "from": "B",
  "cause": 1000},
 {"ts": 2000,
  "to": "B",
  "reward": {"A": {"USD": -2}},
  "prev": 1100,
  "label": "Ask",
  "from": "A"},
 {"ts": 2150,
  "to": "A",
  "prev": 2000,
  "label": "Answer",
  "from": "B",
  "cause": 2000},
 {"ts": 3000,
  "to": "B",
  "reward": {"A": {"USD": -3}},
  "prev": 2150,
  "label": "Ask",
  "from": "A"},
 {"ts": 4000,
  "to": "B",
  "reward": {"A": {"USD": -4}},
  "prev": 3000,
  "label": "Ask",
  "from": "A"},
 {"ts": 4120,
  "to": "A",
  "prev": 4000,
  "label": "Answer",
  "from": "B",
  "cause": 4000}]
```

5.4 Promises on a stream

A `require` promise is judged on the stream as it was at design time, with causality in place of probability. An entry hits when a target lies in its causal future within the deadline; an answer that names no cause does not count, however soon it comes. An entry whose deadline outlives the stream is pending, not missed. Each window gets an exact one-sided binomial test, but only once it has closed; until then it is open:

```
Requirements against the stated assumptions (exact, per entry):
- at least 99/100 of q reach Answer within 300 ms over 300 entries;
  MET: per entry the probability of reaching Answer within 300 ms is 1. The window holds the 300 entries
     it needs to decide at 19/20 confidence.

Requirements on the observed event stream:
- at least 99/100 of q reach Answer within 300 ms over 300 entries;
  OPEN on the stream: 4 entries, 3 reached a target, 1 missed, 0 pending. Windows: 3/4 open (no
     observation period has closed yet, so nothing is decided). First miss: the entry at 3000.
Each role's state is its view: A 7 events, B 7 events.
The reward vector at the last event (4120), found by walking prev back to the latest vector: A -4 USD.
At 3000: 5 events in its causal past; reward vector A -3 USD.
```

One miss in four would break a promise of 99 in 100 if it were judged at once. The 300-entry period has not closed, so the promise is open. Over 300 closed entries, 300 hits is met at 95/100 confidence ($0.99^{300} \approx 0.049$), 299 is undecided, and 290 is violated.

5.5 Streams from the protocol

The same probabilities that judge a promise can generate streams. A seeded generator walks the protocol, drawing each choice, delay and amount by its exact probability, so the same seed gives the same stream. Every generated session is a walk of the protocol, so it conforms by construction; fed back as an execution, it gives each session's realized rewards, and over many sessions their mean approaches the expected value of Chapter 3. Generated streams are how an implementation is exercised before real traffic exists, and real streams are how its assumptions are corrected once it does.

Part II

What a Protocol Generates

Chapter 6

What a Protocol Generates

A LeanFM project has one program, the protocol, and many outputs. This chapter is what comes out, why each output is generated rather than written, and how the outputs are reviewed.

6.1 Diagrams

The top-level protocol's diagram is what people read first, the way an OpenAPI viewer is where people meet an HTTP interface. It is always LeanFM's rendering of the current text (Figure 1.1), never a drawing of it, so it cannot drift from the program. Every generated view also records why it exists: the prompt that asked for it, the question it answers, its event source, the reducer that computed it, its unit, and its boundary. A view without those is a picture; with them it is evidence.

Large protocols are read from the top: an included protocol is a frame that opens, and a nested `sessions` block is a service drawn once. The same structure could drive an animated canvas, with each frame folding open for detail.

6.2 Code each role is written on

From the typestates of Chapter 2, LeanFM writes a module with one class per state of each role, in Python or TypeScript. A method exists only where the protocol allows the move, fills in the cause the protocol names, logs the event in the format conformance reads, and returns the next state. The client of `PlaceOrder`, in TypeScript:

```
export class ClientS0 extends State {
  readonly state = "ClientS0" as const;

  sendCreateOrder(body?: CreateOrder): ClientS1 {
    const ep = this.take();
    ep.send("Shop", "CreateOrder", "o", null, body);
    return new ClientS1(ep);
  }
}

/** Client at S1: receive(msg): Quote from Shop, SoldOut from Shop. */
export class ClientS1 extends State {
  readonly state = "ClientS1" as const;
```

```
  receive(msg: Message): { label: "Quote"; msg: Message; next: ClientS2 } | { label: "SoldOut";
     msg: Message; next: End } {
    const ep = this.ep;
    if (ep.accepts(msg, "Shop", "Quote")) {
      this.take();
      ep.received(msg, "q");
      return { label: "Quote", msg, next: new ClientS2(ep) };
    }
    if (ep.accepts(msg, "Shop", "SoldOut")) {
      this.take();
      ep.received(msg, null);
      return { label: "SoldOut", msg, next: new End(ep) };
    }
    return ep.refuse("ClientS1", msg, "Quote from Shop, SoldOut from Shop");
  }
}

/** Client at S2: sendPay -> ClientS3. */
export class ClientS2 extends State {
  readonly state = "ClientS2" as const;

  sendPay(body?: Pay): ClientS3 {
    const ep = this.take();
```

In TypeScript a move the state does not allow does not type-check, and `receive` returns a union tagged by label, so handling every answer the protocol allows is a type obligation. The implementer writes only the decisions (which branch, what each message carries) and the transport. Because the generated code logs every message, conformance (Chapter 5) can check the running program without further instrumentation. With each message it logs exactly the fields of its body that some `where` condition reads, and nothing else: `PlaceOrder`'s client logs the amount it pays, never the card number in the same body.

6.3 Interfaces

A role that serves HTTP gets its interface written out as OpenAPI 3.1: one operation per route, the label as its operation id, and every response the protocol allows under its status. The shop's, abridged:

```
{"paths":
 {"/orders":
  {"post":
   {"summary": "CreateOrder from Client",
    "responses":
    {"409":
     {"description": "SoldOut",
      "content":
      {"application/json":
       {"schema": {"$ref": "#/components/schemas/SoldOut"}}}},
     "201":
     {"description": "Quote",
      "content":
      {"application/json":
       {"schema": {"$ref": "#/components/schemas/Quote"}}}}},
    "requestBody":
```

```
    {"content":
     {"application/json":
      {"schema": {"$ref": "#/components/schemas/CreateOrder"}}}},
    "operationId": "CreateOrder"}}},
 "openapi": "3.1.0",
 "info":
 {"version": "0.1.0",
  "title": "PlaceOrder: Shop",
  "description":
  "The requests Shop receives in protocol PlaceOrder, generated by LeanFM from the session
    type."},
 "components":
 {"schemas":
  {"SoldOut":
   {"type": "object",
    "title": "SoldOut",
    "required": [],
    "properties": {},
    "additionalProperties": false},
   "Quote":
   {"type": "object",
    "title": "Quote",
    "required": ["price"],
    "properties": {"price": {"type": "integer", "minimum": 0}},
    "description": "The shop's price for the order, in cents.",
```

Each body's schema is its message type, with only its declared fields. The same types are written out as protobuf, to fix the shape and bytes of each message so every role and every stored record agrees on them:

```
// What the client orders.
message CreateOrder {
  string item = 1;
  uint64 quantity = 2;
}

// The shop's price for the order, in cents.
message Quote {
  uint64 price = 1;
}

// The client pays; the card number is a secret, never logged.
message Pay {
  uint64 amount = 1;
  string card_secr = 2;
}

message Charge {
  uint64 amount = 1;
  string card_secr = 2;
}
```

and as Lean structures that convert to and from JSON, for code written in Lean:

```
/-- What the client orders. -/
structure CreateOrder where
  item : String
```

```
  quantity : Nat
  deriving Repr, BEq, Lean.ToJson, Lean.FromJson

/-- The shop's price for the order, in cents. -/
structure Quote where
  price : Nat
  deriving Repr, BEq, Lean.ToJson, Lean.FromJson

/-- The client pays; the card number is a secret, never logged. -/
structure Pay where
  amount : Nat
  card_secr : String
  deriving Repr, BEq, Lean.ToJson, Lean.FromJson
```

The skeletons carry them too: a Python dataclass or a TypeScript interface per message, and each send method takes its message's type. None of these say anything about order or about who decides; that is the protocol's job. The LeanFM server's own messages are an example (Appendix B).

6.4 Requirements and implementation, reviewed in git

The durable record of a project is two Lean files written out from the checked protocol, and the work proceeds in three layers, each checked before the next:

1. **Requirements** (`Requirements.lean`): the observable behaviour, one clause per behaviour, each with what must always hold and what must never happen.
2. **Implementation mapping** (`Implementation.lean`): how each clause is realized and which test or proof covers it, including every negative requirement.
3. **Source**, written in one pass from the two.

Review is what a team already does: read the diff and commit it. LeanFM records no approvals, and a passing check is evidence for the review, not the review itself. Nothing lives only in a chat: the protocol, the two Lean files, the generated views and the source are all in the project's one directory, so a new session resumes from the files. LeanFM is itself such a project; its own requirements are `LeanFM/LLMGenerated/SelfHosting.lean`, and every feature in this book is a clause there with an implementation decision and a test.

Chapter 7 is the discipline that keeps those clauses honest: no feature is complete until its forbidden behaviours are stated and each has a control and a negative test.

6.5 Building software from a protocol

1. Write the protocol and check it until it has no issues.
2. Show it: the diagram, each role's typestate, a walk through one session, and a generated stream.
3. State the assumptions and promises, and check the promises against the assumptions.
4. Generate each role's states and implement the role on them.

5. Run it, and check its log for conformance and its promises over closed windows; fix the code, or the protocol if the protocol was wrong.

6. State each rule the structure does not show as a CTL question.

7. When the protocol changes, ask which roles can be redeployed first.

Chapter 7

Feature Completion as Positive and Negative Proof

A request to show a message, list source, accept a transaction, render a diagram or produce an MP4 states a desirable observation. The change can also introduce stale state, disclosure, resource exhaustion, weaker authentication, corruption or success without an artifact. Completion therefore requires a paired argument:

1. a *positive invariant* states what the system must do; and
2. applicable *negative invariants* state what the system must never do.

For a feature that crosses a boundary, the positive invariant is a property of the global type, and the negative ones usually name a message that must never be sent or accepted under a channel's physics: a replayed request, a response to the wrong session, a call after its deadline. Review is proportional: button spacing needs no cryptographic audit, filesystem access needs authorization and path analysis, and a material risk without a mapped, tested control blocks completion.

7.1 The logical shape of a feature

Let $good_f$ mean the observable contract of feature f, and let $bad_{f,1}, \ldots, bad_{f,n}$ name reachable failures introduced or exposed by that feature. A common safety-shaped completion condition is

$$AG\, good_f \;\wedge\; \bigwedge_{i=1}^{n} AG\, \neg bad_{f,i}.$$

The first conjunct is positive; each other is negative: no permitted execution reaches a forbidden observation. The counterexample form

$$AG\, \neg bad \;\equiv\; \neg EF\, bad$$

is what a negative test does: it tries to build the $EF\, bad$ witness (another user's project path, a stale response after a project switch, an oversized upload, a secret in a log), and finding one refutes completion. Some features are liveness obligations, such as $AG(submitted \rightarrow AF(complete \vee failed))$; review must not swap liveness for safety or hide fairness and availability assumptions.

7.2 Stability review before security review

Before editing code, identify the existing invariant and review the applicable ways an environment change could break it:

Regression. Which accepted behavior, file format, route, proof, or UI state can this change invalidate? Preserve conforming behavior and add a regression test at the narrowest durable boundary.

Failure. What happens when a dependency, renderer, compiler, provider, filesystem operation, or network request fails? Failure must become an explicit terminal observation rather than a false success or endless wait.

Concurrency. Can a late response, simultaneous session, queued prompt, or overlapping write be attributed to the wrong project or revision? Correlation identity and revision checks must survive reordering.

Resources. What bounds memory, bytes, queue occupancy, recursion, process time, media dimensions, and response size? A new feature must not silently turn a finite resource contract into an unbounded one.

Durability and recovery. Which state is authoritative after restart or partial failure? Atomic writes, rebuildable indexes, and project-local ownership make recovery inspectable.

Compatibility. Which clients, stored revisions, generated artifacts, and build environments must continue to work? Version or migrate a changed representation rather than reinterpreting old bytes silently.

Record the concrete invariants at the changed boundary, and say briefly when a dimension is non-material.

7.3 Security as forbidden observable behavior

“A user never reads a project they do not own” states protected behavior; “use authentication” names a mechanism. Before choosing a control, identify:

1. the trust boundary crossed by the feature;
2. the asset being protected;
3. the attacker’s relevant capability; and
4. the forbidden observable outcome.

Forbidden behaviors include unauthorized access, cross-user or cross-project disclosure, secret persistence or logging, injection, path or symlink escape, replay, request/response confusion, privilege expansion, executable media, resource exhaustion and failing open. State them testably (“a source path absent from the authorized inventory never returns bytes”), and map each material one to a control (path confinement, authenticated identity, an allowlist, a byte limit, an atomic revision check) and a negative test or proof; a mechanism alone proves nothing.

7.4 The feature-completion ledger

For each feature, retain the following compact traceability record in the typed requirements and implementation mapping:

Field	Required content
Prompt	The stakeholder request whose completion is being claimed.
Positive claim	Observable always or liveness property, with actor, scenario, boundary, and revision.
Negative claims	Applicable never or always-not properties; omitted families are explicitly judged non-material.
Controls	Concrete implementation mechanisms mapped one-to-one or many-to-one to the negative claims.
Evidence	Positive conformance tests plus negative tests, proofs, generated traces, and named checks.
Residual risk	Stated limitation the stakeholder reviews in the diff, assumption, deferred mitigation, or an explicit completion blocker.

Both sides need evidence: ruling out failures does not demonstrate the feature, and a demonstration does not rule out disclosure or corruption.

7.5 Worked example: the project Source view

“Show implementation files in the Source tab.” Positive: selecting an authorized project lists its files, and selecting a readable UTF-8 file shows it; switching projects clears the selection, so a late response cannot show stale content. The feature crosses a filesystem boundary, protecting source, account records, repository metadata and credentials from a user who can choose URL paths and guess names. Negative invariants:

- a user never reads another user’s project source;
- a guessed ignored or metadata path never returns bytes;
- a path or symlink never escapes the authorized source root;
- a binary or oversized file never becomes an unbounded text response;
- a project switch never retains the previous project’s source state; and
- an empty source directory never accompanies a claim that implementation succeeded.

Controls are an authenticated explicit project ID, canonical root resolution, exact source-inventory membership, UTF-8 reads, a finite response limit and project-keyed React state. Negative tests request another account’s file and a guessed `.git/config`; both return not-found without revealing existence. The positive test creates, lists and opens a project-local file and checks its exact content.

7.6 Worked example: chat auto-scroll

Even a small convenience has stability boundaries. Positive: loading a conversation and every new message reveal the bottom of the chat pane, scrolling that pane, not the page or the sidebar, and resetting with the project-keyed component. No authorization boundary is crossed, so a threat model would be noise, but project isolation and stale responses still matter.

7.7 When the result is not complete

The ledger is incomplete with an untested positive claim, a missing negative requirement, an uncontrolled risk, a control without a negative test or proof, a liveness claim without its fairness assumptions, skipped validation, or a check of a substitute artifact. Report the blocker and return to the relevant layer, stating any residual risk where the stakeholder will see it before committing. A green build or a persuasive diagram does not fill these gaps.

Part III

Worked Examples

Chapter 8

The Bakery: Declared and Measured

The bakery is the quantitative worked example. Its tasks are written as a *core*, a session between customers, staff and the oven that says only what can happen, plus *overlays*: separate tables of branch probabilities and delays. A *declared* overlay states what we expect; a *measured* overlay is reduced from recorded events. Both answer the same questions about the same unchanged core, and Chapter 3 develops the idea in general. The corpus is the 81,039-event stream `examples/bakery-events.jsonl`, a reproducible synthetic population generated by `scripts/generate_bakery_ev` its numbers describe that corpus, not a real bakery. Every figure and table below is regenerated by `lake env lean -run BakeryFlowExample.lean`.

8.1 The morning, generated

Bread is baked to a schedule, not to order. Each batch is baked, queued on a truck, and stocked at the storefront before opening. The stocking task's core lists only those four messages; the overlay measured from the 140 recorded batches supplies how long each step takes. The figure is the most likely stocking run under that overlay.

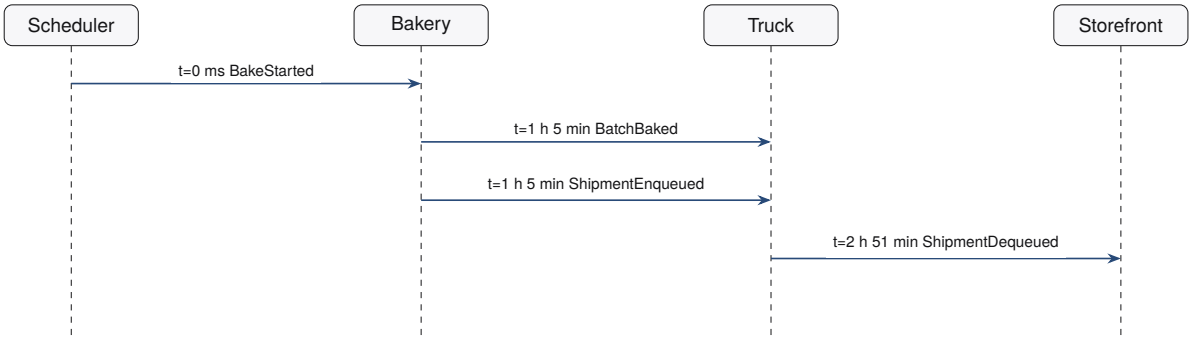


Figure 8.1: One batch from oven to shelf, with mean step times measured from the corpus.

8.2 A sale: what can happen

A customer asks for bread. The storefront either reserves stocked inventory or turns the customer away; a reserved order is paid for, and the payment either completes the sale or is declined. The

core states this and nothing more. It already proves that every sale attempt ends and that a customer *can* be turned away.

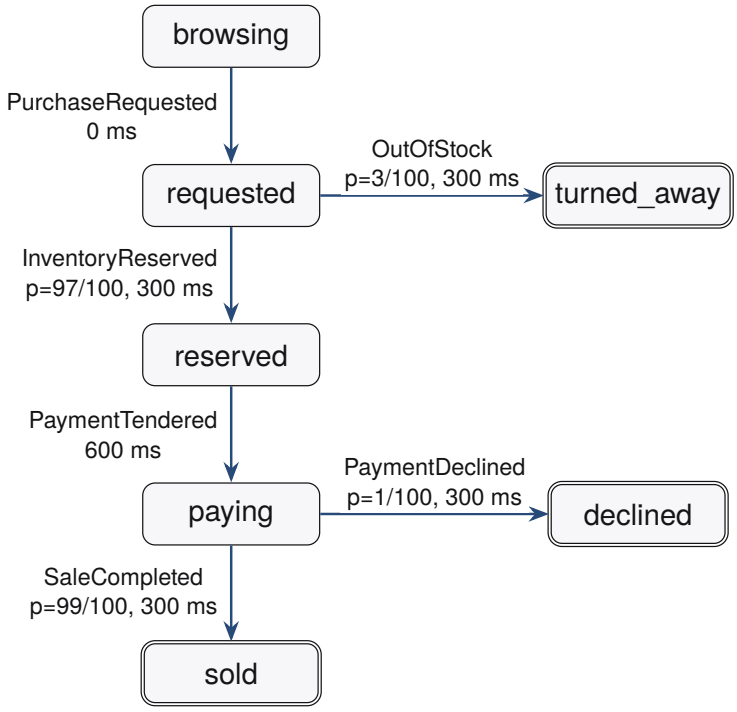


Figure 8.2: The retail-sale core, with the bakery's declared expectations drawn on its edges. Double borders mark terminal states.

8.3 What we expected, and what the corpus shows

The declared overlay was written before looking at any data: a 3% stockout rate, a 1% payment-decline rate, and round-number delays. The measured overlay has exactly the same shape. `LeanFM.Overlay.measure` walks each of the 20,000 recorded sales through the core, counts how often each branch was taken, and averages the time since the previous message. A recorded session that left the grammar would be reported rather than counted.

Transition	Declared p	Declared delay	Measured count	Mean delay
browsing → `PurchaseRequested`	1	0 ms	20000 / 20000	0 ms
requested → `InventoryReserved`	97/100	300 ms	20000 / 20000	289 ms
requested → `OutOfStock`	3/100	300 ms	0 / 20000	—
reserved → `PaymentTendered`	1	600 ms	20000 / 20000	574 ms
paying → `SaleCompleted`	99/100	300 ms	20000 / 20000	300 ms
paying → `PaymentDeclined`	1/100	300 ms	0 / 20000	—

Table 8.1: Declared expectations against branch counts and mean delays reduced from the corpus.

8.4 The same questions under both overlays

Question	Formula	Declared	Measured
Does every sale attempt end?	`AF terminal`	proved	(core, no overlay)
Can a customer be turned away?	`EF turned_away`	proved	(core, no overlay)
How likely is a sale?	`P=? [F sold]`	9603/10000 (0.9603)	1
Sold within 1.2 s?	`P=? [F<=1200ms sold]`	9603/10000 (0.9603)	1; raw data 0.5454
How often out of stock?	branch `OutOfStock`	3/100	0 of 20000; 95% bound $\leq$ 3/20000
Expected sale duration	`E[duration]`	1173.0 ms	1163.0 ms

Table 8.2: One unchanged core, two overlays. The first two rows need no numbers at all.

Two rows deserve a closer reading. The corpus contains no stockout in 20,000 sales, but "never observed" is not "impossible": with zero occurrences in n trials the usual 95% upper bound on the rate is about $3/n$, so the honest measured answer is a bound, not zero. The core, meanwhile, proves that turning a customer away is possible, so the path must still be designed and tested even though the data never exercised it.

8.5 The deadline lesson: a mean hides the spread

The measured overlay records each step's *mean* delay. Its predicted sale takes 1163 ms, inside the 1.2 s deadline, so the model answers $P[F_{\leq 1200\,\mathrm{ms}}\ sold] = 1$. The recorded sales say otherwise: only 0.545 of them finished within 1.2 s. Real sales took between 348 and 1956 ms.

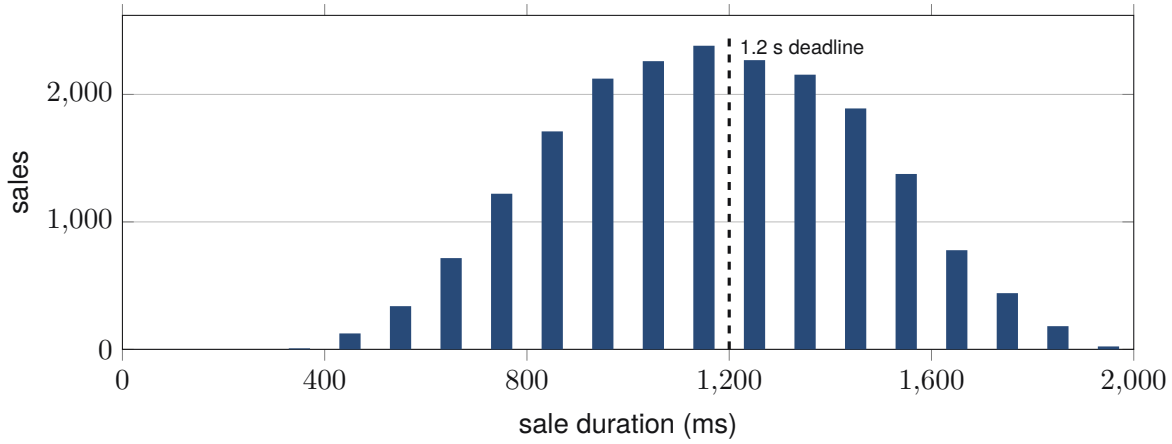


Figure 8.3: Duration of every recorded sale. The mean sits just inside the deadline; almost half of the sales do not.

This is why a measured overlay must eventually carry distributions, not just means, as FizzBee's performance files do: the deadline question is about the spread, and a point estimate answers it with false confidence. Until then, the raw-data column keeps the model honest by comparing its prediction with the recorded fraction.

8.6 Generated runs

The most likely sale under the measured overlay is the one a newcomer should see first. The most likely *failing* sale comes from the declared overlay, because the measured one gives every failure probability zero: it is the stockout, the case the corpus never shows and the core says can happen.

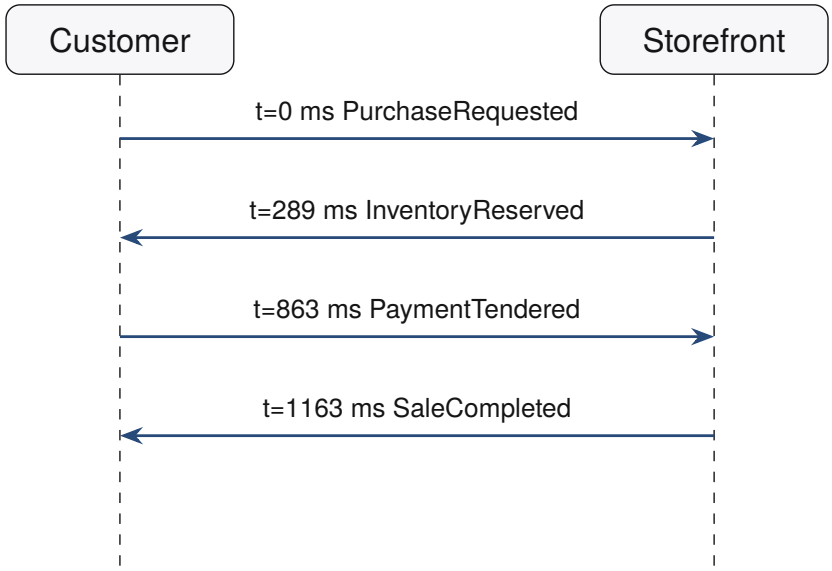


Figure 8.4: The usual sale, with mean times measured from the corpus.

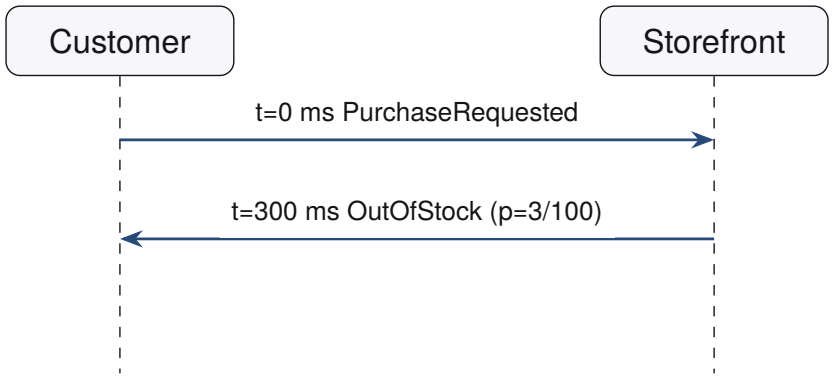


Figure 8.5: The most likely failing sale under the declared overlay: a stockout, probability $3/100$, never seen in the corpus.

8.7 Money and waste

The same corpus answers economic questions through ordinary reducers. For each calendar day, net revenue is the sum of `SaleCompleted` amounts less refunds; ingredient cost comes from `BatchBaked`; employee pay from `ShiftPaid`; and modeled contribution is revenue less ingredients and pay. Donated bread is already in ingredient cost, so its value is reported separately rather than subtracted twice, and it is not a tax saving until tax rules are supplied. Costs that have no events in the corpus are excluded, so this is contribution, not business profit.

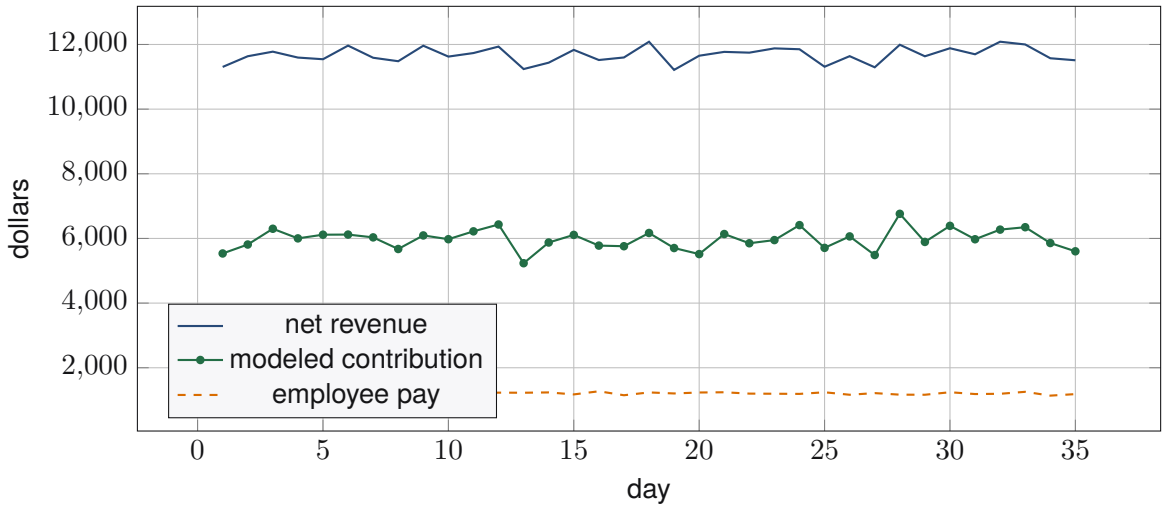


Figure 8.6: Daily economics reduced from sale, batch and payroll events.

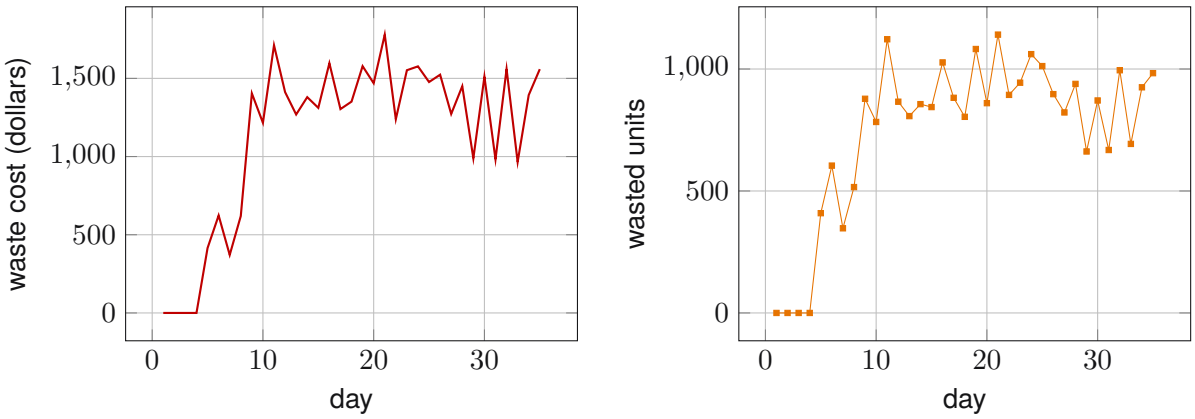


Figure 8.7: Donation-eligible cost and units by day: inventory left when a batch's freshness window closes. Cost and units differ because a donated sourdough loaf and a donated croissant are not worth the same.

Grouping the same events differently—by product, batch age, shift, or order volume—gives further charts without changing the corpus. The axes are always named reductions over events, which is what lets a measured overlay and an economic chart come from one source of truth. How these specifications drive code generation is the subject of Chapter C.

Chapter 9

Declared Physics: TFTP and the Sorcerer's Apprentice

A grammar does not say what happens to a message after it is sent: whether it queues, how large the queue is, whether the network loses it, duplicates it or delays it. Those facts are part of what a protocol means. The example is the Trivial File Transfer Protocol (TFTP) [20, 21] and its best-known defect, the *Sorcerer's Apprentice* bug, named and fixed in RFC 1123 [2].

This is the book's central claim in its smallest form. As a multiparty session type [11], the original and the fixed sender are the same well-formed protocol. With the physics declared, LeanFM finds a walk in which one early timer makes every later block travel twice for the original, and proves that none exists for the fix.

Status: the verdicts are real `check_ctl` results, and the protocol and its physics are checked Lean in `SessionExample.lean`. The product of roles and queues is still built by a small script, `video/tftp_model.mjs` in the companion video repository; deriving it from the declarations is future work.

9.1 The grammar

TFTP moves a file in numbered blocks. The sender sends `DATA` n; the receiver answers `ACK` n; the sender then sends `DATA` $n+1$. A block shorter than the full size ends the transfer.

In LeanFM's session-type syntax, with the physics beside it:

```
protocol Transfer (Sender, Receiver) {
  rec Block {
    d: Data from Sender to Receiver;
    choice {
      Ack from Receiver to Sender cause d;
      continue Block;
    } or {
      LastAck from Receiver to Sender cause d;
    }
  }
}

def tftpPhysics : Physics Role :=
```

```
{ channels := [ (.sender, .receiver, { udp 2 with duplication := false })
              , (.receiver, .sender, { udp 2 with duplication := false }) ]
  timers := [{ role := .sender }] }
```

Without a timer this is a textbook global type: the receiver chooses, answering `Ack` for a full block and `LastAck` for a short one, and the projected roles never deadlock or receive an unexpected message. But a real network loses packets, so the sender needs a timer that resends a block when no `ACK` arrives in time. That retransmission is the whole engineering content of TFTP, and the classic reading has nowhere to put it, because it assumes every message arrives exactly once, in order.

9.2 The physics

`tftpPhysics` above declares each channel as `udp` with one difference written out. Each direction holds at most two packets, delivered in either order or dropped, and the sender's timer may fire any time a block is unacknowledged, since a slow answer looks like a lost one. The network is declared never to duplicate: every duplicate below is made by the sender. The model has three blocks. The two senders differ in one line, the reaction to a repeated `ACK`:

Original (RFC 783).
Every `ACK` n, even a repeated one, makes the sender send `DATA` $n{+}1$.

Fixed (RFC 1123).
A repeated `ACK` is ignored. A block is sent again only when its own timer fires.

The checked machine is the product of sender, receiver and both queues: 201 reachable states and 891 transitions for each sender. Its events are `timeout`, `drop`, `data` (a new block delivered), `dup`k (block k delivered again, and acknowledged again), `ack` (the first `ACK` for the current block), `dupack` (an `ACK` for an earlier block) and `done`.

9.3 The question

Suppose the timer fires early once, at the start, so `DATA` 1 is in flight twice, and afterwards nothing is lost and the timer never fires. Can block 3 still arrive twice? From the state after that `timeout`:

$$E\,[\,\neg(\mathit{timeout} \vee \mathit{drop})\;U\;\mathit{dup}_3\,]$$

"Some path has no `timeout` and no `drop` until block 3 is delivered a second time." For the original sender LeanFM proves it, with the witness in table 9.1; the queue columns are read from the states on the path.

The pattern repeats every four steps: a doubled block brings a doubled `ACK`, whose first copy sends the next block and whose late second copy sends it again. Nothing fails and every packet arrives, yet the duplication sustains itself, like the brooms that keep multiplying, doubling the traffic of real transfers. For the fixed sender the formula is disproved over all 509 reachable nodes: the repeated `ACK` 1 is ignored and the duplication dies out.

step	event	meaning	DATA queue	ACK queue
1	`timeout`	timer fires early; DATA 1 sent again	1, 1	–
2	`data`	DATA 1 delivered	1	1
3	`dup1`	DATA 1 delivered again; ACK 1 sent again	–	1, 1
4	`ack`	ACK 1 received; DATA 2 sent	2	1
5	`data`	DATA 2 delivered	–	1, 2
6	`dupack`	ACK 1 received again; DATA 2 sent again	2	2
7	`dup2`	DATA 2 delivered again; ACK 2 sent again	–	2, 2
8	`ack`	ACK 2 received; DATA 3 sent	3	2
9	`data`	DATA 3 delivered	–	2, 3
10	`dupack`	ACK 2 received again; DATA 3 sent again	3	3
11	`dup3`	DATA 3 delivered again; ACK 3 sent again	–	3, 3

Table 9.1: LeanFM's witness for the original sender: one early timer, then no loss and no further timeouts, and every later block is sent and delivered twice.

9.4 Liveness needs fairness

Two more questions are asked of both senders.

$$AG\,EF\ terminal \text{ holds}, \qquad AF\ terminal \text{ fails}.$$

From every state some continuation finishes, but not every path does: LeanFM returns a lasso in which the timer fires forever, each copy dropped by a full queue, and nothing is delivered. No protocol can force a network to deliver, so a promise that the transfer *will* finish needs a stated fairness assumption on the channel, such as "a packet sent infinitely often is delivered infinitely often".

9.5 What the classic session-type reading leaves out

sender	as a session type	with the physics declared
original (RFC 783)	well-formed lock step	one early timer doubles every later block
fixed (RFC 1123)	the same well-formed lock step	no cascade; checked over all 509 nodes

Table 9.2: The same grammar, two meanings.

The grammar is the same in both rows of table 9.2; only its meaning changed. With the theory's unbounded, reliable, ordered queues [11] a retransmission timer is unnecessary, and the Sorcerer's Apprentice cannot even be expressed. LeanFM's position follows from the actor view:

- Every choice is an internal choice of one actor, made only from that actor's own history: what it sent, and what arrived in its queue (`prev` and `cause`).
- Every queue has a declared capacity, and a declared policy for when it is full.
- Every channel declares what the network may do: drop, reorder, duplicate, delay.

- Liveness claims carry their fairness assumptions as named requirements.

UDP protocols make the physics impossible to ignore, and TFTP is the smallest with a famous bug that stays invisible until the queues and the early timer are modeled.

Chapter 10

Worked Example: OAuth Authorization Code Flow

This chapter is a behavioural sketch inspired by the OAuth authorization-code flow, starting above HTTP and protobuf, to exercise scenario decomposition, parameter flow, alternatives, requirements and rendering. In session-type terms it is a global type with five roles, a grant-or-deny choice by the authorization service, and parameters such as `state` and the code that flow from one message to a later one. Its channels are HTTP calls and browser redirects, whose physics (a redirect never followed, a callback replayed by an attacker) is where the security requirements live.

The listings are *proposed notation*, not compiled; the generated figures are real. The flow is simplified: the authorization request omits `client_id` and `response_type`, and the token request carries only the code, without `redirect_uri`, client authentication or PKCE, all required by RFC 6749 and current guidance. The knowledge claims hold for this simplified flow only.

10.1 Actors, values, and observable events

The person is modelled separately from the browser, because redirects are observable messages between software participants.

```
inductive Actor where
  | user | browser | app | authorizationServer | resourceServer

opaque State       : Type
opaque AuthCode    : Type
opaque AccessToken : Type
opaque Resource    : Type
opaque Profile     : Type
opaque Password    : Type
opaque Bytes       : Type
```

Every terminal elaborates to the common event envelope; “authorization redirected”, for example, becomes:

```
{ timeAt := t2, id := e2, priorIds := [e1]
, session := browserSession, task := oauthAuthorizationTask
, from := Actor.app, to := Actor.browser
```

```
, kind := "authorization redirected"
, fields := [field "state" state, field "redirectUri" callback]
}
```

Authentication detail used by Production 2

OAuth does not prescribe how the authorization service authenticates the person; a password branch exercises knowledge flow without pretending the password is an OAuth parameter.

```
def authenticateAtAuthorizationServer (password : Password) : Scenario :=
  scenario "authenticate resource owner" do
    from_ .browser
    to_ .authorizationServer
    emitBytes "credentials submitted"
      (protectFor .authorizationServer
        (encode [field "password" password]))

    from_ .authorizationServer
    to_ .browser
    oneOf "authentication result"
      [ emitBytes "authentication accepted" (encode [tag 1])
      , emitBytes "authentication rejected" (encode [tag 0])
      ]
```

`protectFor` promises confidentiality to the named endpoint, realized later by TLS, message-level encryption or a trusted local channel. Without it, a network observer would derive the password from the bytes.

10.2 Production 1: application request and redirect

```
def applicationRequestAndRedirect (resource : Resource) : Scenario :=
  scenario "application request and redirect" do
    let state <- declare State "state"
    let callback := Uri.https "app.example" "/oauth/callback"

    from_ .browser
    to_ .app
    emit "application request" [field "resource" resource]

    from_ .app
    to_ .browser
    emit "authorization redirected"
      [field "state" state, field "redirectUri" callback]

    export "state" state
    export "redirectUri" callback
    export "resource" resource
```

The production establishes the correlation value `state` and exports it; nothing says how the application stores it.

10.3 Production 2: authorize, then grant or deny

```
def authorizationGranted : Scenario :=
  scenario "authorization granted" do
    use state <- imported State "state"
    let code <- declare AuthCode "code"

    from_ .authorizationServer
    to_ .browser
    emit "authorization code returned"
      [field "state" state, field "code" code]

    from_ .browser
    to_ .app
    emit "authorization callback"
      [field "state" state, field "code" code]
    export "code" code

def authorizationDenied : Scenario :=
  scenario "authorization denied" do
    use state <- imported State "state"
    from_ .authorizationServer
    to_ .browser
    emit "authorization error returned"
      [field "state" state, field "reason" "access_denied"]
    from_ .browser
    to_ .app
    emit "authorization error callback"
      [field "state" state, field "reason" "access_denied"]

def authorizeAndDecide (password : Password) : Scenario :=
  scenario "authorize and decide" do
    use state       <- imported State "state"
    use redirectUri <- imported Uri "redirectUri"

    from_ .browser
    to_ .authorizationServer
    emit "authorization requested"
      [field "state" state, field "redirectUri" redirectUri]

    include (authenticateAtAuthorizationServer password)
    oneOf "authorization decision"
      [ include authorizationGranted
      , include authorizationDenied
      ]
```

Collapsed, the production is one node; expanded, it shows authentication and the choice. A probability distribution may be attached later without changing the legal traces.

10.4 Scenario 3: exchange the code and use the token

```
def exchangeAndFetch : Scenario :=
  scenario "exchange code and fetch resource" do
    use code     <- imported AuthCode "code"
    use resource <- imported Resource "resource"
    let token    <- declare AccessToken "accessToken"
    let profile  <- declare Profile "profile"

    from_ .app
    to_ .authorizationServer
    emit "token requested" [field "code" code]

    from_ .authorizationServer
    to_ .app
    emit "access token issued" [field "accessToken" token]

    from_ .app
    to_ .resourceServer
    emit "protected resource requested"
      [field "accessToken" token, field "resource" resource]

    from_ .resourceServer
    to_ .app
    emit "protected resource returned"
      [field "resource" resource, field "profile" profile]

    from_ .app
    to_ .browser
    emit "application response" [field "profile" profile]
```

The token is visible only where the protocol transmits it; the grammar says nothing about the server's database or the application's sessions.

10.5 Reading the scenarios together

```
def oauthAuthorization (resource : Resource) (password : Password) : Scenario :=
  scenario "OAuth authorization code flow" do
    include (applicationRequestAndRedirect resource)
    include (authorizeAndDecide password)
    after authorizationGranted include exchangeAndFetch

def oauthSystem (resource : Resource) (password : Password) : SystemSpec :=
  generatedBy (oauthAuthorization resource password)
  |> requireAll oauthRequirements
```

The `after` line is a guarded nonterminal reference: the denial branch ends without exchanging a code.

10.6 The parallel protobuf alphabet

The scenario says which messages may occur, in what order, with which parameter relationships; the protobuf file assigns field numbers and wire types to the same messages:

```
syntax = "proto3";

package leanfm.oauth;

enum Actor {
  ACTOR_UNSPECIFIED = 0;
  BROWSER = 1;
  APPLICATION = 2;
  AUTHORIZATION_SERVER = 3;
  RESOURCE_SERVER = 4;
}

message EventEnvelope {
  string event_id = 1;
  repeated string prior_ids = 2;
  string session_id = 3;
  string task_id = 4;
  uint64 clock_tick = 5;
  Actor src = 6;
  Actor dst = 7;

  oneof payload {
    ApplicationRequest application_request = 20;
    AuthorizationRedirect authorization_redirect = 21;
    AuthorizationRequest authorization_request = 22;
    CredentialsSubmitted credentials_submitted = 23;
    AuthenticationResult authentication_result = 24;
    AuthorizationCodeReturned authorization_code_returned = 25;
    AuthorizationCallback authorization_callback = 26;
    AuthorizationErrorReturned authorization_error_returned = 27;
    AuthorizationErrorCallback authorization_error_callback = 28;
    TokenRequest token_request = 29;
    AccessTokenIssued access_token_issued = 30;
    ProtectedResourceRequest protected_resource_request = 31;
    ProtectedResourceReturned protected_resource_returned = 32;
    ApplicationResponse application_response = 33;
  }
}

message ApplicationRequest {
  string resource = 1;
}

message AuthorizationRedirect {
  string state = 1;
  string redirect_uri = 2;
}

message AuthorizationRequest {
  string state = 1;
  string redirect_uri = 2;
}

message CredentialsSubmitted {
  bytes protected_credentials = 1;
}

message AuthenticationResult {
  bool accepted = 1;
}
```

```
message AuthorizationCodeReturned {
  string state = 1;
  bytes code = 2;
}

message AuthorizationCallback {
  string state = 1;
  bytes code = 2;
}

message AuthorizationErrorReturned {
  string state = 1;
  string reason = 2;
}

message AuthorizationErrorCallback {
  string state = 1;
  string reason = 2;
}

message TokenRequest {
  bytes code = 1;
}

message AccessTokenIssued {
  bytes access_token = 1;
}

message ProtectedResourceRequest {
  bytes access_token = 1;
  string resource = 2;
}

message ProtectedResourceReturned {
  string resource = 1;
  bytes profile = 2;
}

message ApplicationResponse {
  bytes profile = 1;
}
```

Each scenario message maps to one member of `EventEnvelope.payload`; the envelope carries identity, `prior_ids`, the `(session_id, task_id)` key, a clock tick and the endpoints. Order, queues, probability and dwell stay in the scenario; byte tags stay in the schema. Validation checks that every terminal has one variant with matching endpoints and field types. Protobuf does not promise one canonical byte sequence, so byte-for-byte comparison also fixes deterministic serialization and length-delimited framing.

10.7 Generate one legal trace and its artifacts

Generation chooses productions and field values; everything after is a deterministic projection of the resulting events. Choose the accepted and granted alternatives and instantiate

```
session = "s1"              task = "oauth"
resource = "profile"        state = "x7"
redirectUri = "https://a/cb"
protectedCredentials = aa bb
```

```
code = 01 02                accessToken = 03 04
profile = 05 06
```

The production derivation is

```
oauth
=> applicationRequestAndRedirect ;
   authorizeAndDecide ;
   after(authorizationGranted, exchangeAndFetch)
=> applicationRequestAndRedirect ;
   authorizationRequested ;
   credentialsSubmitted ; authenticationAccepted ;
   authorizationGranted ; exchangeAndFetch
```

The grant admits `exchangeAndFetch`; denial would disable it. Expanding the remaining nonterminals gives this terminal word:

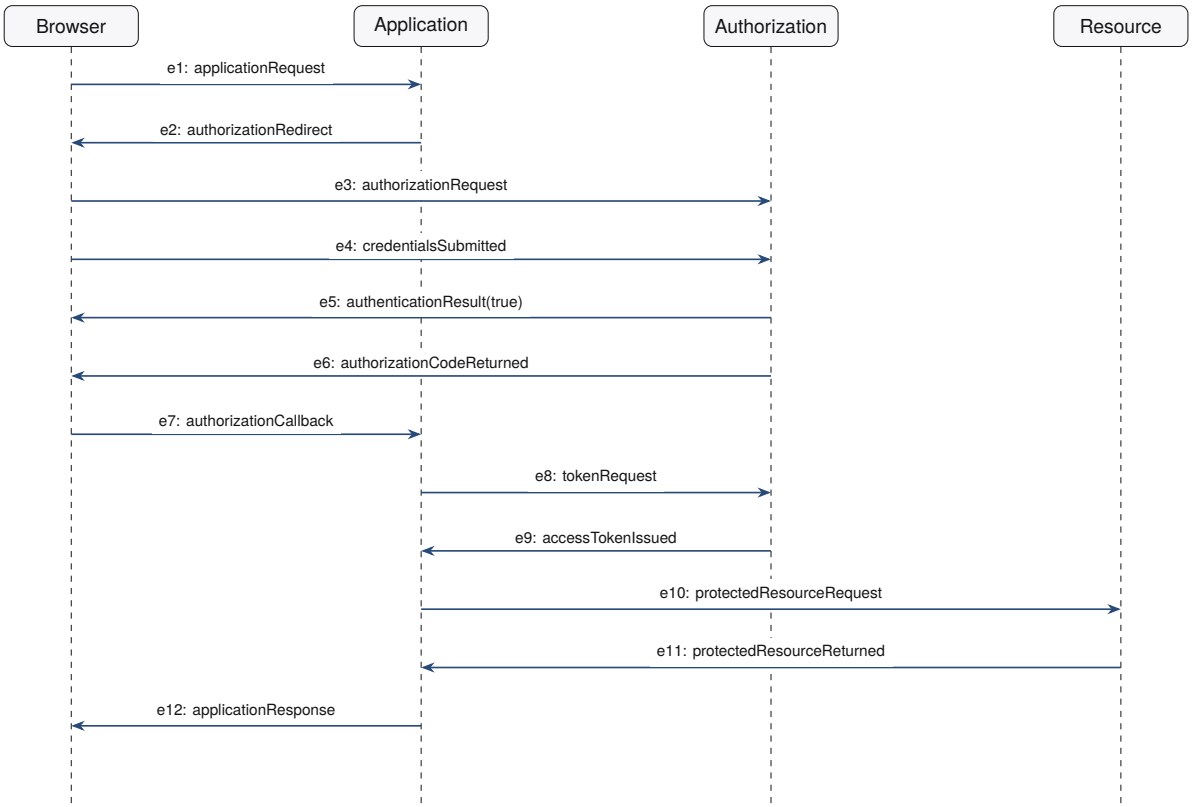


Figure 10.1: The selected terminal word as an interaction diagram. Event IDs label messages on arrows; no actor-to-actor arrow is written as implication-like text.

The choice has disappeared into ordinary terminals: `e5`'s bytes select the accepted authentication, and `e6`'s oneof tag, source and bytes distinguish grant from error. No separate "chooser" value is needed.

10.7.1 Envelope stream and concrete bytes

Clock ticks and identifiers increase; the chain is linear, so `priorIds(`e_n`)` is $[e_n - 1]$ (a fork or join would change the lists, not the procedure). One envelope from the companion fixture:

```
e1   0  application_request          31  0a0265311a02733122056f6175746830013802a201090a0770726f66696c65
```

`examples/oauth-granted.bytes` holds all twelve (440 bytes; largest 46). There is no generator or round-trip check for this fixture, so it is an example, not evidence of reproduced serialization.

10.7.2 Event graph with stored backpointers

Parsing each envelope selects a payload alternative, checks its `src`/`dst`, and consumes one terminal. The event graph:

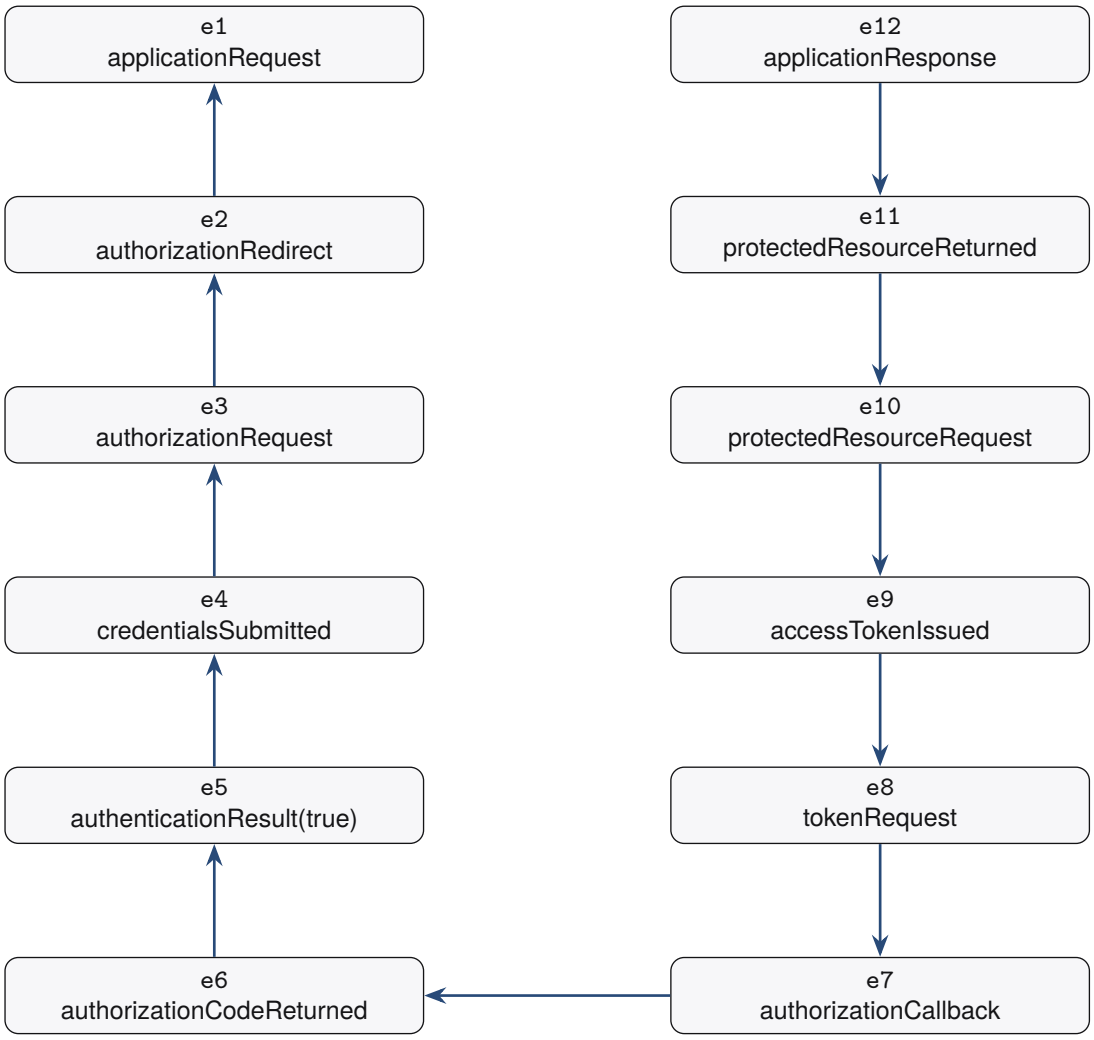


Figure 10.2: The accepted trace as an event graph. Arrows are stored backpointers: `e1` has none and each later event points to its immediate predecessor.

This is one path; the residual state machine, derived from every legal residual, also has the rejected and denied branches.

10.7.3 Actor projections

Projection keeps each actor's terminals in order: Resource receives `e10` and sends `e11`. The global word stays authoritative for cross-component order and parameter equality, which a projection cannot show.

10.7.4 Event order, queues, and memory

The interaction diagram and the state-machine path are the same order graph laid out by actor and by residual; neither is drawn independently. The word does not fix a transport schedule; the canonical one runs `produce; transport; receive` per event, so every queue peaks at one message and the global queued-byte peak is 46. Another interleaving may peak higher, so this schedule is a witness, not the memory-budget proof.

10.7.5 Measurements, plots, and verdicts

From the clocks and correlation key, the generated task attempt is

```
{ key = ("s1", "oauth"),
  started = { kind = applicationRequest, timeAt = 0 },
  completed = { kind = applicationResponse, timeAt = 23 },
  outcome = succeeded, latency = 23 ticks }
```

It contributes a latency point `(maxConcurrency=1, latency=23)`, a byte series ending at `(clock=23, bytes=440)` and a success to the outcome chart, but no success probability: estimating the grant's 0.87 needs many sessions, including incomplete ones. Trace monitors accept `state=x7`, the code `01 02`, and no exchange after denial; CTL obligations need every reachable residual. The pipeline:

```
grammar + field values
  -> legal terminal word
  -> typed EventEnvelope stream
  -> framed protobuf bytes
  -> partial-order graph + interaction diagram
  -> residual FSM path + per-actor projections
  -> queue schedule + byte-pressure samples
  -> task attempt + probability/latency/XY reducers
  -> replay fixture + trace-local property verdicts
```

Only branch and value generation, and the transport schedule, add choices; every other arrow is reproducible.

10.8 Interaction diagrams by scenario

10.9 Requirements as observable constraints

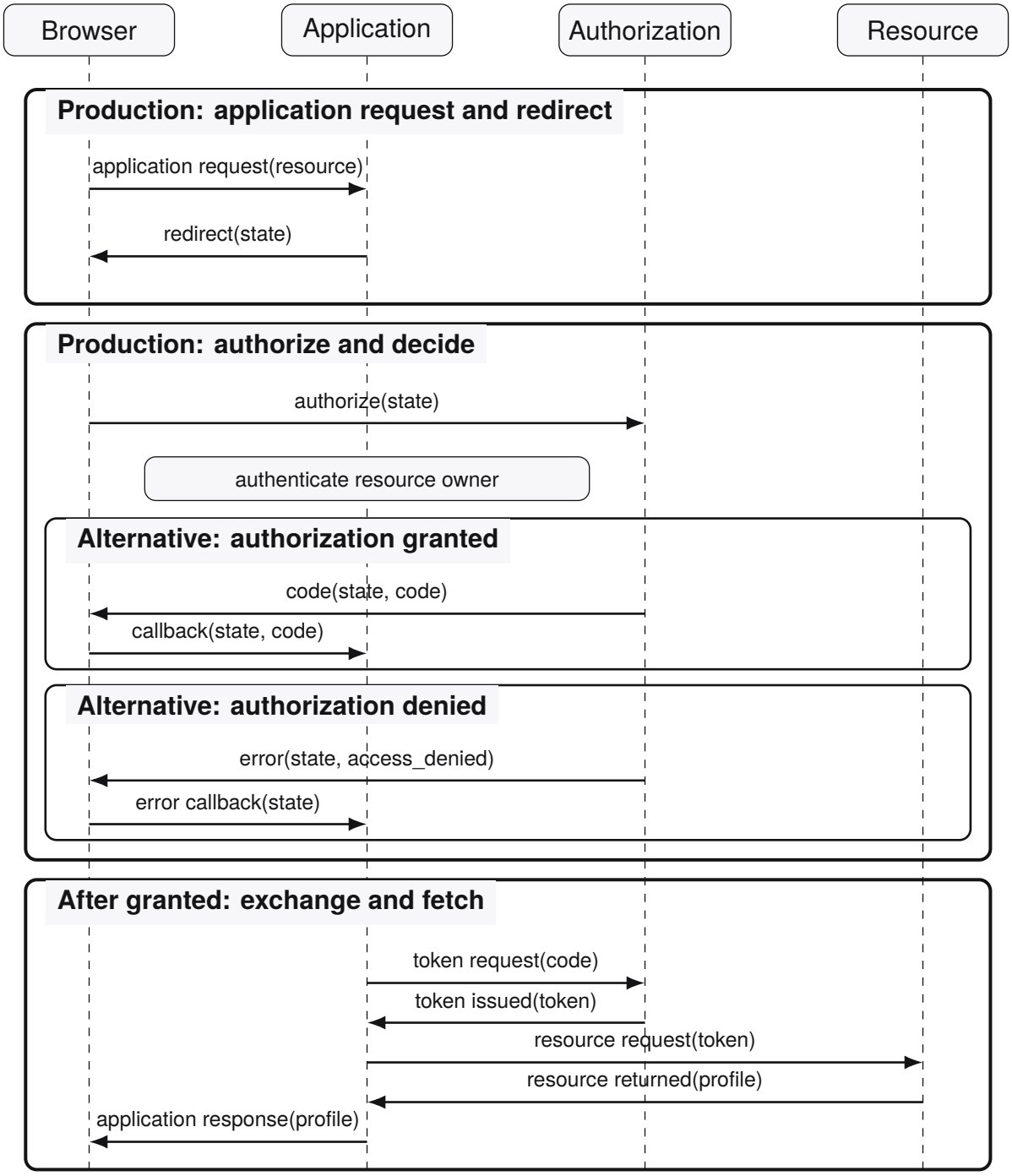


Figure 10.3: OAuth interaction view. Production frames provide collapsible abstraction boundaries. The authorization production contains its request and both alternatives; the guarded exchange production is enabled only by the granted alternative.

```
def oauthRequirements : List Requirement :=
  [ require "callback state matches request"
      (on "authorization callback" (previouslySame "state"
        "authorization requested"))

  , forbid "code exchange after denial"
      (previously "authorization denied" && event "token requested")

  , require "issued token came from a code"
      (on "access token issued" (previouslySame "code" "token requested"))

  , ctl "every decision reaches the application"
      (AG (decisionAtAuthorizationServer --> AF callbackAtApplication))

  , ctl "a protected result is never returned without a token request"
      (AG (resourceReturned --> previously tokenRequested))
  ]
```

10.9.1 Who knows the password?

This OAuth instance applies the trace-derived knowledge closure from Section 12.5:

```
def passwordKnowers (trace : Trace WireEvent) : List Actor :=
  allActors.filter fun actor =>
    Derives (knowledgeFrom actor trace) (value password)

theorem oauth_password_knowers
    (trace : Trace WireEvent)
    (accepted : (oauthSystem resource password).Accepts trace)
    (initial : InitiallyKnown password [.browser, .authorizationServer])
    (honest : Uncompromised [.browser, .authorizationServer])
    (protection : PerfectProtection protectFor) :
    passwordKnowers trace = [.browser, .authorizationServer] := by
  exact knowledgeClosureOfProtectedField accepted initial honest protection
```

Proposed notation: neither the theorem nor the library lemma it uses exists yet.

The browser knows a typed password; a narrower boundary needs a trusted credential actor, and compromise, logging or failed protection change the result.

10.9.2 Byte-level refinement without choosing a transport

Every vague OAuth message can be assigned a canonical byte term before HTTP or protobuf is chosen:

```
refineMessage "authorization requested" where
  term := tuple
    [ tagBytes 0x01
    , namedBytes "state" (encode state)
    , namedBytes "redirectUri" (encode callback)
    ]
```

```
  bytes := encodeCanonical term

refineMessage "token requested" where
  term := tuple [tagBytes 0x02, namedBytes "code" (encode code)]
  bytes := encodeCanonical term
```

An HTTP mapping might put these bytes in form fields, a protobuf mapping in numbered fields; either must prove that decoding and re-encoding preserve the terminal and its bindings. `previously` is a trace predicate for now.

10.10 Probabilities and charts

The randomness belongs to the user, who grants or denies consent; everything else reacts deterministically. As written, though, the server emits both grant and error, so by the deciding-actor rule of Chapter 2 the server chooses. A faithful model adds a consent event from the browser, `ConsentGiven` or `ConsentRefused`, making the user's choice observable. Weights live in an overlay (Chapter 3); the proposed `weightedOneOf` writes them inline:

```
def consentDecision : Scenario :=
  weightedOneOf "consent decision"
    [ weighted (87 / 100) authorizationGranted
    , weighted (13 / 100) authorizationDenied ]

def oauthCharts : List ChartSpec :=
  [ pie "authorization outcomes" do
      source events; groupBy terminalOutcome; value count
  , line "end-to-end authorization latency" do
      source (correlate events by "state")
      x clock; y (elapsed "authorization requested" "application response")
  ]
```

The branches then begin with different consent bytes, and every later transition may have probability one: the grant branch carries mass 0.87 through queues, transport and the deterministic response. Deterministic processing transports probability mass; it does not remove it.

For the simplified path with no loss or timeout,

$$P(\text{grant callback}) = 0.87 \cdot 1 \cdot 1 \cdot 1 = 0.87, \qquad P(\text{denial callback}) = 0.13.$$

Each factor of one is a deterministic step; dwell affects latency, not mass. Scheduling (which enabled actor runs next) remains nondeterministic, so the composed system is an MDP, analysed under a fixed or quantified policy (Chapter 3). With several sessions in flight, its state holds every actor's queues and per-`(session, task)` residuals, since one user's choice changes queue pressure for others. The percentages are inputs or measurements; the grammar says only what is possible.

10.11 Residual state machine

The automaton is behavioural: cookies, database rows, threads and cryptographic state are refinements of this contract, not part of it.

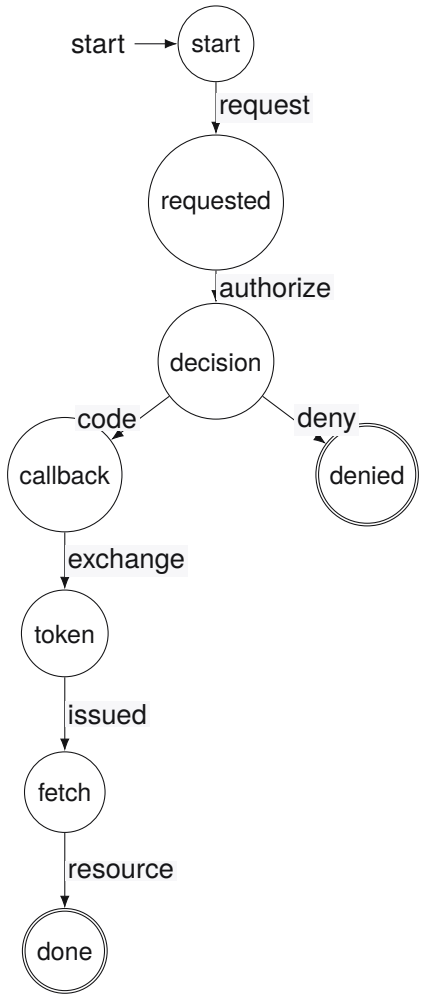


Figure 10.4: Illustrative OAuth state machine, drawn by hand. The full machine also contains the rejected-authentication branch.

Chapter 11

A Legal Example: Lawful, Right, Known, Required

Section 12.5 needed two questions beyond temporal logic: *who knows which values*, and *who may hold them*. Law presses harder on the same machinery: obligations have deadlines, are triggered by what someone knew, create new ones when broken, and two codes can disagree. This chapter works one small example end to end; its tables and diagram are generated by `LegalDepositExample.lean` from the event histories.

The statute below is invented, and so is the moral code. Neither describes any real jurisdiction, and nothing here is legal advice.

11.1 Three meanings of "legal"

LeanFM already calls histories the global type admits *legal*. Two further meanings are needed, and conflating them is the commonest error in normative reasoning about software.

Term	Judged against	Operators
admissible	the protocol grammar: what can happen at all	CTL A, E, F, G over generated futures
lawful / unlawful	a declared statute's ideal futures	O_a^{Act}, P_a^{Act}
right / wrong	a declared moral code with a named author	O_a^{Code}, P_a^{Code}

Here "admissible" means what the grammar allows. An unlawful act is usually admissible, since people can break the law, so $EF\ unlawful$ is normally true: statutes constrain which futures are *ideal*, not which are *possible*. A moral code uses the same machinery with another relation. LeanFM never decides what is right; it checks histories against a code someone wrote and signed, and shows where codes disagree.

11.2 The Model Tenancy Deposit Act and a declared code

The actors are a tenant T, a landlord L, a deposit registry, the post and a court. Writing $Ideal_a^N(h)$ for the futures that code N regards as ideal for actor a after history h, the obligation and permission

operators of the previous chapter are indexed by the code:

$$O_a^N\psi(h) \iff \forall\pi \in Ideal_a^N(h).\ \psi(\pi), \qquad P_a^N\psi(h) \iff \exists\pi \in Ideal_a^N(h).\ \psi(\pi).$$

Prohibition is obligation of the negation: $Forbidden_a^N\psi \equiv O_a^N\neg\psi$. $F_{\leq d}$ is a bounded path operator: within d days.

Act §1 (protection). Within 30 days of receiving a deposit, the landlord registers it.

$$AG\,(paid \to O_L^{Act}\,F_{\leq 30}\,protected)$$

Act §2 (return, triggered by knowledge). Once the lease has ended *and the landlord knows the tenant's forwarding address*, the landlord returns the deposit or itemises deductions within 14 days.

$$AG\,((ended \wedge knows(L, address)) \to O_L^{Act}\,F_{\leq 14}\,(returned \vee itemised))$$

Act §3 (reparation). If §2 is violated, the landlord pays the deposit plus an equal penalty.

$$AG\,(violated_{\S 2} \to O_L^{Act}\,F\,penaltyPaid)$$

Act §4 (prohibition). A deposit is never applied to rent.

$$O^{Act}\,G\,\neg offset \;\equiv\; Forbidden^{Act}\,F\,offset$$

Code (declared by the stakeholder). (a) Never charge for normal wear and tear. (b) A landlord who holds another way to reach the tenant makes a reasonable effort to use it. (c) Help a tenant in documented hardship when it costs nothing you are owed.

11.3 Knowledge from the causal past

Act §2 depends on what the landlord knew, which LeanFM computes: an actor knows a value at an event when a message carrying it reached the actor somewhere in the event's causal past:

$$knows(a, v)@e \iff \exists e' \in past(e) \cup \{e\}.\ dst(e') = a \wedge v \in derive(msg(e'))$$

This is *possession* knowledge, the kind security protocols need (the MCP `knows` tool decides it); K_a, over histories a cannot tell apart, is for claims like "the landlord could not have known".

In the *lost in the post* history the tenant posts the address and the post never delivers it. She records that she *believes* §2 was violated, and is wrong: the address never enters the landlord's causal past, so §2 never triggers. Belief is an event and is not factive; computed knowledge is. The history is still *wrong* under Code (b): the landlord had the tenant's email from the lease (e1, in every causal past) and never used it.

Id	Day	From → To	Message	Detail	prevIds	causeIds
e1	0	Landlord → Tenant	`LeaseSigned`	rent 1000, deposit 1000, tenant email on file		
e2	1	Tenant → Landlord	`DepositPaid`	1000	e1	e1
e3	12	Landlord → Registry	`DepositProtected`	scheme ref 7731	e2	e2
e4	365	Tenant → Landlord	`LeaseEnded`	keys returned	e2	
e5	366	Tenant → Post	`AddressPosted`	letter to landlord	e4	
e6	400	Tenant → Tenant	`BelievesViolated`	no refund after 34 days	e5	

Figure 11.1: The *lost in the post* history. No event delivers `ForwardingAddress` to the landlord, so $knows(L, address)$ never holds and Act §2 is never triggered.

11.4 Violation and reparation

A naive deontic logic breaks on the late return: after day 380 the landlord seems obliged both to have returned the deposit and to pay for not having done so. This is the contrary-to-duty problem, Chisholm's paradox. Ideal futures computed *from the current history* dissolve it: before day 380 every ideal future has a timely return; after it, none is admissible, so the ideal futures are the best remaining ones, in which the penalty is paid. The obligations change as the history grows without becoming inconsistent, and the checker reports *fulfilled*, *violated*, or *violated, then repaired*.

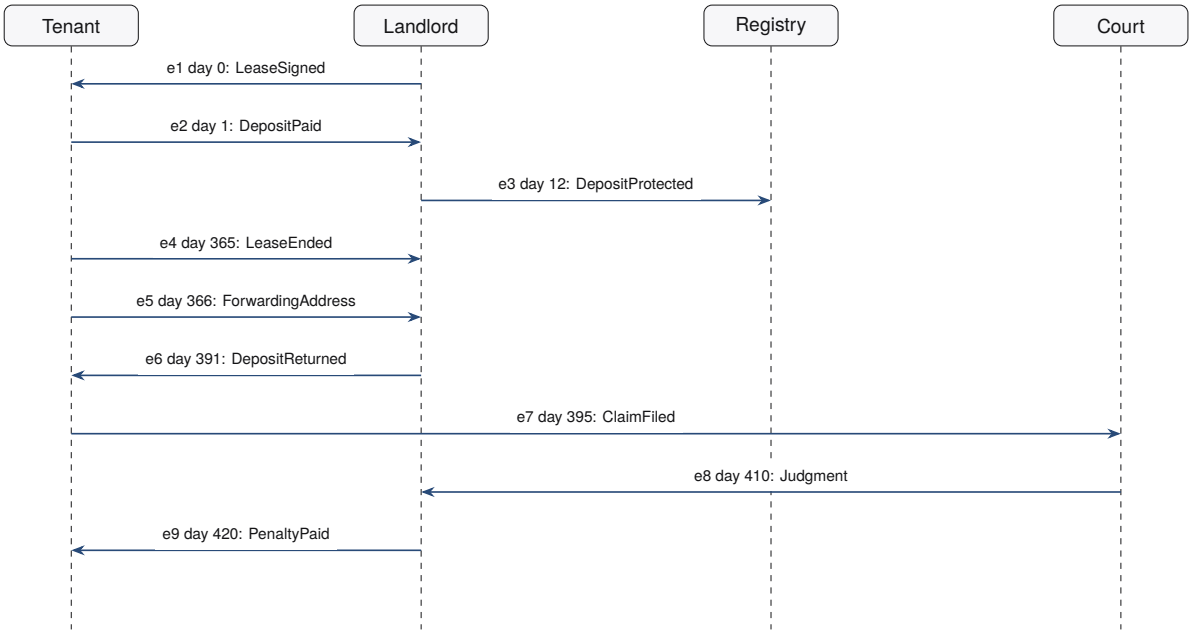


Figure 11.2: The *late return* history, rendered by `MDPDiagram.renderInteractionTeX` from its events. The address reaches the landlord on day 366, so the §2 deadline is day 380; the return on day 391 violates it, and the penalty on day 420 repairs it.

Id	Day	From → To	Message	Detail	prevIds	causeIds
e1	0	Landlord → Tenant	`LeaseSigned`	rent 1000, deposit 1000, tenant email on file		
e2	1	Tenant → Landlord	`DepositPaid`	1000	e1	e1
e3	12	Landlord → Registry	`DepositProtected`	scheme ref 7731	e2	e2
e4	365	Tenant → Landlord	`LeaseEnded`	keys returned	e2	
e5	366	Tenant → Landlord	`ForwardingAddress`	12 Elm St	e4	
e6	391	Landlord → Tenant	`DepositReturned`	1000	e5, e3	e5
e7	395	Tenant → Court	`ClaimFiled`	Act s2 deadline day 380 missed	e6	e6
e8	410	Court → Landlord	`Judgment`	pay penalty 1000 under s3	e7	e7
e9	420	Landlord → Tenant	`PenaltyPaid`	1000	e8	e8

Figure 11.3: The same history as events. `causeIds` pair each response with its cause (the return answers the address; the judgment answers the claim); `prevIds` order each actor's own events.

11.5 Lawful, right, and the four quadrants

Table 11.1 evaluates every rule in every history, and table 11.2 summarises each history under each code. Keeping the codes separate shows every combination:

- *lawful but wrong*: an on-time return that deducts for normal wear (Code a), and the lost address the landlord could have chased (Code b);
- *violated, then repaired*: the late return, under the Act, with no moral duty arising;
- *unlawful but right*: the landlord who, on a laid-off tenant's request, applied the deposit to the final month's rent. That breaks Act §4 and fulfils Code (c).

History	Act §1	Act §2	Act §3 (CTD)	Act §4	Code (a)	Code (b)	Code (c)
Itemised deduction on time	fulfilled	fulfilled	not triggered	fulfilled	violated	not triggered	not triggered
Returned late, penalty paid	fulfilled	violated, then repaired	fulfilled	fulfilled	not triggered	not triggered	not triggered
Address lost in the post	fulfilled	not triggered	not triggered	fulfilled	not triggered	violated	not triggered
Hardship: deposit used as last rent	fulfilled	fulfilled	not triggered	violated	not triggered	not triggered	fulfilled

Table 11.1: Every rule in every history, generated by `LegalDepositExample.lean`. §3 is a contrary-to-duty rule: it is triggered only on histories where §2 was violated.

The hardship history is a genuine dilemma: the Act requires $G\,\neg \mathit{offset}$ and the Code requires $F\ \mathit{offset}$, so no future is ideal under both. LeanFM reports the conflict:

$$\mathit{Conflict}(N_1, N_2, a, \psi, h) \iff O_a^{N_1}\psi(h) \wedge O_a^{N_2}\neg\psi(h).$$

It does not resolve it. Choosing law over conscience, or the reverse, is a stakeholder decision, recorded as an explicit ordering in the typed requirements.

11.6 What CTL says, and what it does not

A and E range over admissible futures, so they cannot express "ought", but they carry most of the weight once normative facts are computed as state predicates, as knows, allows and $\mathit{requires}$ are

History	Under the Act	Under the Code	Landlord knows address from	Normative conflict
Itemised deduction on time	lawful	wrong	day 366	none
Returned late, penalty paid	violated, then repaired	no duty arose	day 366	none
Address lost in the post	lawful	wrong	never	none
Hardship: deposit used as last rent	unlawful	right	day 366	$O_L^{Act}\, G\, \neg offset$ (Act §4) and $O_L^{Code}\, F\, offset$ (Code c): no future is ideal under both

Table 11.2: Each history under the Act and under the declared Code, when the landlord first knew the forwarding address, and any normative conflict.

(Section 12.5):

- A deposit registry's own conduct is ordinary safety: $AG\,\neg(receiptIssued \wedge \neg protected)$.
- Remedies must stay available. Following *possibility means preparedness*, a violation requires a supported path to a claim: $AG\,(violated_{\S 2} \rightarrow EF\; claimFiled)$, with a handling plan in the requirements.
- The *late return* history is simultaneously a *counterexample* for the universal claim that §2 is always met and a *witness* for $EF\; repaired$. Checker output should name both roles.
- Knowledge never exceeds permission, the security invariant, has the same shape here: a registry must not disclose the tenant's address beyond those permitted, $AG\,(knows(a, address) \rightarrow permitted^{Act}(a, address))$.

CTL does not provide per-code quantification over ideal futures, deadlines from a knowledge-triggered start, or the contrary-to-duty recomputation; the example computes those over each history.

11.7 Computing what is owed: Catala

The Act mixes two questions: *what* the law computes from the facts (amounts, deductions, penalties), and *when* duties arose, deadlines passed, and violations were repaired. This chapter answers only the second. The first is the territory of Catala [16], a language for literate law whose default logic writes "the rule is X, except when Y, except when Z" directly, which a priority between codes here handles only crudely. Its formally specified date library is what deadlines such as "within one month" of 31 January need before $AF_{\leq 14\,\mathrm{days}}$ can be trusted on a real statute. The two compose: Catala computes what is owed, and the grammar and CTL check who owed what by when. A verdict stays conditional on its premises: the governing code is chosen, open-textured terms such as "reasonable effort" are atoms a person supplies, and the facts are only as good as the history.

11.8 Status

`LegalDepositExample.lean` is a book example that hard-codes four rules and three code clauses, not a general normative checker. A language feature would need a typed normative code: its author; each rule's actor, trigger (a state formula that may use $knows$), deadline, path condition and reparation; and an optional priority, with a status per rule per history and its trace. Regenerate with `lake env lean -run LegalDepositExample.lean`; provenance is in `book/generated/legal-provenance.txt`.

Chapter 12

Protocol Exercises: Roles, Attackers, and Knowledge

The author learned from Ryan et al. [19] to treat a security protocol as communicating roles plus an active intruder, and security as properties of every trace that intruder can produce. This chapter keeps that idea and none of that book's examples. Three small invented protocols each press on one part of the method:

1. an *on-call handoff*, where three separately written roles must compose into one protocol;
2. a *sealed-bid auction*, where an attacker who cannot read a bid can still copy it; and
3. *speaker pairing*, where a code shown on a screen is visible to some actors and not to others.

Status: the three examples are a design exercise. Their `RoleSpec`, `WireTerm` and `composeRoles` listings are proposed notation, and there is no role-composition engine yet. The attacker's deduction rules are implemented (`LeanFM.Session.Knowledge`), and so is a checker that generates the attacker from a declaration (`LeanFM.Session.Attack`), which Section 12.8 uses on a real protocol.

12.1 Roles compose by sharing events

This is the session-type picture seen from the other side: each role is written first, as a grammar over its own view of the events, and the global protocol is whatever their composition admits. Composition synchronizes on shared events; it does not concatenate scripts. The attacker is the role session types usually leave out: one that controls the channels and can read, drop, replay or forge what passes through them, the most hostile physics a channel can declare (Chapter 1).

```
structure RoleSpec where
  actor    : Actor
  accepts  : Grammar (ProjectedEvent actor)
  initialKnowledge : Knowledge actor

def composeRoles (roles : List RoleSpec) : GlobalGrammar :=
  synchronize roles where
    sendReceiveAgree := sameEventId && sameBytes && oppositeEndpoints
```

For a global trace t, let $t \restriction_a$ keep only the events actor a sends or receives. The composition accepts a trace exactly when every role accepts its own view of it:

$$L(\mathrm{composeRoles}(R_1, \ldots, R_n)) = \{t \mid \forall i,\ t \restriction_{a_i} \in L(R_i)\}.$$

A message is one global event with one identity, one byte string, one sender and one recipient, and both endpoint grammars constrain it. An unmatched send, an unexpected receive, or disagreement about the bytes empties the composed language from that prefix on, and the checker reports the prefix. A role may bind values from events it has seen and test later events against them; the bindings are a parser environment for the trace, not a claim about implementation memory.

12.2 A transport-neutral byte algebra

Security claims need to say which bytes expose which values, without choosing HTTP, protobuf or a cipher suite. Four symbolic constructors are enough for this chapter:

```
abbrev Bytes := ByteArray

inductive WireTerm where
  | atom    : Name -> Bytes -> WireTerm
  | tuple   : List WireTerm -> WireTerm
  | encrypt : KeyRef -> WireTerm -> WireTerm
  | hash    : WireTerm -> WireTerm

structure WireEvent where
  timeAt   : ClockTimestamp
  id       : EventId
  priorIds : List EventId
  session  : SessionId
  task     : TaskId
  from     : Actor
  to       : Actor
  channel  : Channel
  term     : WireTerm
  bytes    : Bytes
  encodedBy : bytes = encodeCanonical term
```

Anyone may hash a term they know; nobody may invert a hash; decryption needs the key. `channel` records who can observe an event at all, which the pairing example needs. A concrete refinement replaces this idealized algebra with real transports and cryptography whose assumptions justify it.

12.3 Example 1: an on-call handoff

A pager service records who is on call for a team. Exactly one engineer is on call at any time. Olu is on call and wants to hand off to Ines. The pager must not move the duty until Ines has actually accepted it, and Olu must learn that the handoff completed. Each person shares a long-term key with the pager: K_O for Olu, K_I for Ines.

1. Olu $\rightarrow$ Pager: $\{Ines, h\}_{K_O}$, with a fresh handoff id h.
2. Pager $\rightarrow$ Ines: $\{Olu, h, c\}_{K_I}$, with a fresh challenge c.
3. Ines $\rightarrow$ Pager: $\{h, c, accept\}_{K_I}$.
4. Pager $\rightarrow$ Olu: $\{Ines, h, done\}_{K_O}$.

12.3.1 Three roles, written separately

```
def oluRole (olu ines pager : Actor) (ko : SharedKey) : RoleSpec :=
  role olu "outgoing engineer" do
    let h <- fresh Nonce "h"
    from_ olu; to_ pager
    send (encrypt ko.ref (tuple [actor ines, nonce h]))
    from_ pager; to_ olu
    receive (encrypt ko.ref (tuple [actor ines, nonce "h", atom "done"]))
      where_ field "h" == h

def inesRole (olu ines pager : Actor) (ki : SharedKey) : RoleSpec :=
  role ines "incoming engineer" do
    from_ pager; to_ ines
    receive (encrypt ki.ref (tuple [actor olu, nonce "h", nonce "c"]))
    let h <- take Nonce "h"
    let c <- take Nonce "c"
    from_ ines; to_ pager
    send (encrypt ki.ref (tuple [nonce h, nonce c, atom "accept"]))

def pagerRole (olu ines pager : Actor) (ko ki : SharedKey) : RoleSpec :=
  role pager "pager service" do
    from_ olu; to_ pager
    receive (encrypt ko.ref (tuple [actor "next", nonce "h"]))
      where_ field "next" == ines
    let h <- take Nonce "h"
    let c <- fresh Nonce "c"
    from_ pager; to_ ines
    send (encrypt ki.ref (tuple [actor olu, nonce h, nonce c]))
    from_ ines; to_ pager
    receive (encrypt ki.ref (tuple [nonce "h", nonce "c", atom "accept"]))
      where_ field "h" == h && field "c" == c
    from_ pager; to_ olu
    send (encrypt ko.ref (tuple [actor ines, nonce h, atom "done"]))
```

The `where_` clauses are the context-sensitive part: the message sequence is context-free, but whether a reply is legal depends on values bound earlier in the trace.

12.3.2 What composition catches

Suppose a later edit lets the pager send *done* as soon as it has sent the offer, before Ines replies. Olu's role still accepts that order: Olu never sees messages 2 and 3. Ines's role still accepts it too. Only the composed protocol exposes the problem, through the duty properties below:

$$AG\ exactlyOneOnCall.$$

$$onCall(Olu)\ AW\ accepted(Ines, h).$$

$$AG\big(toldDone(Olu, h) \rightarrow \mathbf{P}\ accepted(Ines, h)\big).$$

The second formula uses weak until on purpose: Ines may never answer, and then Olu simply remains on call. Strong AU would wrongly demand that every handoff eventually succeeds. The third is a past claim ($\mathbf{P}$ reads "previously"). With the faulty edit, the checker returns a counterexample trace in which Olu is told *done* with no earlier acceptance, drawn both as the global interaction diagram and as Olu's local view, in which nothing looks wrong.

The same mechanism catches disagreement about values. If the pager's grammar could forward the offer to someone other than the engineer Olu named, the `where_ field "next" == ines` guard and Ines's projection cannot both accept the same event, and the composed language is empty at that prefix.

12.4 Example 2: a sealed-bid auction

An auctioneer collects bids in two phases. While bidding is open, each bidder sends a *commitment*: the hash of the amount and a fresh random salt. After bidding closes, each bidder reveals the amount and salt, and the auctioneer accepts a reveal only if it hashes to the commitment. The highest revealed bid wins. Ana and Ben bid honestly. Ivo is also a registered bidder; he is the attacker, and he sees every message on the network.

```
def bidderRole (b auctioneer : Actor) (amount : Amount) : RoleSpec :=
  role b "bidder" do
    let s <- fresh Nonce "salt"
    from_ b; to_ auctioneer
    send (tuple [actor b, hash (tuple [value amount, nonce s])])   -- commit
    from_ auctioneer; to_ b
    receive (atom "closed")
    from_ b; to_ auctioneer
    send (tuple [actor b, value amount, nonce s])                  -- reveal

def auctioneerRole (auctioneer : Actor) (bidders : List Actor) : RoleSpec :=
  role auctioneer "auctioneer" do
    let commits <- collectWhileOpen fun b =>
      receive (tuple [actor b, ?commitment]) where_ b in bidders
    broadcast (atom "closed")
    forEach commits fun (b, commitment) =>
      receive (tuple [actor b, ?amount, ?salt])
        where_ hash (tuple [amount, salt]) == commitment
    announce (highestRevealed commits)
```

12.4.1 Secrecy holds, and the auction is still broken

Before close, Ivo has seen Ana's commitment but cannot invert the hash, so he does not know her amount. That secrecy property holds:

$$\neg\,\mathrm{knows}(Ivo, a_{Ana})\ AU\ closed.$$

The auction is still broken. Ivo does not need to know Ana's amount to bid it. He copies her commitment and sends it under his own name. That is a legal attacker move: he observed the hash term, and resending a known term is always derivable. After close he waits for Ana's reveal, which he also observes, and sends the same amount and salt under his name. The auctioneer's check passes, because the hash really does match. Ivo has tied Ana's bid without ever having known it while bidding was open.

The property that fails is about who *formed* each commitment, not about who knows what:

$$AG\big(\mathrm{acceptsCommit}(Auctioneer, b, x) \rightarrow \mathbf{P}\ \mathrm{formed}(b, x)\big).$$

The checker's search for a violation, $EF(\mathrm{acceptsCommit}(\ldots) \wedge \neg\mathbf{P}\,\mathrm{formed}(\ldots))$, returns the copy trace above as its witness.

12.4.2 The fix: bind the bidder into the commitment

Put the bidder's own name inside the hash: $hash(b, amount, s)$, and have the auctioneer recompute the hash with the name of the bidder who sent the reveal. Ivo's copied commitment was formed with Ana's name. To open it under his own name he would need a salt that makes $hash(Ivo, \ldots)$ equal $hash(Ana, \ldots)$, and no rule derives one. Copying is now useless.

This is the acceptance test for an intruder model: it must find the copy trace for the original commitment and none for the fixed one. A model that cannot find the first attack is too weak to trust when it finds nothing in the second.

The symbolic model has a visible limit here. If amounts came from a small set, Ivo could hash each candidate and compare; the salt stops that. Dolev–Yao deduction has no notion of guessing, so "salts are fresh and unguessable" must be written down as an assumption.

12.5 Knowledge and agreement as theorems

The attacker is one more role. It observes the events its channels let it see, may block or replay them, and may send any term it can derive from them.

```
inductive Derives : Knowledge -> WireTerm -> Prop where
  | initial : term in knowledge -> Derives knowledge term
  | tuple   : (forall x in xs, Derives knowledge x) ->
              Derives knowledge (.tuple xs)
  | project : Derives knowledge (.tuple xs) -> x in xs ->
              Derives knowledge x
  | encrypt : Derives knowledge key -> Derives knowledge body ->
              Derives knowledge (.encrypt key body)
  | decrypt : Derives knowledge (.encrypt key body) ->
```

```
              Derives knowledge (inverse key) -> Derives knowledge body
  | hash    : Derives knowledge x -> Derives knowledge (.hash x)
  -- no rule recovers x from .hash x

def intruderRole (ivo : Actor) : RoleSpec :=
  adversary ivo do
    observe (channelsVisibleTo ivo)
    mayBlock
    mayReplay
    maySend fun knowledge term => Derives knowledge term
```

Knowledge is calculated for every actor from every accepted trace, not only the intended run, so security claims become theorems over accepted traces, derived knowledge and explicit compromise assumptions. For the fixed auction:

```
theorem auction_bid_sealed_until_close
    (trace : Trace WireEvent)
    (accepted : fixedAuction.Accepts trace)
    (fresh : SaltsFreshAndUnguessable trace) :
    forall b x, beforeClose trace x ->
      not (Derives (knowledgeAt .ivo trace x) (value (amountOf b))) := by
  exact hashHidesSaltedInput accepted fresh

theorem auction_commitments_not_copyable
    (trace : Trace WireEvent)
    (accepted : fixedAuction.Accepts trace) :
    forall b c, acceptsCommit trace b c -> formedBy trace b c := by
  exact boundNameDefeatsCopy accepted
```

The two theorems are still proposed notation: the lemmas they name do not exist yet. The deduction rules do. `LeanFM.Session.Knowledge` defines them as `Derives`, over atoms, symmetric keys, key pairs, pairs, encryption and hashes; a ciphertext under a private key is a signature, opened with the public key. It decides them one value at a time: `analz` takes what an actor has seen apart until nothing new appears, and `synth` asks whether the value can be built from the pieces. (Enumerating everything an actor could build would never end.) The decision is proved sound:

```
theorem knows_sound {K : List Msg} {v : Msg}
    (h : knows K v = true) : Derives K v
```

So when the checker says the attacker knows a value, a derivation exists. Completeness, that `knows` misses nothing `Derives` allows, is tested on examples but not yet proved, and secrecy verdicts lean on it.

12.5.1 Calculated knowledge and policy inside CTL

Knowledge and policy need not start as new modal operators. Define state predicates by reducing the event history, then use them inside ordinary CTL. `knows` says who can derive a value; `allows` and `requires` say what a declared policy permits and obliges:

```
def knows (actor : Actor) (value : Secret) (s : ProtocolState) : Bool :=
  value in knowledgeClosure actor s.events s.compromise
```

```
def allows (policy : Policy) (actor : Actor)
    (action : Action) (s : ProtocolState) : Bool :=
  policy.allows actor action s

def requires (policy : Policy) (actor : Actor)
    (action : Action) (s : ProtocolState) : Bool :=
  policy.requires actor action s
```

These functions label a state; CTL says when the labels may or must hold:

$$\neg\,\mathrm{knows}(\mathit{Auctioneer}, a_b)\ AU\ \mathit{closed}.$$

$$EF\ \mathrm{knows}(\mathit{Auctioneer}, a_b).$$

$$AG\big(\mathrm{knows}(x, c) \to \mathrm{allows}(\mathit{DutyPolicy}, x, \mathrm{possess}(c))\big).$$

$$AG\big(\mathit{dutyMoved}(h) \to (\mathrm{requires}(\mathit{DutyPolicy}, \mathit{Pager}, \mathrm{notify}(\mathit{Olu}, h)) \wedge AF\ \mathit{toldDone}(\mathit{Olu}, h))\big).$$

A universal claim is checked by searching for its violation, and the search result is the useful artifact:

$$AG(\mathrm{discloses}(a, s) \to \mathrm{allows}(P, a, \mathrm{disclose}(s))) \ \equiv\ \neg EF(\mathrm{discloses}(a, s) \wedge \neg\,\mathrm{allows}(P, a, \mathrm{disclose}(s))).$$

This state-local `knows` is not the epistemic modality $K_a\varphi$ over indistinguishable histories of Chapter 4. It tests whether one deduction closure contains a value, which suffices for most secrecy and authorization claims; K_a is for requirements that compare histories, as the next example shows.

12.6 Example 3: pairing a speaker by a code on its screen

A new speaker shows a six-digit code d on its own display. Its owner types the code into a phone, and the phone proves to the speaker that it knows d. The network is hostile; the display is not on the network.

1. Speaker $\to$ Owner, on channel *display*: d.
2. Owner $\to$ Phone, on channel *keypad*: d.
3. Phone $\to$ Speaker, on channel *network*: $p,\ \mathit{hash}(p, d, \mathit{Phone})$, with a fresh nonce p.
4. Speaker $\to$ Phone, on channel *network*: *paired* or *rejected*.

```
receive (tuple [nonce "p", ?proof]) on network
  where_ sender != speaker
  where_ proof == hash (tuple [nonce p, value shownCode, actor sender])
  where_ failedAttempts < 3
```

Three things are new compared with the first two examples.

Channels decide what each actor observes. The display and keypad events belong to the observations of the speaker, the owner and the phone, not to Ivo's. That is what keeps d out of Ivo's knowledge closure, and it is a modeling assumption about the room, so it is stated as one. Remove it and the checker should immediately find Ivo pairing his own phone.

A counter is context, too. `failedAttempts < 3` depends on how many earlier proofs failed in this trace. Like the equality guards, it keeps the grammar context-free and moves the counting into a named predicate.

Guessing is a probability, not a deduction. Ivo cannot derive d, but he can guess it. The symbolic model says $EF\, paired(Ivo)$ is true: some future has a lucky guess. That verdict is correct and useless on its own. The requirement is a bound on how likely that future is, carried by a probability overlay: with three attempts against 10^6 codes,

$$P[F\, paired(Ivo)] \leq 3/10^6.$$

The next section explains why the number attaches to E.

Some claims compare histories. "Ivo cannot tell which code was shown" is not true or false of one state: it says any two histories differing only in d look identical to Ivo. That is K_a over indistinguishable histories, a relational theorem rather than a CTL state formula, and the one claim in this chapter that the table below does not cover.

12.7 Where each modality attaches

The examples have used every kind of claim a requirement needs: what can or must happen, how likely, by when, who knows, who may, and who must. None needs a new logic. A CTL operator is a path quantifier, A or E, fused with a temporal operator such as F, G or U: probability belongs to the quantifier, deadlines to the temporal operator, and knowledge and policy are predicates on states.

The path quantifier E invites "how likely?" E says that *some* future satisfies the path formula; the refinement replaces "some" with a probability over futures, whatever follows the E:

$$EF\, p \;\rightsquigarrow\; P[F\, p], \qquad EG\, p \;\rightsquigarrow\; P[G\, p], \qquad p\; EU\; q \;\rightsquigarrow\; P[p\; U\; q].$$

The number comes from an overlay of branch probabilities, as in the bakery chapter, and is reported beside the CTL verdict, never in place of it. The overlay cannot make $EF\,p$ false: a future with probability $3/10^6$ is still possible, and so still needs the handling that possibility demands. Nor does A become "probability one": a set of futures can have probability one and still leave out a future that A must cover, which is why Chapter 4 keeps A and E about legal futures only.

The temporal operator F invites "by when?" F has no bound, and a user waiting on a handoff needs one, so the deadline is written on F (and on the right of U): $AF_{\le t}\,q$ says every future reaches q within t, $EF_{\le t}\,q$ that some future does. A deadline under A is a promise the grammar must keep: every waiting state needs a timeout branch ending in an explicit outcome, or some future waits past t.

$$AF_{\le 15\,\mathrm{min}}\big(accepted(Ines, h) \lor expired(h)\big).$$

$$AG\Big(closed \to AF_{\le T_{reveal}}\big(revealed(b) \lor forfeited(b)\big)\Big).$$

So the handoff needs an *Expired* message to both engineers when the offer times out, and the auction a *Forfeited* outcome for a bidder who never reveals: new legal branches, with decodable messages and code that handles them.

The two annotations compose. Probability on the quantifier and a deadline on F combine into the question stakeholders usually ask: how likely is it to happen in time? An owner pairing a speaker within a minute is

$$P[F_{\le 60\,\mathrm{s}}\, paired(Phone)],$$

which a measured overlay reports from real pairing attempts. Keep a guaranteed deadline ($AF_{\le t}$, enforced by a timeout branch) separate from a usual one ($P[F_{\le t}]$, measured). The bakery chapter shows why: a declared delay promised a 1.2 s sale while only about half the recorded sales met it.

`knows`, `allows` and `requires` complete the set. These are about actors rather than futures, and each is computed from the history up to a state, so the temporal operators carry them unchanged. Secrecy is `knows` under AU; authorization is `knows` implying `allows` under AG; an obligation is `requires` paired with $AF_{\le t}$ on the act that discharges it.

Question	Attaches to	Example from this chapter
can it happen? must it?	E, A	$EF\ paired(Ivo)$
how likely?	E	$P[F\ paired(Ivo)] \le 3/10^6$
by when?	F, U	$AF_{\le 15\,\mathrm{min}}(accepted \lor expired)$
who knows?	state: `knows`	$\neg\,\mathrm{knows}(Ivo, a_{Ana})\ AU\ closed$
who may?	state: `allows`	$AG(\mathrm{knows}(x, c) \to \mathrm{allows}(Duty, x, \ldots))$
who must?	state: `requires`	$AG(dutyMoved \to \mathrm{requires}(\ldots) \land AF\ toldDone)$

12.8 Needham–Schroeder, checked

The public-key protocol of Needham and Schroeder (1978) has three messages:

```
protocol NS (A, B) {
  m1: Msg1 from A to B;
  m2: Msg2 from B to A cause m1 where m2.na = m1.na;
  m3: Msg3 from A to B cause m2;
}
```

Alice runs it once, with a peer she chooses: Bob, or Ivo. Bob answers once, whoever claims to be calling. Ivo is a legitimate user with his own key pair, so Alice may well choose him; he is also the network, seeing every message and sending anything he can derive.

Nobody builds the state space. Each honest role is a short script over message patterns: a choice, sends, receives and claims; a variable in a received pattern binds whatever the message carries there.

```
def alice (v : Variant) : Role := { name := "Alice", steps :=
  [ .choose "peer" [.atom "B", .atom "I"]
  , .claim "running" [name "A", .var "peer"]
  , .send (.var "peer") (.enc (.pk (.var "peer")) (.tuple [.lit na, name "A"]))
  , .recv (.enc (.pk (name "A")) (match v with
      | .original => .tuple [.lit na, .var "n"]
      | .lowe => .tuple [.lit na, .var "n", .var "peer"]))
  , .send (.var "peer") (.enc (.pk (.var "peer")) (.var "n"))
  , .claim "finished" [name "A", .var "peer"] ] }

def bob (v : Variant) : Role := { name := "Bob", steps :=
  [ .recv (.enc (.pk (name "B")) (.tuple [.var "m", .var "x"]))
  , .send (.var "x") (.enc (.pk (.var "x")) (match v with
      | .original => .tuple [.var "m", .lit nb]
      | .lowe => .tuple [.var "m", .lit nb, name "B"]))
  , .recv (.enc (.pk (name "B")) (.lit nb))
  , .claim "commit" [name "B", .var "x"] ] }

def declared (v : Variant) : Protocol :=
  { attacker := "Ivo"
  , attackerKnows := [.atom "A", .atom "B", .atom "I", .pk "A", .pk "B", .pk "I", .sk
     "I", ni]
  , domain := [na, nb, ni]
  , domains := [("x", [.atom "A", .atom "B", .atom "I"])]
  , roles := [alice v, bob v]
  , flags :=
    [ ⟨"SecrecyBreach", fun w => w.claimed "commit" [.atom "B", .atom "A"] && w.knows
     nb⟩
    , ⟨"Impersonation", fun w => w.claimed "commit" [.atom "B", .atom "A"] && !w.
    claimed "running" [.atom "A", .atom "B"]⟩
    , ⟨"HonestRun", fun w => w.claimed "commit" [.atom "B", .atom "A"] && w.claimed "
    finished" [.atom "A", .atom "B"]⟩ ] }
```

`LeanFM.Session.Attack` generates the model. Every message an honest role sends goes to the network, which is Ivo; every message it receives is one Ivo can build in the expected shape. That

is the Dolev–Yao channel, declared once rather than written into each role. A state is each role's position and bindings, Ivo's knowledge (kept taken apart, so `knows` is an ordinary label), and the claims made so far.

Requirements become flags that events trip. `SecrecyBreach` trips when Bob has committed to Alice and Ivo can derive Bob's nonce; `Impersonation` when Bob has committed to Alice although Alice never claimed to be running with Bob. `check_ctl` decides `never` of each, and that an honest run can complete. The last row asks secrecy with `knows` and `claimed` atoms instead of a flag and gets the same verdicts. The same declaration, as JSON, is what the MCP tool `check_protocol` takes:

LeanFM `check_ctl`	1978 (242 states)	Lowe (222 states)
Ivo never learns the nonce Bob shares with Alice	disproved	proved
Bob never commits to Alice unless Alice ran with Bob	disproved	proved
Alice and Bob can complete an honest run	proved	proved
the same secrecy, as `never (claimed commit(B, A) and knows(Ivo, nb))`	disproved	proved

For the 1978 protocol the counterexample is the attack Lowe found in 1995, in the checker's own words:

```
1. Alice chooses peer = I
2. Alice claims running(A, I)
3. Alice -> I: {na, A}pk(I)
4. Bob <- {na, A}pk(B)
5. Bob -> A: {na, nb}pk(A)
6. Alice <- {na, nb}pk(A)
7. Alice -> I: {nb}pk(I)
8. Bob <- {nb}pk(B)
9. Bob claims commit(B, A)
10. SecrecyBreach
```

Alice talks to Ivo, as she meant to. Ivo re-encrypts her first message for Bob, who answers "Alice" with his nonce under Alice's key. Ivo cannot read that answer, but he does not need to: he forwards it, and Alice, who believes it is Ivo's reply, decrypts it and sends Bob's nonce back to Ivo under Ivo's key. Ivo now knows the nonce, and returns it to Bob, who finishes believing he talked to Alice. Lowe's fix adds Bob's name to the second message, and Alice checks it is the peer she chose. With it both properties hold on every reachable state.

The model is bounded, and the bounds are where attacks could still hide: each session runs once, and a variable binds a name or nonce from a declared domain, never a whole ciphertext, so Ivo forwarding a sealed blob he cannot read is out of reach. Lowe's attack needs one session of each role; an attack needing a second run of Bob needs a second session, which a declaration can ask for.

12.8.1 Two sessions: reflection

Some attacks play one session of a role against another. The standard example is a challenge–response with one key, `kAB`, shared by Alice and Bob. Alice challenges with a fresh nonce; Bob answers it under the key and adds his own challenge; Alice answers his. The flag `named` adds a fix:

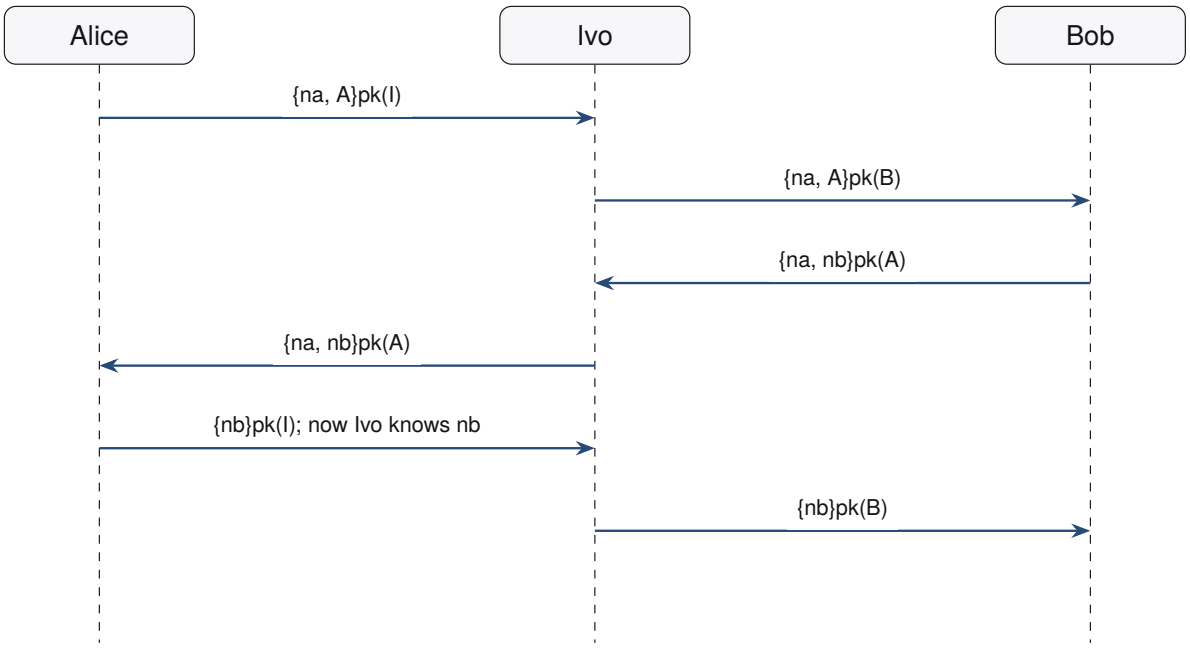


Figure 12.1: Lowe's attack as it crosses the wire: the checker's walk replayed through the generated model, which draws every message through Ivo, because Ivo is the network. The label marks where his knowledge first contains Bob's nonce.

```
/-- With ‘named‘, each answer carries the name of the party who made it. -/
def answer (named : Bool) (x : String) (who : String) : Pat :=
  .enc kAB (if named then .pair (.var x) (name who) else .var x)

def challenger (named : Bool := false) : Role := { name := "Alice", steps :=
  [ .fresh "na"
  , .claim "running" [name "A", name "B"]
  , .send (name "B") (.var "na")
  , .recv (.pair (answer named "na" "B") (.var "m"))
  , .send (name "B") (answer named "m" "A")
  , .claim "finished" [name "A", name "B"] ] }

def responder (named : Bool := false) : Role := { name := "Bob", steps :=
  [ .recv (.var "c")
  , .fresh "nb"
  , .send (name "A") (.pair (answer named "c" "B") (.var "nb"))
  , .recv (answer named "nb" "A")
  , .claim "commit" [name "B", name "A"] ] }

def reflection (bobSessions : Nat) (named : Bool := false) : Protocol :=
  let roles := (challenger named).sessions 1 ++ (responder named).sessions
    bobSessions
  { attacker := "Ivo"
  , attackerKnows := [.atom "A", .atom "B", ni]
  , domain := coverFresh roles [na, nb, ni]
```

```
  , roles
  , flags := [] }
```

A `fresh` step makes a value no other session has and the attacker does not know: `nb` in Bob's first session, `nb#2` in his second, `Bob#2`. The requirement is agreement: Bob never commits to Alice unless Alice started a run with him.

LeanFM `check_ctl`	one Bob (33)	two Bobs (527)	two Bobs, names inside (433)
Bob never commits to Alice unless Alice ran with Bob	proved	disproved	proved
Alice and Bob can complete an honest run	proved	proved	proved

With one session of Bob the property holds: the only party that encrypts under `kAB` on demand is Bob, and he does it before he has made the nonce that would need answering. With two sessions Ivo needs no key at all:

```
1. Bob <- ni
2. Bob makes nb
3. Bob -> A: {ni}kAB, nb
4. Bob#2 <- nb
5. Bob#2 makes nb#2
6. Bob#2 -> A: {nb}kAB, nb#2
7. Bob <- {nb}kAB
8. Bob claims commit(B, A)
```

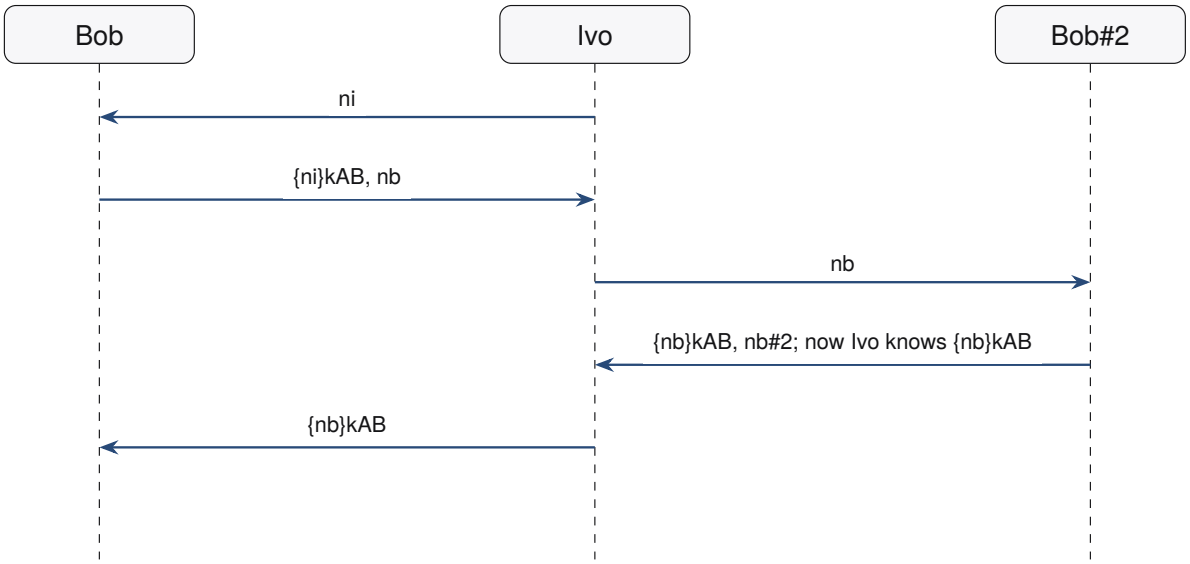


Figure 12.2: Reflection: Bob's own second session answers the challenge of his first. Alice never appears; the label marks where Ivo first holds the answer he could not compute.

Ivo opens a session with Bob as Alice and gets Bob's challenge `nb`, then offers `nb` as the challenge in a second session. Bob's second session encrypts it, as it should, and Ivo hands that answer to the

first, which commits to Alice. Nothing was decrypted; the key was used as an oracle. The usual fixes put the sender's name or the direction inside the encryption, or use a key per direction. With `named`, Bob's second session produces `{nb, B}kAB` while his first waits for `{nb, A}kAB`, which only Alice makes; the table's third column checks the fix with the same two sessions.

12.8.2 A defender on the wire

A real path often has a defender on it too: an intrusion-prevention system, a filtering proxy, a gateway that intercepts TLS. A declaration may put one in front of named roles; every message delivered to them passes through it, and it drops what matches a drop pattern, rewrites what matches a rewrite rule, and forwards the rest. It matches only what it can see, which is `knows` from the defender's side: it looks inside a ciphertext only when it can build the opening key, and a blob it cannot open matches nothing, since real encryption is randomised. Here Bob accepts whatever reaches him, and Guard drops `bad`:

```
def guarded (bobReceives : Json) (defender : Option Json) (formula : Json) : Json :=
  let base := json% {
    roles: [{name: "Bob", steps: [
      {recv: $(bobReceives)},
      {claim: {name: "accept", args: [{atom: "B"}, {var: "x"}]}}]}],
    attacker: {name: "Ivo", knows: [{atom: "B"}, {atom: "good"}, {atom: "bad"}, {pk:
    "B"}]},
    domain: [{atom: "good"}, {atom: "bad"}],
    formula: $(formula) }
  match defender with
  | some d => base.setObjVal! "defender" d
  | none => base

def plain := json% {var: "x"}
def sealedForBob := json% {enc: {key: {pk: "B"}, body: {var: "x"}}}
def filter := json% {name: "Guard", protects: ["Bob"], drop: [{atom: "bad"}]}
def intercepting := json% {name: "Guard", protects: ["Bob"], knows: [{sk: "B"}], drop
    : [{atom: "bad"}]}
```

Bob receives, through	Bob never accepts bad	good still arrives
no defender, plaintext	disproved	proved
filter, plaintext	proved	proved
filter, sealed for Bob	disproved	proved
intercepting filter (holds sk(B)), sealed for Bob	proved	proved

Sealed for Bob, `bad` goes straight past the filter: the counterexample shows Guard itself delivering `{bad}pk(B)`. Give Guard Bob's private key, as an intercepting gateway has, and it drops it again. The table is the argument for and against interception in four rows.

12.8.3 Marking secrets

Computing everything an attacker could derive is the expensive part of these checks, and most of it answers questions nobody asked. Ivo learning Alice's name matters to no one. So a declaration

marks what must stay secret, and the mark is the question: *the attacker never derives this, on any path*. That is `AG not knows(Ivo, s)`, decided once for each marked secret.

There are two ways to mark a secret. A name ending in `_secr` marks an atom or key by name: `passwd_secr`, or a fresh `na_secr`, whose copies in later sessions are secrets too. Any other term, compound ones included, is listed in `secrets`. The distinction matters for hashes. A hash hides its input, so `h(passwd_secr)` reveals nothing about the password, and the hash is not secret merely because its input is. But a server that accepts the hash in place of the password has made the hash a password. It must be marked, and listing `{hash: {atom: passwd_secr}}` does that.

Here is a password login where the client sends the hash of its password, first in clear, then sealed under the server's public key:

Login	Secret	Marked	Verdict
in clear	`passwd_secr`	by name	kept
in clear	`h(passwd_secr)`	listed	leaked
sealed for Server	`passwd_secr`	by name	kept
sealed for Server	`h(passwd_secr)`	listed	kept

`sk(Server)` protects `passwd_secr`, `h(passwd_secr)` (Client, step 1).

Warning: session_token looks secret but is not marked: name it session_token_secr or list it in secrets, or it is not checked.

In clear, the password stays secret and the hash leaks, which is exactly the pass-the-hash attack: Ivo replays what he saw. The leak comes with the path and the message on the wire. Sealed, both are kept, and the answer names what keeps them: `sk(Server)`, because every ciphertext that carries a secret is only as safe as its opening key. A leak of that key would be a leak of the password. The last line is a warning from a one-line declaration that sends a `session_token`. A name that looks secret but carries no mark is reported rather than silently left unchecked, because a forgotten marker would otherwise look exactly like a pass.

The idea is close to Perl's taint mode, from the other side. Taint protects *integrity*: data from outside is marked, the mark follows it, and it may not reach a sensitive operation unchecked. A secret mark protects *confidentiality*: the marked data may not reach the attacker. Both are labels that follow data, and both need a rule for when the label stops: Perl's is matching the data against a pattern, ours is a one-way function, and in both cases the rule is wrong exactly when the result is itself a credential.

A mark asks an unconditional question. In Needham–Schroeder with `nb_secr` marked, the nonce leaks even with Lowe's fix, because Bob may legitimately run with Ivo and tell him. Secrecy that depends on whom a role talks to stays a formula, like `never (claimed commit(B, A) and knows(Ivo, nb))` above. The marks are for values that must never leave, whoever asks: passwords, private keys, the hash a server accepts.

12.8.4 Reassembly

Signature matching has a subtler gap. A defender scanning a TCP stream for `EVIL` must reassemble it as the host behind it will, and the attacker controls segmentation: any byte range, in any order, repeated with different bytes. Where segments overlap, one implementation keeps the first bytes and another the last, and operating systems have differed in exactly this. `check_reassembly` models each stream position as empty, the signature's byte, or another byte (written `.`), applies every segment

to the defender's and the host's copies with their own policies, and resets the stream when a blocking defender matches. The search covers every segmentation:

defender keeps	host keeps	never delivered	evasion
first	last	disproved	`.` at 0, then `EVIL` at 0
last	first	disproved	`E` at 0, then `.VIL` at 0
first	first	proved	—
last	last	proved	—

When the policies differ, two segments are enough: a first-wins defender fed a decoy byte at offset 0 and then the whole signature keeps the decoy and sees `.VIL`, while a last-wins host sees `EVIL`. When they agree, no segmentation separates the copies. The same disagreement makes a host fingerprintable: send overlapping segments and see which bytes it acts on. The fix, as later intrusion-prevention systems did, is to normalize: the defender forwards the bytes it scanned, leaving the host no overlap to resolve differently.

12.9 What these exercises demand from the library

The three examples produce a concrete checklist:

1. actor-local grammar definitions and trace projection;
2. synchronization by common event identity and bytes;
3. typed symbolic variables and prefix-dependent guards, including equalities and counters;
4. a transport-neutral term-to-byte relation with encryption and hashing;
5. channels, so that each actor's observations are declared;
6. explicit initial knowledge, compromise and freshness assumptions;
7. attacker closure under construction, decryption and hashing (done: `LeanFM.Session.Knowledge`, decision proved sound), and an attacker generated from role declarations (done: `LeanFM.Session.Attack`);
8. past operators for agreement claims such as "accepted only if previously formed";
9. probabilities on the E quantifier, from overlays, reported next to CTL verdicts;
10. deadlines on F and U, with timeout branches wherever A must meet them;
11. relational theorem schemas for indistinguishability; and
12. counterexamples drawn both as a global interaction diagram and as each actor's local projection.

None of this is security decoration. It tests whether separately written, context-sensitive role grammars stay modular while still expressing the information flow that real behavioural claims depend on.

Appendix A

The MCP Tools

The LeanFM server speaks the Model Context Protocol at `/mcp`. An agent that connects is given the language reference as a resource and these tools, every one read-only: none changes a file, and none runs submitted text as code. The table is generated from the server's own tool list, so it is the list an agent sees.

Tool	What it does	Arguments (* required)
validate_generated_artifacts	Validate generated artifacts	
requirements_interrogation	Read requirements interrogation	
code_generation_prompt	Read one-shot implementation prompt	
explain_formula	Explain a CTL formula	formula*, sentence, task
check_ctl	Check a CTL formula against a task	formula*, task*
check_trace	Check an event trace and draw it	drafts, events*
classify_requirement	Classify requirement observability	formula, predicates, task
check_protocol	Find attacks on a security protocol	attacker*, defender, domain, domains, formula, roles*, secrets
check_reassembly	Find stream-reassembly evasions	box, formula, host, mode, signature
knows	What an actor can derive	actor*, at*, events*, initial, query, seesAll
session_check	Check a session type	lean, mayCall, notHttp, overlay, protocol
session_diagram	Draw a session type	lean, overlay, protocol, theme
session_mdp	Render a session decision process and terminal probabilities	events, horizon, lean, optimize, overlay, protocol, theme, view, walk
session_sla	Check a session type's service-level requirements	at, events, lean*, overlay, theme
session_simulate	Generate an event stream from a session type	instances, lean, loops, maxEvents, overlay, protocol, seed, spread
session_conform	Check a recorded run against a session type	allowIncomplete, events*, horizonMs, lean, overlay, protocol
session_next	What may happen next in a session	events, lean, overlay, protocol
session_ctl	Check a temporal property of a session type	formula*, lean, overlay, protocol
session_review	Review a session type design	lean, overlay, protocol
session_openapi	A role's HTTP interface as OpenAPI	lean, overlay, protocol, role*
session_types	The message types as protobuf, Lean and JSON schema	lean, protocol
session_compat	Which roles can upgrade on their own	lean, old*, overlay, protocol
session_skeleton	Generate code to implement a session type	language, lean, protocol

The session tools take a protocol as the `lean` argument, the text of Chapter 1. Every answer carries a one-line reading for people and the whole result as structured content, and the server's `/metrics` counts each call by tool and verdict, with its time and model size.

A.1 How a consultation travels

An agent first reads durable context, the book, the canonical requirements and the language references, and then calls tools. Both diagrams are generated from LeanFM's own requirements.

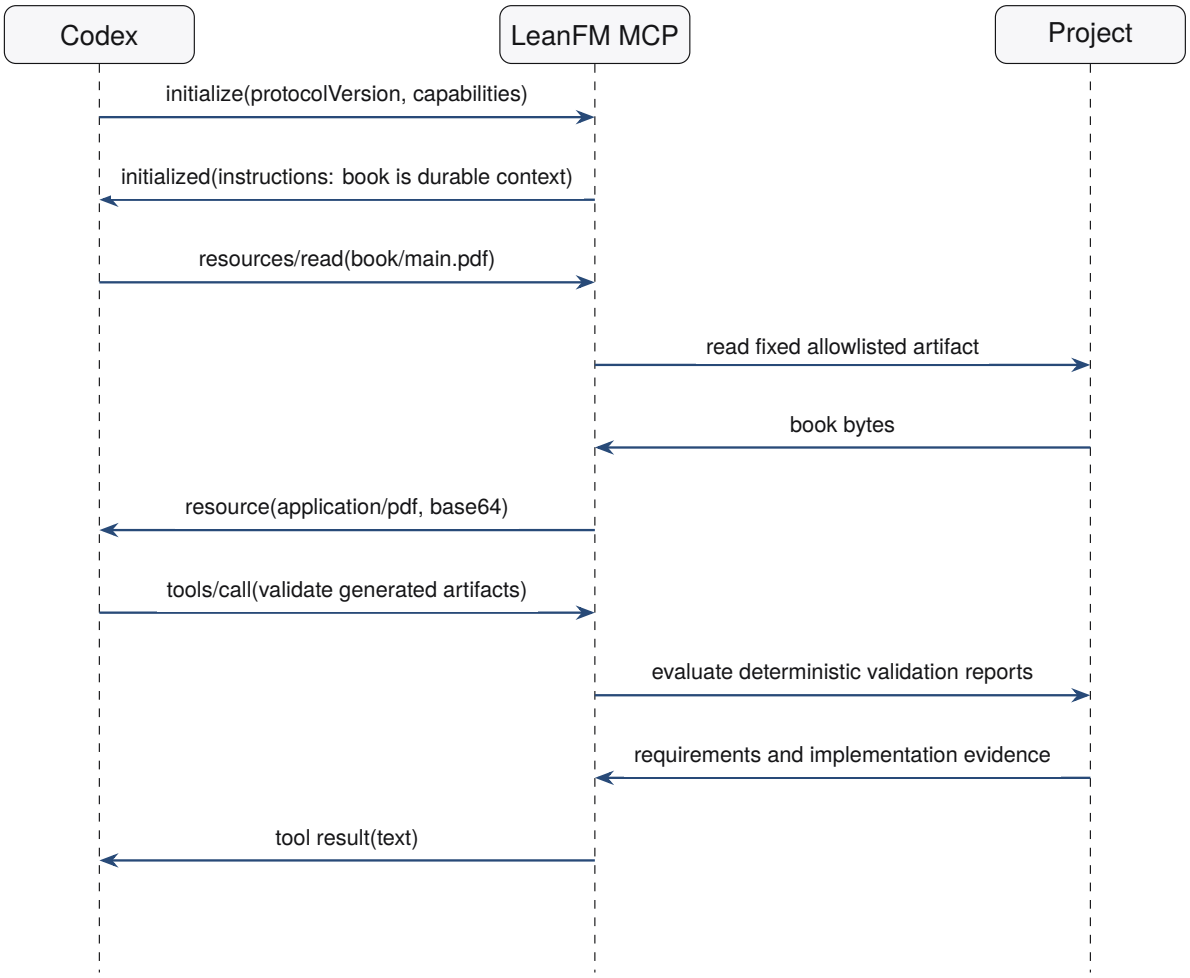


Figure A.1: An agent obtains durable project context and deterministic evidence through MCP. Source: `LeanFM.MCP`; reducer and provenance: `book/generated/mcp-interaction-provenance.txt`.

A logic consultation is one request answered by exactly one of two outcomes: a verdict carrying its evidence, or a rejection naming the input fault.

These requests and responses are produced by running `LeanFM.Logic` while the book is built, so they cannot drift from the implementation:

A never-claim that fails (`check_ctl`)

```
{"task": "mcp_logic_consult",
 "formula": {"never": {"after": "LogicInputRejected"}}}
```

returns

```
{"verdict": "disproved",
```

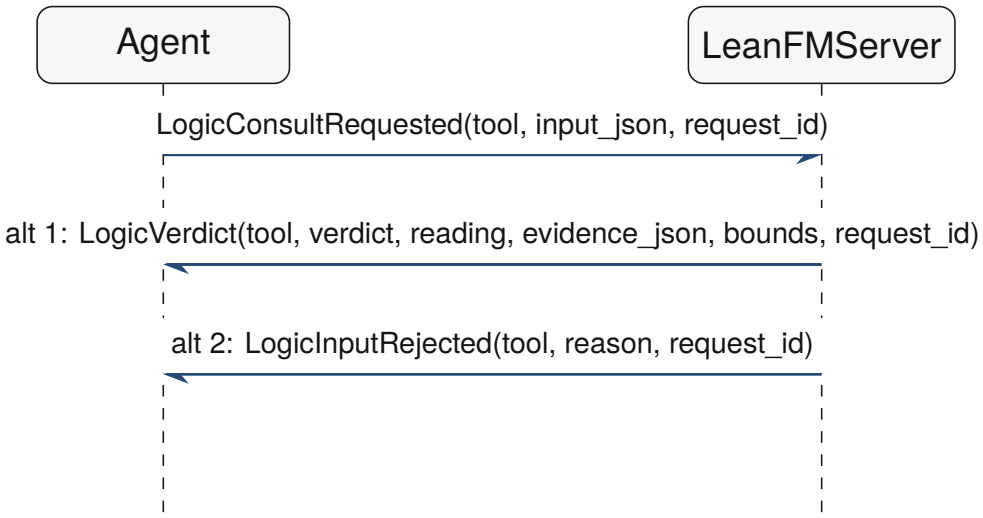


Figure A.2: An agent consults LeanFM about a formula, trace or requirement. Source: `logicGrammar` in the self-hosting requirements; provenance: `book/generated/mcp-logic-interaction-provenance.txt`.


```
 "tool": "check_ctl",
 "task": "mcp_logic_consult",
 "reading":
 "on every path, never LogicInputRejected was just delivered",
 "reachableNodes": 4,
 "mode": "never",
 "formula": {"never": {"after": "LogicInputRejected"}},
 "evidence":
 {"path":
  [{"state": "drafting", "message": null},
   {"state": "interpreting", "message": "LogicConsultRequested"},
   {"state": "rejected", "message": "LogicInputRejected"}],
  "kind": "counterexample"},
 "deadlocks": []}
```


A requirement over hidden state (`classify_requirement`)

```
{"task": "mcp_logic_consult", "predicates": ["the cache is consistent"]}
```

returns


```
{"verdict": "classified",
 "tool": "classify_requirement",
 "reading":
 "some atoms are implementation-only; restate them over proposed events before the
    requirements gate",
 "atoms":
 [{"restated":
   {"AG":
    {"implies":
     [{"after": "CacheConsistentObserved"},
```

```
      {"field":
       {"value": "true",
        "name": "holds",
        "message": "CacheConsistentObserved"}}]}},
   "reason":
   "no declared message or state reveals this; no event trace can test it",
   "proposedEvent":
   {"to": "an observer that can check it",
    "message": "CacheConsistentObserved",
    "from": "the actor that owns this state",
    "fields": ["subject", "holds", "observed_at"]},
   "class": "implementation-only",
   "atom": "the cache is consistent",
   "advice":
   "move it to Implementation.lean as an internal invariant, or restate it over an
    event that exposes it"}]}
```

Designing a sequence diagram by writing events (`check_trace`)

```
{"events":
 [{"task": "mcp_logic_consult",
   "src": "Agent",
   "message": "LogicConsultRequested",
   "id": "e1",
   "dst": "LeanFMServer"},
  {"task": "mcp_logic_consult",
   "src": "LeanFMServer",
   "prevIds": ["e1"],
   "message": "LogicVerdict",
   "id": "e2",
   "dst": "Agent",
   "causeIds": ["e1"]}]}
```

returns

```
{"verdict": "admissible",
 "tool": "check_trace",
 "reading": "2 events over 1 task(s); 0 concurrent pair(s)",
 "issues": [],
 "concurrentCount": 0,
 "concurrent": [],
 "acceptance":
 [{"task": "mcp_logic_consult",
   "states": ["answered"],
   "reachedTerminal": true,
   "accepted": true}]}
```

with the generated Mermaid sequence diagram

```
sequenceDiagram
    participant Agent
    participant LeanFMServer
    Agent->>LeanFMServer: e1 LogicConsultRequested
    LeanFMServer->>Agent: e2 LogicVerdict
```

Who can derive what (`knows`)

```
{"query": {"atom": "nb"},
 "initial": [{"sk": "I"}],
 "events":
 [{"term":
   {"enc":
    {"key": {"pk": "A"},
     "body": {"tuple": [{"atom": "na"}, {"atom": "nb"}]}}},
   "src": "Bob",
   "id": "e1",
   "dst": "Ivo"},
  {"term": {"enc": {"key": {"pk": "I"}, "body": {"atom": "nb"}}},
   "src": "Alice",
   "id": "e2",
   "dst": "Ivo",
   "causeIds": ["e1"]}],
 "at": "e2",
 "actor": "Ivo"}
```

returns

```
{"visible": ["e1", "e2"],
 "verdict": "proved",
 "tool": "knows",
 "reading": "Ivo can derive nb at e2",
 "query": "nb",
 "holds":
 [{"term": "sk(I)", "how": "known at the start", "from": "initial"},
  {"term": "{na, nb}pk(A)",
   "how": "delivered in e1 (Bob to Ivo)",
   "from": "e1"},
  {"term": "{nb}pk(I)",
   "how": "delivered in e2 (Alice to Ivo)",
   "from": "e2"},
  {"term": "nb", "how": "opened {nb}pk(I) with sk(I)", "from": "e2"}],
 "evidence":
 {"kind": "derivation",
  "derivation":
  {"term": "nb", "how": "opened {nb}pk(I) with sk(I)", "held": "e2"}},
 "causalPast": ["e1", "e2"],
 "at": "e2",
 "actor": "Ivo"}
```

The surface is read-only and listens only on loopback. A deployment that exposes it to a network must add authentication and authorization first.

Appendix B

The LeanFM Web-Server Protocol

This is the protocol reference for the LeanFM web server, written mainly for an implementation agent. The behavior is defined by `LeanFM/ChatUI/Requirements.lean`, the bytes by `LeanFM/ChatUI/Requirements.proto`, and the transports by `LeanFM/ChatUI/Implementation.lean`; the tables and grammars below are generated from them. The *accepted protocol* is the complete grammar and intended HTTP mapping; the *running adapter* is the subset `LeanFM/ChatUI/Server.lean` dispatches today.

B.1 One exchange across four representations

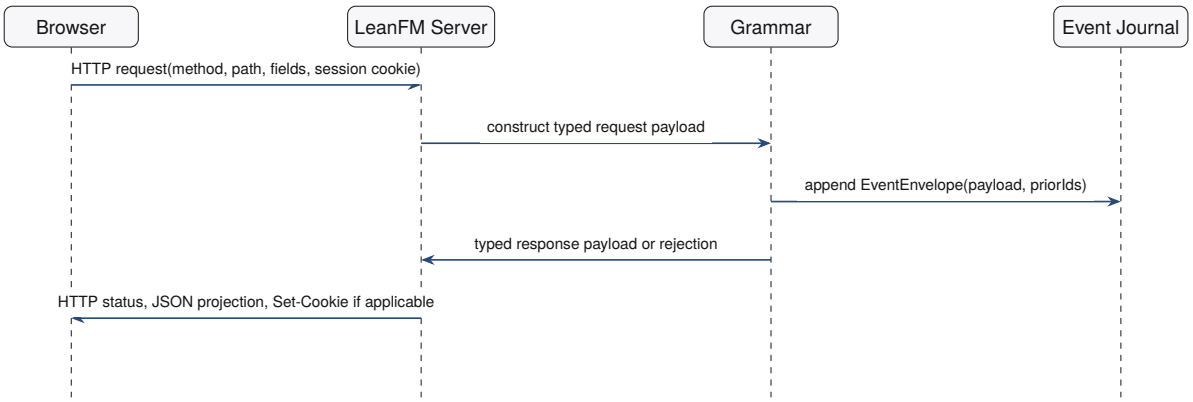


Figure B.1: HTTP is an adapter around the typed LeanFM conversation. Source, reducer, question, and unit are recorded in `generated/web-protocol-provenance.txt`.

HTTP is decoded into a grammar-permitted protobuf value, recorded as an `EventEnvelope`, and the response or rejection is projected back. Provider messages stay in-process. Protobuf fixes values and tags; HTTP chooses the browser representation (today, form-encoded POST bodies and JSON responses, with a secure session cookie), and the journal stores length-delimited binary envelopes.

B.1.1 Conservative default HTTP projection

Without an explicit binding in `Implementation.lean`, LeanFM generates an HTTP projection and its OpenAPI description: an initiating browser-to-server message is the request, its correlated server-to-browser alternatives are the responses, and their protobuf schemas the operation schemas. The default is conservative. Only a field the implementation classifies as a bounded, URL-safe scalar may go in a path or query; bytes, repeated or nested fields, secrets and unbounded strings go in a `POST` body, encoded with the protobuf JSON mapping as `application/json`, so a generator cannot leak credentials into URLs. `GET /openapi.yaml` publishes the result: method, path, content type, request schema, every permitted response and status, authentication, correlation fields and requirement identifier. The protobuf and grammar remain the definitions.

B.2 The conversion, field by field

For a request binding `httpRequest(method, pathTemplate)`, conversion is defined in this order:

1. Match the HTTP method exactly and split the path into decoded segments.
2. For each `{field}` in the template, take the protobuf field of that name, encode it as one path segment, and reject slashes, traversal, or a value that escapes the authenticated project root.
3. For the currently implemented POST subset, encode remaining browser input fields as UTF-8 form values. A protobuf `bytes` field is opaque bytes at the protocol boundary; a form adapter must use an explicitly declared textual encoding and must never place credentials in a URL, trace, diagnostic, or returned object.
4. Recover the authenticated user from the `leanfm_session` HttpOnly, SameSite cookie. The cookie is transport authorization, not a field silently added to every protobuf message. It authorizes model calls and use of that user's provider keys; it does not decide which project may be read, because projects are shared.
5. Validate required fields and task `where_` constraints. On success, assign the envelope metadata and the oneof payload. On failure, emit the named rejection alternative when the grammar has one; otherwise return an HTTP error without pretending a successful protocol event occurred.

A response binding `httpResponse(RequestName)` reverses this over the request's finite response set: scalars become JSON values, repeated fields arrays, nested messages objects, `bytes` an explicit download or Base64 field, and secrets are redacted. Statuses follow convention (200, 201, 400, 401/403, 404, 409, 502), but the named response message, not the status, decides the grammar transition.

Several exchanges may be in flight, so every response envelope sets `cause_id` to its request's event identifier in the same session and task, and the adapter follows it to answer on the request's connection, never by adjacency, arrival order or a global "current request". A projection is valid only for a causally closed exchange: each request named by exactly one terminal response, a failure being a response too. An orphan request, a causeless or cross-session response, or two terminal responses fail validation; interim or streaming responses are modeled explicitly, with one terminal.

B.3 The running adapter

The implemented subset (static assets, diagnostics, downloads, MCP and health checks are outside the task grammar):

POST `/api/accounts`
Request: AccountCreateSubmitted. Response: AccountCreated or AccountCreateRejected. Encoding: application/x-www-form-urlencoded.

POST `/api/login`
Request: LoginSubmitted. Response: LoginAccepted or LoginRejected. Encoding: application/x-www-form-urlencoded.

GET `/api/me`
Request: authenticated session. Response: user identity JSON. Encoding: cookie only.

GET `/api/projects`
Request: ProjectListRequested. Response: ProjectListReturned. Encoding: cookie plus path.

POST `/api/projects`
Request: ProjectCreateRequested. Response: ProjectCreated. Encoding: application/x-www-form-urlencoded.

GET `/api/projects/{project_id}/conversation`
Request: project identifier. Response: conversation JSON. Encoding: path field plus cookie.

GET `/api/projects/{project_id}/tasks`
Request: project identifier. Response: task model JSON. Encoding: path field plus cookie.

GET `/api/projects/{project_id}/artifacts`
Request: ArtifactListRequested. Response: ArtifactListReturned. Encoding: path field plus cookie.

POST `/api/projects/{project_id}/chat`
Request: ChatPromptSubmitted. Response: ChatResponseRendered or ChatTurnFailed. Encoding: application/x-www-form-urlencoded.

Project routes are shared and need no login; an unknown project is 404. Model calls, key management and application launch need a session, or return 401. The server binds to loopback; exposing it needs an authenticating proxy.

B.4 The accepted HTTP projection

Rows missing from the running table are still to be implemented.

POST `/api/login`
Request: `LoginSubmitted`. Response: `LoginAccepted`, `LoginRejected`.

POST `/api/accounts`
Request: `AccountCreateSubmitted`. Response: `AccountCreated`, `AccountCreateRejected`.

GET `/api/projects/{project_id}/tasks/{task_id}`
Request: `TaskModelRequested`. Response: `TaskModelReturned`.

GET `/api/projects/{project_id}/tasks/{task_id}/sessions/{session_id}/events`
Request: `MessageSequenceRequested`. Response: `MessageSequenceReturned`.

GET `/api/projects/{project_id}/events/{event_id}`
Request: `ProtobufObjectRequested`. Response: `ProtobufObjectReturned`.

POST `/api/projects/{project_id}/research`
Request: `WebResearchRequested`. Response: `WebResearchReturned`, `WebResearchFailed`.

POST `/api/projects/{project_id}/markerboard/assist`
Request: `MarkerboardAssistRequested`. Response: `MarkerboardSyntaxHelpReturned`, `MarkerboardPatchPro`

POST `/api/projects/{project_id}/markerboard/assist/{artifact_id}/accept`
Request: `MarkerboardPatchAccepted`. Response: `MarkerboardDraftUpdated`.

POST `/api/projects/{project_id}/markerboard/assist/{artifact_id}/reject`
Request: `MarkerboardPatchRejected`. Response: `HTTP204(nopayload)`.

GET `/api/projects`
Request: `ProjectListRequested`. Response: `ProjectListReturned`.

POST `/api/projects`
Request: `ProjectCreateRequested`. Response: `ProjectCreated`.

GET `/api/projects/{project_id}`
Request: `ProjectOpenRequested`. Response: `ProjectOpened`, `ProblemInquiryPresented`.

DELETE `/api/projects/{project_id}`
Request: `ProjectDeleteRequested`. Response: `ProjectDeleted`.

POST `/api/projects/{project_id}/specification/problem`
Request: `ProblemStatementSubmitted`. Response: `AmbiguityQuestionPresented`, `RequirementsReviewPrese`
`ImplementationReviewPresented`.

POST `/api/projects/{project_id}/specification/ambiguities/{ambiguity_id}`
Request: `ClarificationSubmitted`. Response: `ProjectBookRevised`.

POST `/api/credentials`
Request: `ProviderCredentialSubmitted`. Response: `ProviderCredentialAccepted`, `ProviderCredentialRej`

POST `/api/projects/{project_id}/conversations/{conversation_id}/messages`
Request: `ChatPromptSubmitted`. Response: `ChatResponseRendered`, `ChatTurnFailed`.

POST `/api/projects/{project_id}/markerboard/mermaid`
Request: `MermaidDraftSaved`. Response: `MermaidPreviewRendered`.

POST `/api/projects/{project_id}/markerboard/media`
Request: `MediaUploaded`. Response: `MediaRendered`.

POST `/api/projects/{project_id}/artifacts/{artifact_id}/promotions`
Request: `ArtifactPromoted`. Response: `CandidateRequirementUpdated`.

GET `/api/projects/{project_id}/artifacts`
Request: `ArtifactListRequested`. Response: `ArtifactListReturned`.

GET `/api/projects/{project_id}/artifacts/{artifact_id}/revisions/{revision}`
Request: `ArtifactOpenRequested`. Response: `ArtifactOpened`.

Server-to-provider messages use the bounded in-process channels `provider.requests` and `provider.results`, keeping their message and request identities. Because key, session, user, project, conversation and request stay request-local rather than in a process-wide variable, two users can use different credentials at once.

B.5 Accepted message endpoints

Every event is a length-delimited `EventEnvelope`; payload tags 20 and up identify the message. Senders and receivers:

`LoginSubmitted`
UserBrowser sends to LeanFMServer.

`LoginAccepted`
LeanFMServer sends to UserBrowser.

`LoginRejected`
LeanFMServer sends to UserBrowser.

`AccountCreateSubmitted`
UserBrowser sends to LeanFMServer.

`ProviderKeyCheckRequested`
LeanFMServer sends to ModelProvider.

`ProviderKeyVerified`
ModelProvider sends to LeanFMServer.

`ProviderKeyRejected`
ModelProvider sends to LeanFMServer.

`AccountCreated`
LeanFMServer sends to UserBrowser.

`AccountCreateRejected`
LeanFMServer sends to UserBrowser.

`TaskModelRequested`
UserBrowser sends to LeanFMServer.

`TaskModelReturned`
LeanFMServer sends to UserBrowser.

`MessageSequenceRequested`
UserBrowser sends to LeanFMServer.

`MessageSequenceReturned`
LeanFMServer sends to UserBrowser.

`ProtobufObjectRequested`
UserBrowser sends to LeanFMServer.

`ProtobufObjectReturned`
LeanFMServer sends to UserBrowser.

`WebResearchRequested`
UserBrowser sends to LeanFMServer.

`WebResearchDispatched`
LeanFMServer sends to ModelProvider.

`WebSearchPerformed`
ModelProvider sends to LeanFMServer.

`WebPageOpened`
ModelProvider sends to LeanFMServer.

`WebResearchReturned`
LeanFMServer sends to UserBrowser.

`WebResearchFailed`
LeanFMServer sends to UserBrowser.

`MarkerboardAssistRequested`
UserBrowser sends to LeanFMServer.

`MarkerboardSyntaxHelpReturned`
LeanFMServer sends to UserBrowser.

`MarkerboardAssistDispatched`
LeanFMServer sends to ModelProvider.

`MarkerboardAssistReturned`
ModelProvider sends to LeanFMServer.

`MarkerboardPatchProposed`
LeanFMServer sends to UserBrowser.

`MarkerboardPatchAccepted`
UserBrowser sends to LeanFMServer.

`MarkerboardPatchRejected`
UserBrowser sends to LeanFMServer.

`MarkerboardDraftUpdated`
LeanFMServer sends to UserBrowser.

`ProjectListRequested`
UserBrowser sends to LeanFMServer.

`ProjectListReturned`
LeanFMServer sends to UserBrowser.

`ProjectCreateRequested`
UserBrowser sends to LeanFMServer.

`ProjectCreated`
LeanFMServer sends to UserBrowser.

`ProjectOpenRequested`
UserBrowser sends to LeanFMServer.

`ProjectOpened`
LeanFMServer sends to UserBrowser.

`ProjectDeleteRequested`
UserBrowser sends to LeanFMServer.

`ProjectDeleted`
LeanFMServer sends to UserBrowser.

`ProblemInquiryPresented`
LeanFMServer sends to UserBrowser.

`ProblemStatementSubmitted`
UserBrowser sends to LeanFMServer.

`AmbiguityQuestionPresented`
LeanFMServer sends to UserBrowser.

`ClarificationSubmitted`
UserBrowser sends to LeanFMServer.

`ProjectBookRevised`
LeanFMServer sends to UserBrowser.

`RequirementsReviewPresented`
LeanFMServer sends to UserBrowser.

`ImplementationReviewPresented`
LeanFMServer sends to UserBrowser.

`ProviderCredentialSubmitted`
UserBrowser sends to LeanFMServer.

`ProviderCredentialAccepted`
LeanFMServer sends to UserBrowser.

`ProviderCredentialRejected`
LeanFMServer sends to UserBrowser.

`ChatPromptSubmitted`
UserBrowser sends to LeanFMServer.

`ModelRequestIssued`
LeanFMServer sends to ModelProvider.

`ModelResponseReturned`
ModelProvider sends to LeanFMServer.

`ChatResponseRendered`
LeanFMServer sends to UserBrowser.

`ChatTurnFailed`
LeanFMServer sends to UserBrowser.

`ConversationCheckpointRequested`
UserBrowser sends to LeanFMServer.

`ConversationCheckpointBlocked`
LeanFMServer sends to UserBrowser.

`ConversationCheckpointCommitted`
LeanFMServer sends to UserBrowser.

`ConversationHistoryCleared`
LeanFMServer sends to UserBrowser.

`MermaidDraftSaved`
UserBrowser sends to LeanFMServer.

`MermaidPreviewRendered`
LeanFMServer sends to UserBrowser.

`MediaUploaded`
UserBrowser sends to LeanFMServer.

`MediaRendered`
LeanFMServer sends to UserBrowser.

`ArtifactPromoted`
UserBrowser sends to LeanFMServer.

`CandidateRequirementUpdated`
LeanFMServer sends to UserBrowser.

`ArtifactListRequested`
UserBrowser sends to LeanFMServer.

`ArtifactListReturned`
LeanFMServer sends to UserBrowser.

`ArtifactOpenRequested`
UserBrowser sends to LeanFMServer.

`ArtifactOpened`
LeanFMServer sends to UserBrowser.

B.5.1 Canonical byte schema

This schema is normative: a reconstruction reproduces its tags, cardinalities, enum values and names exactly.

```
syntax = "proto3";

package leanfm.chatui;

message EventEnvelope {
  string event_id = 1;
  repeated string prior_ids = 2;
  string session_id = 3;
  string task_id = 4;
  uint64 time_at_ms = 5;
  string src = 6;
  string dst = 7;
  // Direct causal parent for any event, including nonterminal and intermediate
  // events. Empty for a causal root. Unlike prior_ids, this does not constrain
  // otherwise legal concurrent interleavings. Renderers may use the causal
  // subtree as nested interaction boundaries.
  optional string cause_id = 8;

  // Former approval messages; review and acceptance now happen in git.
  reserved 81, 83, 84;

  oneof payload {
    LoginSubmitted login_submitted = 20;
    LoginAccepted login_accepted = 21;
    LoginRejected login_rejected = 22;
    ProjectListRequested project_list_requested = 23;
    ProjectListReturned project_list_returned = 24;
    ProjectCreateRequested project_create_requested = 25;
    ProjectCreated project_created = 26;
    ProjectOpenRequested project_open_requested = 27;
    ProjectOpened project_opened = 28;
    ProjectDeleteRequested project_delete_requested = 29;
    ProjectDeleted project_deleted = 30;
    ProviderCredentialSubmitted provider_credential_submitted = 31;
    ProviderCredentialAccepted provider_credential_accepted = 32;
    ProviderCredentialRejected provider_credential_rejected = 33;
    ChatPromptSubmitted chat_prompt_submitted = 34;
    ModelRequestIssued model_request_issued = 35;
    ModelResponseReturned model_response_returned = 36;
    ChatResponseRendered chat_response_rendered = 37;
    ChatTurnFailed chat_turn_failed = 38;
    MermaidDraftSaved mermaid_draft_saved = 39;
    MermaidPreviewRendered mermaid_preview_rendered = 40;
    MediaUploaded media_uploaded = 41;
    MediaRendered media_rendered = 42;
    ArtifactPromoted artifact_promoted = 43;
    CandidateRequirementUpdated candidate_requirement_updated = 44;
    ArtifactListRequested artifact_list_requested = 45;
    ArtifactListReturned artifact_list_returned = 46;
    ArtifactOpenRequested artifact_open_requested = 47;
    ArtifactOpened artifact_opened = 48;
    AccountCreateSubmitted account_create_submitted = 49;
    ProviderKeyCheckRequested provider_key_check_requested = 50;
    ProviderKeyVerified provider_key_verified = 51;
    ProviderKeyRejected provider_key_rejected = 52;
    AccountCreated account_created = 53;
    AccountCreateRejected account_create_rejected = 54;
    TaskModelRequested task_model_requested = 55;
    TaskModelReturned task_model_returned = 56;
    MessageSequenceRequested message_sequence_requested = 57;
    MessageSequenceReturned message_sequence_returned = 58;
    ProtobufObjectRequested protobuf_object_requested = 59;
    ProtobufObjectReturned protobuf_object_returned = 60;
    WebResearchRequested web_research_requested = 61;
    WebSearchPerformed web_search_performed = 62;
    WebPageOpened web_page_opened = 63;
    WebResearchReturned web_research_returned = 64;
    WebResearchFailed web_research_failed = 65;
    WebResearchDispatched web_research_dispatched = 66;
    MarkerboardAssistRequested markerboard_assist_requested = 67;
    MarkerboardSyntaxHelpReturned markerboard_syntax_help_returned = 68;
    MarkerboardAssistDispatched markerboard_assist_dispatched = 69;
    MarkerboardAssistReturned markerboard_assist_returned = 70;
    MarkerboardPatchProposed markerboard_patch_proposed = 71;
    MarkerboardPatchAccepted markerboard_patch_accepted = 72;
```

```
    MarkerboardPatchRejected markerboard_patch_rejected = 73;
    MarkerboardDraftUpdated markerboard_draft_updated = 74;
    ProblemInquiryPresented problem_inquiry_presented = 75;
    ProblemStatementSubmitted problem_statement_submitted = 76;
    AmbiguityQuestionPresented ambiguity_question_presented = 77;
    ClarificationSubmitted clarification_submitted = 78;
    ProjectBookRevised project_book_revised = 79;
    RequirementsReviewPresented requirements_review_presented = 80;
    ImplementationReviewPresented implementation_review_presented = 82;
    ConversationCheckpointRequested conversation_checkpoint_requested = 85;
    ConversationCheckpointBlocked conversation_checkpoint_blocked = 86;
    ConversationCheckpointCommitted conversation_checkpoint_committed = 87;
    ConversationHistoryCleared conversation_history_cleared = 88;
  }
}

enum ContentKind {
  CONTENT_KIND_UNSPECIFIED = 0;
  MARKDOWN = 1;
  LATEX = 2;
  MERMAID = 3;
  GRAPHVIZ_DOT = 4;
  IMAGE = 5;
  VIDEO = 6;
  CODE = 7;
  FILE = 8;
  LEANFM_VIEW = 9;
  URL_CITATION = 10;
}

message ChatContentPart {
  string part_id = 1;
  ContentKind kind = 2;
  string text = 3;
  string artifact_id = 4;
  uint64 artifact_revision = 5;
  string media_type = 6;
  bytes inline_content = 7;
  string alt_text = 8;
  string url = 9;
  string title = 10;
}

message LoginSubmitted {
  string username = 1;
  bytes credential = 2;
  bytes openai_api_key = 3;
  bytes anthropic_api_key = 4;
  bytes new_password = 5;
  bytes new_password_confirm = 6;
}
message LoginAccepted { string user_id = 1; string session_id = 2; }
message LoginRejected { string reason = 1; }

message ProblemInquiryPresented { string project_id = 1; string question = 2; }
message ProblemStatementSubmitted { string project_id = 1; string statement = 2; }
message AmbiguityQuestionPresented { string project_id = 1; string ambiguity_id = 2; string question = 3; repeated string affected_artifacts
       = 4; }
message ClarificationSubmitted { string project_id = 1; string ambiguity_id = 2; string answer = 3; }
message ProjectBookRevised { string project_id = 1; string book_revision = 2; repeated string decision_ids = 3; string light_pdf = 4; string
       dark_pdf = 5; }
message RequirementsReviewPresented { string project_id = 1; string requirements_revision = 2; repeated string derived_artifact_ids = 3;
      repeated string open_ambiguity_ids = 4; }
message ImplementationReviewPresented { string project_id = 1; string requirements_revision = 2; string implementation_revision = 3; string
      traceability_report = 4; }

message AccountCreateSubmitted {
  string username = 1;
  bytes login_credential = 2;
  string provider = 3;
  bytes api_key = 4;
  bytes openai_api_key = 5;
  bytes anthropic_api_key = 6;
}
message ProviderKeyCheckRequested { string provider = 1; string verification_id = 2; }
message ProviderKeyVerified { string verification_id = 1; string provider_account = 2; }
message ProviderKeyRejected { string verification_id = 1; string reason = 2; }
message AccountCreated { string user_id = 1; string credential_ref = 2; }
message AccountCreateRejected { string reason = 1; }

message TaskModelRequested { string project_id = 1; string task_id = 2; string requirements_revision = 3; }
message TaskModelReturned {
  string task_id = 1;
  string interaction_artifact_id = 2;
  string state_machine_artifact_id = 3;
  string message_sequence_artifact_id = 4;
  repeated string constraint_ids = 5;
}
message MessageSequenceRequested { string project_id = 1; string task_id = 2; string session_id = 3; }
```

```
message MessageSequenceReturned { string task_id = 1; string session_id = 2; repeated string event_ids = 3; }
message ProtobufObjectRequested { string project_id = 1; string event_id = 2; }
message ProtobufObjectReturned { string event_id = 1; string protobuf_type = 2; bytes encoded_object = 3; string redacted_json = 4; }
message WebResearchRequested { string project_id = 1; string conversation_id = 2; string request_id = 3; string question = 4; repeated
      string supplied_urls = 5; }
message WebResearchDispatched { string request_id = 1; string model = 2; string research_mode = 3; }
message WebSearchPerformed { string request_id = 1; repeated string queries = 2; }
message WebPageOpened { string request_id = 1; string url = 2; string title = 3; }
message WebResearchReturned { string request_id = 1; repeated ChatContentPart parts = 2; repeated string source_urls = 3; }
message WebResearchFailed { string request_id = 1; string reason = 2; }
message MarkerboardAssistRequested { string project_id = 1; string artifact_id = 2; uint64 base_revision = 3; string diagram_kind = 4;
      string source = 5; string instruction = 6; string mode = 7; string credential_ref = 8; }
message MarkerboardSyntaxHelpReturned { string artifact_id = 1; uint64 base_revision = 2; string help_markdown = 3; }
message MarkerboardAssistDispatched { string request_id = 1; string artifact_id = 2; uint64 base_revision = 3; string diagram_kind = 4;
      string source = 5; string instruction = 6; }
message MarkerboardAssistReturned { string request_id = 1; string proposed_source = 2; string explanation = 3; uint64 input_tokens = 4;
      uint64 output_tokens = 5; }
message MarkerboardPatchProposed { string artifact_id = 1; uint64 base_revision = 2; string proposed_source = 3; string explanation = 4; }
message MarkerboardPatchAccepted { string artifact_id = 1; uint64 base_revision = 2; string proposed_source = 3; }
message MarkerboardPatchRejected { string artifact_id = 1; uint64 base_revision = 2; }
message MarkerboardDraftUpdated { string artifact_id = 1; uint64 revision = 2; string source = 3; }

message ProjectListRequested { string user_id = 1; }
message ProjectListReturned { repeated string project_ids = 1; }
message ProjectCreateRequested { string display_name = 1; }
message ProjectCreated { string project_id = 1; string display_name = 2; }
message ProjectOpenRequested { string project_id = 1; }
message ProjectOpened { string project_id = 1; string current_requirements_revision = 2; }
message ProjectDeleteRequested { string project_id = 1; string confirmation = 2; }
message ProjectDeleted { string project_id = 1; }

message ProviderCredentialSubmitted {
  string provider = 1;
  bytes api_key = 2;
  string retention = 3;
}
message ProviderCredentialAccepted { string credential_ref = 1; string billing_source = 2; }
message ProviderCredentialRejected { string reason = 1; }

message ChatPromptSubmitted {
  string project_id = 1;
  string conversation_id = 2;
  string message_id = 3;
  repeated ChatContentPart parts = 4;
  repeated string attachment_ids = 5;
  string credential_ref = 6;
  string model = 7;
}
message ModelRequestIssued {
  string project_id = 1;
  string conversation_id = 2;
  string request_id = 3;
  string model = 4;
  string research_mode = 5;
}
message ModelResponseReturned {
  string request_id = 1;
  repeated ChatContentPart parts = 2;
  uint64 input_tokens = 3;
  uint64 cached_input_tokens = 4;
  uint64 output_tokens = 5;
  uint64 context_window_tokens = 6;
}
message ChatResponseRendered {
  string conversation_id = 1;
  string message_id = 2;
  repeated ChatContentPart parts = 3;
  repeated string artifact_ids = 4;
  uint64 input_tokens = 5;
  uint64 cached_input_tokens = 6;
  uint64 output_tokens = 7;
  uint64 context_window_tokens = 8;
  string usage_warning = 9;
  uint64 response_time_ms = 10;
}
message ChatTurnFailed { string request_id = 1; string reason = 2; }
message ConversationCheckpointRequested { string project_id = 1; string conversation_id = 2; string through_message_id = 3; }
message ConversationCheckpointBlocked { string project_id = 1; string conversation_id = 2; string checkpoint_id = 3; string reason_code = 4;
       repeated string blocking_artifact_ids = 5; }
message ConversationCheckpointCommitted { string project_id = 1; string conversation_id = 2; string through_message_id = 3; repeated string
      artifact_revisions = 4; string manifest_digest = 5; }
message ConversationHistoryCleared { string project_id = 1; string conversation_id = 2; string cleared_through_message_id = 3; uint64
      retained_message_count = 4; }

message MermaidDraftSaved {
  string project_id = 1;
  string artifact_id = 2;
  uint64 revision = 3;
  string diagram_kind = 4;
```

```
  string source = 5;
}
message MermaidPreviewRendered { string artifact_id = 1; uint64 revision = 2; bytes svg = 3; }
message MediaUploaded {
  string project_id = 1;
  string artifact_id = 2;
  string media_type = 3;
  bytes content = 4;
}
message MediaRendered { string artifact_id = 1; string media_type = 2; string content_url = 3; }
message ArtifactPromoted { string artifact_id = 1; uint64 revision = 2; string target_status = 3; }
message CandidateRequirementUpdated { string requirements_revision = 1; repeated string source_artifact_ids = 2; }

message ArtifactListRequested {
  string project_id = 1;
  string kind_filter = 2;
  string scenario_filter = 3;
  string status_filter = 4;
}
message ArtifactSummary {
  string artifact_id = 1;
  uint64 current_revision = 2;
  uint64 accepted_revision = 3;
  string status = 4;
  string validation = 5;
  string media_type = 6;
  string title = 7;
  string scenario_id = 8;
  string diagram_kind = 9;
  string updated_by = 10;
  uint64 updated_at_ms = 11;
}
message ArtifactListReturned { repeated ArtifactSummary artifacts = 1; }
message ArtifactOpenRequested { string artifact_id = 1; uint64 revision = 2; }
message ArtifactOpened {
  string artifact_id = 1;
  uint64 revision = 2;
  string status = 3;
  string media_type = 4;
  bytes content = 5;
  string provenance = 6;
}
```

`prior_ids` are immutable predecessor links (any successor index is derived); `session_id` separates conversations, `task_id` selects the grammar, `src`/`dst` must match the schema, and `cause_id` is the acyclic direct-cause relation of Chapter 5.

B.6 Complete LeanFM task grammar

The whole ChatUI grammar, rendered from its typed values. `A ! B : Message` reads “A sends Message to B”; `then` sequences; `choose`/`or` alternates; `optional` allows zero or one; `fork`/`join all` waits for every branch; `race`/`join first` takes the first and cancels the rest (unlike the threshold join of Chapter 2). A message the current residual does not admit is illegal even if its bytes decode.

B.7 Authenticate an individual user

Task: `authenticate_user`. Entry: `signed_out`; terminals: `authenticated, rejected`.

```
UserBrowser ! LeanFMServer : LoginSubmitted
then
choose
  LeanFMServer ! UserBrowser : LoginAccepted
or
  LeanFMServer ! UserBrowser : LoginRejected
```

B.8 Create an account using a user-supplied provider API key

Task: `create_account`. Entry: `registration`; terminals: `created, rejected`.

```
UserBrowser ! LeanFMServer : AccountCreateSubmitted
then
LeanFMServer ! ModelProvider : ProviderKeyCheckRequested
then
choose
  ModelProvider ! LeanFMServer : ProviderKeyVerified
  then
  LeanFMServer ! UserBrowser : AccountCreated
or
  ModelProvider ! LeanFMServer : ProviderKeyRejected
  then
  LeanFMServer ! UserBrowser : AccountCreateRejected
```

B.9 List, create, open, and delete owned projects

Task: `manage_projects`. Entry: `ready`; terminals: `listed, created, opened, deleted`.

```
choose
  UserBrowser ! LeanFMServer : ProjectListRequested
  then
  LeanFMServer ! UserBrowser : ProjectListReturned
or
  UserBrowser ! LeanFMServer : ProjectCreateRequested
  then
  LeanFMServer ! UserBrowser : ProjectCreated
or
  UserBrowser ! LeanFMServer : ProjectOpenRequested
  then
  LeanFMServer ! UserBrowser : ProjectOpened
or
  UserBrowser ! LeanFMServer : ProjectDeleteRequested
  then
  LeanFMServer ! UserBrowser : ProjectDeleted
```

B.10 Turn a fresh project conversation into checked requirements and implementation

Task: `develop_project_specification`. Entry: `project_loaded`; terminals: `ready_to_implement`.

```
LeanFMServer ! UserBrowser : ProblemInquiryPresented
then
UserBrowser ! LeanFMServer : ProblemStatementSubmitted
```

```
then
repeat
  choose
    empty
  or
    LeanFMServer ! UserBrowser : ProjectBookRevised
  then
  LeanFMServer ! UserBrowser : AmbiguityQuestionPresented
  then
  UserBrowser ! LeanFMServer : ClarificationSubmitted
then
choose
  empty
or
  LeanFMServer ! UserBrowser : ProjectBookRevised
then
LeanFMServer ! UserBrowser : RequirementsReviewPresented
then
LeanFMServer ! UserBrowser : ImplementationReviewPresented
```

B.11 Select user-funded or platform-funded model access

Task: `configure_provider`. Entry: `unconfigured`; terminals: `configured, rejected`.

```
UserBrowser ! LeanFMServer : ProviderCredentialSubmitted
then
choose
  LeanFMServer ! UserBrowser : ProviderCredentialAccepted
or
  LeanFMServer ! UserBrowser : ProviderCredentialRejected
```

B.12 Conduct one durable project-scoped requirements turn

Task: `argue_requirements`. Entry: `ready`; terminals: `rendered, failed`.

```
UserBrowser ! LeanFMServer : ChatPromptSubmitted
then
LeanFMServer ! ModelProvider : ModelRequestIssued
then
choose
  ModelProvider ! LeanFMServer : ModelResponseReturned
  then
  LeanFMServer ! UserBrowser : ChatResponseRendered
or
  LeanFMServer ! UserBrowser : ChatTurnFailed
```

B.13 Compact a project conversation into durable specifications before clearing it

Task: `checkpoint_conversation`. Entry: `retained`; terminals: `cleared, blocked`.

```
UserBrowser ! LeanFMServer : ConversationCheckpointRequested
then
choose
  LeanFMServer ! UserBrowser : ConversationCheckpointBlocked
or
  LeanFMServer ! UserBrowser : ConversationCheckpointCommitted
  then
  LeanFMServer ! UserBrowser : ConversationHistoryCleared
```

B.14 Sketch and promote multimodal requirement artifacts

Task: `markerboard`. Entry: `editing`; terminals: `candidate`.

```
choose
  UserBrowser ! LeanFMServer : MermaidDraftSaved
  then
  LeanFMServer ! UserBrowser : MermaidPreviewRendered
  then
  UserBrowser ! LeanFMServer : ArtifactPromoted
  then
  LeanFMServer ! UserBrowser : CandidateRequirementUpdated
or
  UserBrowser ! LeanFMServer : MediaUploaded
  then
  LeanFMServer ! UserBrowser : MediaRendered
  then
  UserBrowser ! LeanFMServer : ArtifactPromoted
  then
  LeanFMServer ! UserBrowser : CandidateRequirementUpdated
```

B.15 Inspect current, accepted, superseded, and stale artifacts

Task: `inspect_artifacts`. Entry: `closed`; terminals: `listed, opened`.

```
UserBrowser ! LeanFMServer : ArtifactListRequested
then
LeanFMServer ! UserBrowser : ArtifactListReturned
then
choose
  empty
or
```

```
  UserBrowser ! LeanFMServer : ArtifactOpenRequested
  then
  LeanFMServer ! UserBrowser : ArtifactOpened
```

B.16 Inspect one task's linked interaction, state machine, constraints, and protobuf events

Task: `inspect_task_model`. Entry: `closed`; terminals: `linked, sequence_visible, object_visible`.

```
UserBrowser ! LeanFMServer : TaskModelRequested
then
LeanFMServer ! UserBrowser : TaskModelReturned
then
choose
  empty
or
  UserBrowser ! LeanFMServer : MessageSequenceRequested
  then
  LeanFMServer ! UserBrowser : MessageSequenceReturned
  then
  choose
    empty
  or
    UserBrowser ! LeanFMServer : ProtobufObjectRequested
    then
    LeanFMServer ! UserBrowser : ProtobufObjectReturned
```

B.17 Search the public web, follow supplied HTTP URLs, and return clickable citations

Task: `research_web`. Entry: `ready`; terminals: `returned, failed`.

```
UserBrowser ! LeanFMServer : WebResearchRequested
then
LeanFMServer ! ModelProvider : WebResearchDispatched
then
choose
  ModelProvider ! LeanFMServer : WebSearchPerformed
  then
  ModelProvider ! LeanFMServer : WebPageOpened
  then
  LeanFMServer ! UserBrowser : WebResearchReturned
or
  LeanFMServer ! UserBrowser : WebResearchFailed
```

B.18 Recall diagram syntax or propose AI-generated detail for one markerboard revision

Task: `assist_markerboard`. Entry: `editing`; terminals: `helped, updated, rejected`.

```
UserBrowser ! LeanFMServer : MarkerboardAssistRequested
then
choose
  LeanFMServer ! UserBrowser : MarkerboardSyntaxHelpReturned
or
  LeanFMServer ! ModelProvider : MarkerboardAssistDispatched
  then
  ModelProvider ! LeanFMServer : MarkerboardAssistReturned
  then
  LeanFMServer ! UserBrowser : MarkerboardPatchProposed
  then
  choose
    UserBrowser ! LeanFMServer : MarkerboardPatchAccepted
    then
    LeanFMServer ! UserBrowser : MarkerboardDraftUpdated
  or
    UserBrowser ! LeanFMServer : MarkerboardPatchRejected
```

B.19 The grammar as an executable canvas

Future interface. The grammar becomes one navigable canvas: a production closes to its typed boundary or opens to its messages, states, parameters, constraints, measurements and nested productions, as presentation only. Changing a typed parameter (a capacity, population, timeout, probability) would mark every dependent stale and regenerate the affected productions, reducers, proofs, codecs and views before the revision is committed. A production that crosses HTTP attaches its binding there (method, path, protobuf types, statuses, content type, codec), with fixtures that emit and decode the exact bytes, so the canvas opens all the way down to bytes.

B.20 What is checked, and what remains

Lean elaboration checks actors, schemas, states, transitions, grammars, constraints, output provenance and mappings; the generator rejects a request with no named response and incomplete route records; and the repository checks validate the protobuf against the requirements. Not every accepted route has a handler yet, and the server does not yet journal every exchange. The accepted grammar is the contract and the running table the honest conformance statement.

Appendix C

The LeanFM Project Bootstrap Contract

This chapter defines the minimum machine-readable context that makes a LeanFM project self-hosting, which an agent applies before turning the rest of the book into checked artifacts. The bootstrap defines the language and its tools, never a hidden copy of the product's requirements or implementation.

The intended handoff is simple: send someone this PDF's URL and ask Claude, Gemini, Codex or another capable agent to read it and make the software. New projects start from built-in templates, not an example directory; the live references and read-only MCP surface supply the authoring types and commands below. An agent must not copy LeanFM's own source or prior specifications as an answer key.

C.1 Required project inputs

A copyable project root contains the following stable inputs:

`book/main.pdf`
: Central product-design argument and maintained system-prompt context.

`book/main-dark.pdf`
: Accessible rendering of the same source.

`LeanFM/`
: Versioned generic authoring types and deterministic validators. Product actors, messages, tasks and routes do not belong in this directory.

Bootstrap programs
: Whatever prints the language references and interrogation checklist, validates the conventional specification, and derives review views from its typed value. In this repository these are the `make` targets and the LeanFM server's routes and MCP tools listed below; a portable project may ship a `bootstrap/` directory instead.

`lean-toolchain` **and** `lakefile.toml`
: The exact compiler selection and module roots needed to run the bootstrap locally.

`project.json`
: Project identity, stable artifact paths, and the classification of design versus language inputs.

All project-owned state lives below the exclusive root: inputs, conversations, revisions, source, tests, views, traces, reports, logs and caches, with temporary files and atomic promotion inside it. Outputs and symlinks that escape are rejected; imports are copied in or recorded as immutable content-addressed dependencies, and shared tools and caches outside may hold no project state. A global index is reconstructible from manifests, and deleting a root removes all its state.

The root is also the complete input to a fresh agent conversation: the agent finds the paths from the manifest and repository instructions, with no transcript to paste. A root with no problem statement gets a minimal book shell and the question “What problem are you trying to solve?”, which starts requirements discovery.

C.2 Conventional generated interface

The requirements candidate lives at `spec/Requirements.lean` (this repository keeps its own in `LeanFM/LLMGenerated/`), imports `LeanFM.Artifacts`, and defines:

```
namespace LeanFMProject.Requirements
def spec : LeanFM.RequirementSpec := ...
def all : List LeanFM.GeneratedRequirement := [.requirement spec]
end LeanFMProject.Requirements
```

The byte vocabulary is `spec/Requirements.proto` and, once the requirements check, the implementation mapping is `spec/Implementation.lean`. Capitalization matters: these are Lean module paths.

C.3 Required commands and order

The project supplies deterministic equivalents of these commands. In this repository the references and checklist come from the running server (`/tools/leanfm-language/requirements-reference`, `/tools/requirements/interrogation`, or the MCP resources of the same name), and `make check` validates and renders:

```
make reference       # print the complete requirements-language reference
make interrogation   # print questions that expose missing observations
make validate        # elaborate and run LeanFM plus protobuf validation
make render          # derive interaction and residual-state review views
make check           # validate, then render
```

An agent runs `reference` and `interrogation` before authoring, and never infers constructors from prose or invents a lookalike. The validator and the renderer both consume the exported `RequirementSpec`: elaboration alone is not validation, and a separately drawn diagram is not evidence.

C.4 Three layers, reviewed in git

Work proceeds in three layers, each checked before the next: observable requirements, their realization mapping, then source, which follows the security review of Chapter 7. Review and acceptance

are git commits. LeanFM records no approvals and makes nothing wait for one; its status shows only whether each layer passes its check and is committed.

`Requirements.lean` is the contract: its multi-actor wire grammar is the conformance oracle, under which complete decoded traffic is legal or has a counterexample (an incomplete capture establishes nothing). `Requirements.proto` fixes bytes without choosing a transport. `Implementation.lean` records the choices that realize accepted behavior (queues, transports, journals, browser mechanisms), each mapped to its requirement. An answer that changes nothing observable belongs there; one that changes messages, fields or ordering revises the requirements; an answer made moot by a later requirement is removed. The book explains what is built, the generic distribution makes the language executable, and committed artifacts keep the decisions; none substitutes for another.

C.5 Construction is not a runtime dependency

The construction agent is not part of the delivered server. The user starts LeanFM, opens it in a browser, signs in, supplies a provider key, and argues requirements through LeanFM's own chat, which talks to the provider directly. A reconstruction succeeds only if it builds a standalone executable, serves the whole browser workflow, starts without a platform key, isolates concurrent users and their credentials, and passes its conformance tests with no coding agent present.

C.6 Build, runtime, and publication dependency inventory

One developer's workstation is not an implicit build input. The normative list is `toolDependencyInventory` in `LeanFM/ChatUI/Requirements.lean`; this is its review view. "Required" means the default check, server, book or media path uses it. Versions are fixed by `ui/package-lock.json`, `scripts/chatui_requirements.t` `lean-toolchain` and the `Makefile`.

Lean and Lake — required.
Lean 4.35.0 from `lean-toolchain` compiles the DSL, validators, renderers, tests, and server. Lake resolves modules and drives native linking.

C toolchain and libc headers — required.
A compiler and linker supported by Lean 4.35 link the native executables emitted by Lean.

GNU Make, Bash, and coreutils — required.
The workflows use `timeout`, `cmp`, `cp`, `tar`, `gzip`, and `sha256sum` for bounded commands and verified tool downloads.

Node and npm — required.
Node 22.12 or newer and `npm ci` type-check and bundle the React UI. The lockfile fixes React, ReactDOM, TypeScript, Vite, KaTeX, and their transitive packages.

Mermaid renderer — required for task diagrams.
The existing Mermaid 11 renderer is served locally. `assets/vendor/mermaid.json` records its resolved version, source, MIT license and SHA-256 digest, so diagrams and screenshot scenarios do not require a CDN connection.

Python packages — required.
Python 3.12 or newer with Cryptography 41–50, Pillow 10–12, and Protobuf 5–6 protects retained keys, performs protobuf round trips and reductions, and generates animated GIFs. CairoSVG 2.7–2.x and defusedxml 0.7–0.x normalize static SVG attachments without external resources; Pillow supplies bounded raster decoding. Sandboxed media generators run the system `/usr/bin/python3`, so Pillow must be installed for it too (Debian and Ubuntu: `python3-pil`).

Protocol Buffers compiler — required.
A `protoc` supporting proto3 and Python output generates the temporary byte-round-trip binding. Target-language generators are declared separately when needed.

Graphviz — required.
`dot` with PNG output rasterizes generated DOT diagrams during `make diagrams`.

TeX Live and pdfLaTeX — required.
Install the base, recommended, extra, pictures, and recommended-font collections. The source uses `lmodern`, TeX Gyre Heros, geometry, microtype, xcolor, graphicx, tagpdf, AMS math, mathtools, booktabs, longtable, enumitem, listings, natbib, imakeidx, TikZ, pgfplots, hyperref, and cleveref.

Go — required.
Go 1.23.12, installed under `.toolchains/` by `make go-toolchain` with its archive hash checked, compiles and tests the Go projects and generated applications.

Bubblewrap — required on the Linux server.
`bwrap` with user, process, network, and mount namespaces isolates executable Lean checks, project commands, persistent applications, and project-local media generators. Ubuntu/Debian supplies it in the `bubblewrap` package; Ubuntu 24.04 also needs `kernel.apparmor_restrict_unprivileged_` `make sandbox-check` verifies the host before the full suite; missing or denied support fails explicitly and never allows unsandboxed execution.

FFmpeg — required for shareable video.
FFmpeg with `libx264` H.264 and AAC encoders converts GIF/WebM to validated MP4, adds fast-start metadata, and muxes optional narration.

curl, ripgrep, and iproute2 — required for development checks.
`curl`, `rg`, and `ss` exercise HTTP contracts, select validation evidence, and identify the local server.

Java and TLA+ tools — optional.
A Java runtime and the checksummed `tla2tools.jar` 1.7.4 run the separate `tla-check` example.

Manim and Piper — optional.
A project-local video Python environment and selected voice model render explanatory videos and synthetic narration.

Target compilers and SDKs — conditional.
The accepted `ImplementationSpec` inventories every compiler, code generator, SDK, runtime, native library, and version bound before source generation.

An agent can reconcile this list with the host, reporting each item satisfied, missing, incompatible or optional; installing software is an explicit action, never a side effect of inspection. A tool enters the product contract only through this inventory and the implementation mapping (DuckDB, for instance, is not used and so is not listed).

C.7 Local and server-hosted references

The local commands are authoritative when no server runs. A running server also exposes the language references, the interrogation checklist and read-only MCP context without login; these hold language documentation, not project or account data.

Selected Books and Further Reading

These annotations identify the role each work plays in this book; they are not reviews of the books.

Richards and Hamburger, *Logic and Language Models for Computer Science*.
A source of the author's background in formal languages and logic, not the originators of those methods [18].

Gensler, *Introduction to Logic*.
Background for symbolization, validity, proof, quantification, and modal reasoning [7].

Ryan et al., *The Modelling and Analysis of Security Protocols*.
The CSP treatment of protocol analysis behind the approach of the protocol exercises, whose examples are the author's own [19].

Honda; Honda, Vasconcelos, and Kubo; Honda, Yoshida, and Carbone.
The origin of binary and then multiparty session types, the backbone of the method [9–11].

Yoshida, Hu, Neykova, and Ng, *The Scribble Protocol Language*.
The source of the `from`/`to`/`choice` surface style used for global types in this book (LeanFM drops Scribble's `at`: the role that decides is the one that starts every branch) [22].

Deniélou and Yoshida; Brand and Zafiropulo.
The link between session types and communicating finite-state machines, and why unbounded queues make verification undecidable, which motivates LeanFM's declared capacities [3, 5].

Barwell, Scalas, Yoshida, and Zhou.
One line of work that adds failure to session types, here crash-stop roles; LeanFM instead declares failure per channel [1].

Kröger and Merz, *Temporal Logic and State Systems*.
The author's principal source for temporal-logic background [12].

Lamport, *Specifying Systems*.
A reference point for the book's comparison with TLA+ state and action specifications [13].

Gunther, *Guerrilla Capacity Planning*.
The cited background for the Universal Scalability Law and capacity planning [8].

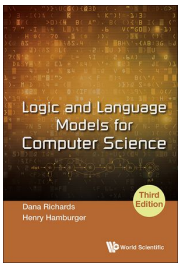
Menascé, Almeida, and Dowdy, *Performance by Design*.
Background for the queueing and capacity-planning discussion [15].

Mancosu, Galvan, and Zach, *An Introduction to Proof Theory*.
Further reading on natural deduction, sequent calculus, normalization, and cut elimination [14].

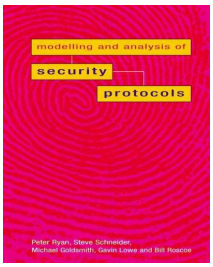
Pavlovic, *Programs as Diagrams*.
Related further reading; the current book does not rely on a specific claim from it [17].

Book-cover snapshots

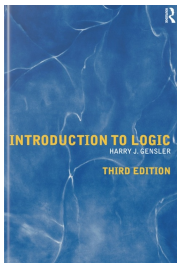
These cover snapshots identify the referenced editions.



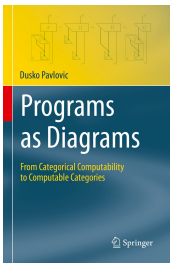
Logic and Language Models for Computer Science
Dana Richards and Henry Hamburger [18]



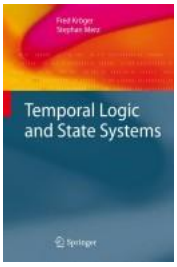
The Modelling and Analysis of Security Protocols
Peter Ryan, Steve Schneider, Michael Goldsmith, Gavin Lowe, and Bill Roscoe [19]



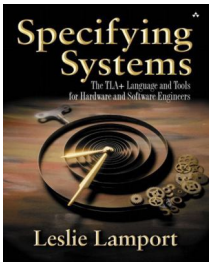
Introduction to Logic
Harry J. Gensler [7]



Programs as Diagrams
Dusko Pavlovic [17]



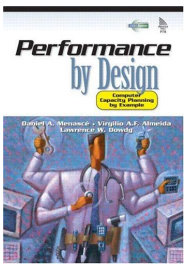
Temporal Logic and State Systems
Fred Kröger and Stephan Merz [12]



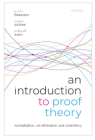
Specifying Systems
Leslie Lamport [13]



Guerrilla Capacity Planning
Neil J. Gunther [8]



Performance by Design
Daniel A. Menascé, Virgilio A. F. Almeida, and Lawrence W. Dowdy [15]



An Introduction to Proof Theory
Paolo Mancosu, Sergio Galvan, and Richard Zach [14]

Cover-image sources, accessed September 25 and 30, 2026: World Scientific edition via Kobo for Richards and Hamburger; Open Library for Ryan et al.; Apple Books for Gensler; and Kobo's Springer edition for Pavlovic. The Kröger and Merz cover snapshot is from Open Library. The Lamport and Menascé–Almeida–Dowdy snapshots are from Open Library; the Gunther snapshot is from Springer Nature; and the Mancosu–Galvan–Zach snapshot is from Google Books. Cover artwork remains the property of its respective publisher.

Bibliography

[1] Adam D. Barwell, Alceste Scalas, Nobuko Yoshida, and Fangyi Zhou. Generalised multiparty session types with crash-stop failures. In *33rd International Conference on Concurrency Theory (CONCUR 2022)*, volume 243 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 35:1–35:25, 2022. doi: 10.4230/LIPIcs.CONCUR.2022.35.

[2] Robert Braden. Requirements for internet hosts – application and support. RFC 1123, 1989. Section 4.2.3.1 describes and fixes the Sorcerer's Apprentice Syndrome.

[3] Daniel Brand and Pitro Zafiropulo. On communicating finite-state machines. *Journal of the ACM*, 30(2):323–342, 1983. doi: 10.1145/322374.322380.

[4] Edmund M. Clarke, E. Allen Emerson, and A. Prasad Sistla. Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM Transactions on Programming Languages and Systems*, 8(2):244–263, 1986. doi: 10.1145/5397.5399.

[5] Pierre-Malo Deniélou and Nobuko Yoshida. Multiparty session types meet communicating automata. In *Programming Languages and Systems (ESOP 2012)*, volume 7211 of *Lecture Notes in Computer Science*, pages 194–213. Springer, 2012. doi: 10.1007/978-3-642-28869-2_10.

[6] Robert Fielding and LeanFM contributors. LeanFM: Grammar-based formal methods in Lean. GitHub repository, 2026. URL `https://github.com/rfielding/leanFM`. Accessed 2026-09-21.

[7] Harry J. Gensler. *Introduction to Logic*. Routledge, 3 edition, 2017. ISBN 978-1-138-91058-4. doi: 10.4324/9781315693361.

[8] Neil J. Gunther. *Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services*. Springer, Berlin, Heidelberg, 2007. ISBN 978-3-540-31010-5. doi: 10.1007/978-3-540-31010-5.

[9] Kohei Honda. Types for dyadic interaction. In *CONCUR '93: 4th International Conference on Concurrency Theory*, volume 715 of *Lecture Notes in Computer Science*, pages 509–523. Springer, 1993. doi: 10.1007/3-540-57208-2_35.

[10] Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. Language primitives and type discipline for structured communication-based programming. In *Programming Languages and Systems (ESOP 1998)*, volume 1381 of *Lecture Notes in Computer Science*, pages 122–138. Springer, 1998. doi: 10.1007/BFb0053567.

[11] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2008)*, pages 273–284. ACM, 2008. doi: 10.1145/1328438.1328472.

[12] Fred Kröger and Stephan Merz. *Temporal Logic and State Systems*. Texts in Theoretical Computer Science. An EATCS Series. Springer, Berlin, Heidelberg, 2008. ISBN 978-3-540-68635-4. doi: 10.1007/978-3-540-68635-4.

[13] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. ISBN 978-0-321-14306-8.

[14] Paolo Mancosu, Sergio Galvan, and Richard Zach. *An Introduction to Proof Theory: Normalization, Cut-Elimination, and Consistency Proofs*. Oxford University Press, Oxford, 2021. ISBN 978-0-19-289593-6. doi: 10.1093/oso/9780192895936.001.0001.

[15] Daniel A. Menascé, Virgilio A. F. Almeida, and Lawrence W. Dowdy. *Performance by Design: Computer Capacity Planning by Example*. Prentice Hall PTR, Upper Saddle River, NJ, 2004. ISBN 978-0-13-090673-1.

[16] Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. Catala: A programming language for the law. *Proceedings of the ACM on Programming Languages*, 5(ICFP), 2021. doi: 10.1145/3473582.

[17] Dusko Pavlovic. *Programs as Diagrams: From Categorical Computability to Computable Categories*. Springer Nature Switzerland, Cham, Switzerland, 2023. ISBN 978-3-031-34826-6. doi: 10.1007/978-3-031-34827-3.

[18] Dana Richards and Henry Hamburger. *Logic and Language Models for Computer Science*. World Scientific, 3 edition, 2017. ISBN 978-981-322-920-4. doi: 10.1142/10507.

[19] Peter Y. A. Ryan, Steve A. Schneider, Michael H. Goldsmith, Gavin Lowe, and A. W. Roscoe. *The Modelling and Analysis of Security Protocols: The CSP Approach*. Addison-Wesley, Harlow, England, 2001. ISBN 978-0-201-67471-2.

[20] Karen R. Sollins. The TFTP protocol (revision 2). RFC 783, 1981.

[21] Karen R. Sollins. The TFTP protocol (revision 2). RFC 1350, 1992.

[22] Nobuko Yoshida, Raymond Hu, Rumyana Neykova, and Nicholas Ng. The Scribble protocol language. In *Trustworthy Global Computing (TGC 2013)*, volume 8358 of *Lecture Notes in Computer Science*, pages 22–41. Springer, 2014. doi: 10.1007/978-3-319-05119-2_3.

Index

agreement, 65
amount, 11
assumption, 10

bytes, 62

call, 9
cause, 18
choice
  decider, 5
  probability, 10
  race, 7
compatibility, 16
conformance, 18
CSP, 61
CTL, 15

deadline, 68
decision tree, 11
delay, 11
device pairing, 67
diagram
  generated, 23

event stream, 18
execution, 18

fairness, 40

generated artifacts, 23
global type, 7
grammar, 7

hidden state, 5
HTTP, 9
  protobuf adapter, 83

Implementation.lean, 26
intruder model, 65

knowledge
  derived from events, 52
knows, 16

LeanFM language, 2
local type, 8
Lowe's attack, 70

Markov chain, 11
MCP, 77
message type, 3
modality, 68
model checking
  cost, 16
Model Context Protocol
  consultation, 79
multiparty session type, 40

Needham–Schroeder, 70

OAuth, 42
observation period, 11, 21
on-call handoff, 62
OpenAPI, 24

physics of a channel, 38
PlaceOrder, 2
possibility, 15
probability overlay, 68
  measured, 33
profit
  derived from events, 33
projection, 7, 61
promise, 10
protobuf, 24

protocol, 2
    LeanFM web server, 83
protocol role, 61
provenance, 23

queue capacity, 39

rate, 11
realizability, 7
require, 11
Requirements.lean, 26
review, 26
reward vector, 20

sealed-bid auction, 64
secrecy, 65
secret, 16
secrets, 74
service level, 11
simulation, 21
skeleton, 23
Sorcerer's Apprentice, 38
statements, 5
structure, 3

temporal logic, 15
TFTP, 38
typestate, 9, 23

UDP, 39

view, 20

waste
    derived from events, 33
where condition, 19

XY plot, 33